

SAPIENZA
UNIVERSITÀ DI ROMA

Distributed Systems

Appunti del corso

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Corso di Laurea Magistrale in Computer Science

Docente
Prof. Alessandro Mei

Appunti di
Alessio Cavedal
GitHub: https://github.com/AlessioCavedal

Anno Accademico 2026/2027

Indice

1 Sistemi distribuiti e computazioni **1**
1.1 Il problema: valutare un predicato globale 1
1.2 Il modello: sistemi distribuiti asincroni 3
1.3 Computazioni distribuite 4

2 Stati globali, tagli e consistenza **7**
2.1 Stati globali, tagli e run 7
2.2 Monitoraggio attivo e deadlock fantasma 9
2.3 Tagli e run consistenti 10
2.4 Il reticolo degli stati globali 12

3 Osservare una computazione: orologio reale e clock logici **15**
3.1 Monitoraggio passivo e osservazioni 15
3.2 Regole di consegna e consegna FIFO 16
3.3 Orologio reale e clock condition 17
3.4 Clock logici di Lamport 18
3.5 Consegnare le notifiche in ordine di clock logico 19

4 Consegna causale e vector clock **21**
4.1 Consegna causale 21
4.2 La strong clock condition 22
4.3 Storie causali 23
4.4 Vector clock 23
4.5 Confrontare i vector clock 25
4.6 Consegna causale con i vector clock 26

5 Snapshot distribuiti **29**
5.1 Che cosa registra uno snapshot 29
5.2 Primo protocollo: un orologio reale 29
5.3 Dall'orologio reale ai marker 30
5.4 Il protocollo di Chandy–Lamport 31
5.5 Perché ogni messaggio è contato una volta 34
5.6 Proprietà dello snapshot 35

6 Guasti e commit atomico **37**
6.1 Transazioni distribuite 37
6.2 Il modello dei guasti 38
6.3 Il problema del commit atomico 39
6.4 Incertezza e blocco 40

7 Il protocollo Two-Phase Commit **42**
7.1 Il protocollo 42
7.2 Timeout e protocollo di terminazione 43
7.3 Recupero e DT log 45
7.4 Valutazione del 2PC 48

8 Consenso e Paxos **50**
8.1 Il problema del consenso 50
8.2 Decidere a maggioranza 52
8.3 Un round, passo per passo 52
8.4 Quando bisogna riprovare 54
8.5 Le regole sui round, caso per caso 59
8.6 Perché Paxos è sicuro 61
8.7 Liveness 62

9 Multi-Paxos ed elezione del leader **64**
9.1 Una sequenza di decisioni 64
9.2 Eleggere il leader in un sistema asincrono 67
9.3 Il nuovo leader 68

10 Fast Paxos **70**
10.1 Saltare il coordinatore 70
10.2 Quorum più grandi 72
10.3 Perché Fast Paxos è sicuro 76

11 Raft **78**
11.1 Un leader forte 78
11.2 Eleggere il leader 79
11.3 Replicare il log 83
11.4 Chi può diventare leader 87
11.5 Le entry dei term precedenti 88
11.6 Perché Raft è sicuro 90

Capitolo 1

Sistemi distribuiti e computazioni

Molti problemi dei sistemi distribuiti hanno la stessa forma: bisogna accorgersi che lo stato complessivo del sistema soddisfa una certa condizione e, quando succede, reagire. Per affrontarli serve prima un modello preciso di che cosa sia un sistema distribuito e di come si svolga una computazione al suo interno. Questo capitolo fissa il modello usato nei capitoli seguenti: processi che comunicano solo con messaggi, senza un orologio comune, e un ordine tra gli eventi basato soltanto su cause ed effetti.

1.1 Il problema: valutare un predicato globale

Ecco alcuni problemi tipici dei sistemi distribuiti:

- rilevare un *deadlock*, cioè un gruppo di processi bloccati, ciascuno in attesa di un altro;
- capire se una computazione distribuita è terminata;
- accorgersi che il *token* di un protocollo a gettone è andato perso;
- trovare gli oggetti che nessun processo può più raggiungere, per liberarne la memoria (*garbage collection* distribuita);
- salvare uno stato da cui ripartire dopo un guasto (*checkpointing*);
- riconfigurare un programma mentre è in esecuzione, per esempio per bilanciare il carico;
- fare *debugging* e monitoraggio di un'applicazione distribuita.

In tutti questi casi bisogna controllare una condizione sullo stato complessivo del sistema, e una condizione così si esprime con un predicato.

Un **predicato** è un'affermazione sullo stato del sistema che può essere vera o falsa, come «la computazione è finita». **Valutarlo** vuol dire stabilire se, in un certo stato, è vera o falsa. Quanto sia facile farlo dipende da quali informazioni servono. Un **predicato locale** dipende solo dallo stato di un processo: «p_1 sta aspettando una risposta» è vero o falso a seconda delle variabili di p_1, e p_1 lo può valutare da

solo. Un **predicato globale** dipende invece dagli stati di più processi insieme, e spesso anche dai messaggi in viaggio tra loro: nessun processo lo può valutare da solo, perché ognuno conosce soltanto il proprio stato.

Le condizioni dei problemi elencati sono predicati globali, che si indicano con Φ (la lettera greca *phi* maiuscola). Per esempio:

- *deadlock*: Φ = «c'è un ciclo di processi, ciascuno in attesa del successivo». p_1 sa di aspettare p_2, ma non sa se p_2 aspetta p_3 e p_3 aspetta p_1: il ciclo si vede solo mettendo insieme gli stati di tutti;
- terminazione: Φ = «tutti i processi sono inattivi e non ci sono messaggi in viaggio». Non basta che ogni processo sia inattivo, perché un processo inattivo che riceve un messaggio si rimette al lavoro;
- *token* perso: Φ = «nessun processo ha il *token* e nessun messaggio in viaggio lo trasporta».

Definizione (Global Predicate Evaluation). La **Global Predicate Evaluation** (GPE, valutazione di un predicato globale) è il problema di stabilire se lo **stato globale** del sistema, cioè l'insieme degli stati dei singoli processi, soddisfa un predicato globale Φ.

Se ne occupa un processo, indicato con p_0, che dal capitolo 2 si chiamerà monitor. GPE è un problema generale perché non dipende da quale Φ si sceglie: un metodo che lo risolve è il cuore della soluzione di tutti i problemi elencati, e per ciascuno resta da scegliere il predicato giusto e la reazione da eseguire quando è vero, per esempio interrompere un processo del ciclo o rigenerare il *token*.

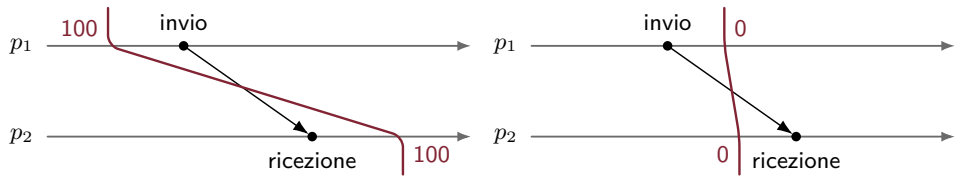
Su un solo computer valutare un predicato è facile: le variabili da cui dipende stanno nella stessa memoria e si leggono tutte nello stesso momento. In un sistema distribuito i processi non condividono memoria e comunicano solo scambiandosi messaggi. Un processo che vuole conoscere lo stato globale deve farsi mandare gli stati degli altri, e i messaggi impiegano tempi variabili e imprevedibili: ogni stato viene letto in un momento diverso, e intanto il sistema va avanti.

Esempio (un bonifico). Due processi gestiscono due conti bancari: p_1 ha 100 euro e p_2 nessuno. p_1 fa a p_2 un bonifico di 100 euro: toglie la somma dal proprio saldo e la invia con un messaggio, e p_2 la aggiunge al suo quando il messaggio arriva. Il denaro non si crea e non si distrugge, quindi i due saldi, più il denaro in viaggio, fanno sempre 100 euro. Il processo p_0 controlla che sia così e valuta Φ = «il denaro complessivo non è 100 euro», cioè «c'è un errore»: chiede il saldo a p_1 e a p_2 e somma le risposte. Le richieste arrivano ai due processi in momenti diversi, e la Figura 1.1 mostra due casi.

(a) La richiesta arriva a p_1 *prima* che invii il bonifico, e p_1 risponde 100; arriva a p_2 *dopo* che l'ha ricevuto, e anche p_2 risponde 100. La somma è 200 euro e p_0 segnala un errore che non c'è mai stato. Lo stato ricostruito è impossibile: p_2 ha ricevuto un bonifico che p_1 non ha ancora inviato.

(b) La richiesta arriva a p_1 *dopo* l'invio, e p_1 risponde 0; arriva a p_2 *prima*

della ricezione, e p_2 risponde 0. Anche qui p_0 segnala un errore, ma per un motivo diverso: lo stato è possibile, e i 100 euro sono nel messaggio in viaggio, che nessuna delle due risposte comprende. Per non sbagliare p_0 deve contare anche i messaggi in viaggio, per esempio facendosi dire da p_1 i bonifici inviati e da p_2 quelli ricevuti.



(a) 100 + 100 = 200 (b) 0 + 0 = 0, e 100 in viaggio

Figura 1.1. Il bonifico dell'esempio: il tempo scorre da sinistra a destra, e la freccia è il messaggio con i 100 euro. Le linee rosse passano per i punti in cui le richieste di p_0 arrivano a p_1 e a p_2; accanto, in rosso, i saldi comunicati.

In generale lo stato che un processo ricostruisce in questo modo può essere:

- *obsoleto*: quando le risposte arrivano, il sistema è già andato avanti, e nel caso (b) il bonifico potrebbe essere già arrivato;
- *incompleto*: alcune informazioni sono ancora in viaggio, come i 100 euro nel caso (b);
- *inconsistente*: mette insieme pezzi presi in momenti incompatibili e descrive una situazione in cui il sistema non si sarebbe mai potuto trovare, come i 200 euro nel caso (a).

Comunicare più spesso rende la vista più aggiornata e più completa, ma non basta a renderla consistente: bisogna ragionare sull'ordine in cui le informazioni vengono osservate e su che cosa contengono. Risolvere GPE vuol dire quindi costruire uno stato globale *consistente*, cioè uno in cui il sistema si sarebbe potuto trovare davvero, e valutare Φ su quello. Il capitolo 2 definisce con precisione la consistenza. I capitoli successivi mostrano due modi di costruire stati globali consistenti: in uno un processo *osserva* passivamente il sistema (capitoli 3 e 4), nell'altro lo *interroga* attivamente (capitolo 5).

1.2 Il modello: sistemi distribuiti asincroni

Definizione (sistema distribuito). Un **sistema distribuito** è un insieme di processi sequenziali $p_1, \ldots, p_n$ e una rete che realizza **canali** di comunicazione unidirezionali tra coppie di processi. Nel nostro modello:

- i canali sono **affidabili**: ogni messaggio inviato prima o poi arriva;
- i canali **non sono FIFO**: possono consegnare i messaggi in un ordine diverso da quello di invio;

- la rete è **fortemente connessa**: ogni processo può comunicare con ogni altro, eventualmente passando per processi intermedi, anche se non c'è un canale diretto tra ogni coppia.

Quando si sceglie un modello conviene fare meno ipotesi possibile. Le due grandezze su cui si può contare o no sono la velocità relativa dei processi e il ritardo dei messaggi.

Definizione (sistema asincrono). Un sistema distribuito è **asincrono** se non esiste alcun limite né alle velocità relative dei processi né ai ritardi dei messaggi. Se invece questi limiti esistono e sono noti, il sistema è **sincrono**.

In un sistema asincrono un processo non può contare su orologi sincronizzati con quelli degli altri, né ragionare in termini di un tempo reale comune: l'unico strumento di sincronizzazione è la comunicazione. Ci sono due buoni motivi per studiare proprio questo modello.

- *È il più debole.* Fa meno ipotesi di qualunque altro, quindi una soluzione che funziona in un sistema asincrono funziona in qualunque sistema distribuito, e il suo costo è un limite superiore per tutti gli altri. Non vale il contrario: un algoritmo pensato per un sistema sincrono può sfruttare i limiti di tempo, per esempio concludendo che un processo non ha inviato nulla perché entro un certo tempo non è arrivato niente, ma in un sistema asincrono questo ragionamento è sbagliato, perché un ritardo lungo è sempre possibile.
- *È realistico.* I componenti fisici sono spesso sincroni (le velocità dei processori e i ritardi sui collegamenti sono limitati), ma gli strati di software che li condividono tra molti processi, come *scheduling*, code e ritrasmissioni, rendono imprevedibili i tempi visti dalle applicazioni. Il sistema che ne risulta si descrive meglio come asincrono.

Il teorema CAP. Per i sistemi che conservano copie degli stessi dati su più nodi, il **teorema CAP** dice che non si possono garantire insieme *Consistency* (tutti i nodi vedono gli stessi dati), *Availability* (ogni richiesta a un nodo funzionante riceve risposta) e *Partition tolerance* (il sistema funziona anche se la rete si divide), ma al più due delle tre. Poiché le partizioni possono sempre capitare, in pratica si sceglie tra consistenza e disponibilità. La *consistency* di CAP riguarda le copie dei dati, e non c'entra con la consistenza dei tagli del capitolo 2.

1.3 Computazioni distribuite

Una **computazione distribuita** è l'esecuzione di un programma distribuito da parte dei processi del sistema. L'attività di ogni processo, che è sequenziale, si descrive come una sequenza di **eventi**. Un evento può essere:

- **interno**: cambia soltanto lo stato locale del processo;

- un **invio** $send(m)$: il processo mette il messaggio m sul canale verso il destinatario;
- una **ricezione** $receive(m)$: il processo preleva m dal canale in arrivo.

Ogni messaggio ha un identificatore unico (per esempio l'indice del mittente e un numero di sequenza), così ogni ricezione corrisponde esattamente a un invio; e qualunque forma di comunicazione di più alto livello, come le chiamate di procedura remota o il *broadcast*, si riduce a coppie di invii e ricezioni.

Definizione (storia locale e storia globale). La **storia locale** di p_i è la sequenza, eventualmente infinita, dei suoi eventi

$$h_i = e_i^1 \, e_i^2 \, e_i^3 \ldots$$

numerati nell'ordine in cui p_i li esegue: è l'**enumerazione canonica**. Con $h_i^k = e_i^1 \ldots e_i^k$ si indica il prefisso dei primi k eventi (h_i^0 è la sequenza vuota). La **storia globale** è l'insieme di tutti gli eventi della computazione: $H = h_1 \cup \cdots \cup h_n$.

1.3.1 Precedenza causale

La storia globale dice quali eventi sono avvenuti, ma non *quando* gli eventi di processi diversi sono avvenuti l'uno rispetto all'altro: in un sistema asincrono non c'è un tempo comune a cui riferirsi. L'unico ordine che ha senso è quello di causa ed effetto: un evento deve venire prima di un altro solo se può averlo influenzato, cioè se c'è stato un flusso di informazione dal primo al secondo.

Definizione (precedenza causale). La **precedenza causale** $\to$ (*causal precedence*) è la relazione tra gli eventi di H definita da tre regole:

1. se e ed e' sono eventi dello stesso processo ed e viene prima di e', allora $e \to e'$;
2. se $e = send(m)$ ed $e' = receive(m)$, allora $e \to e'$;
3. se $e \to e'$ ed $e' \to e''$, allora $e \to e''$ (transitività).

Due eventi sono **concorrenti**, e si scrive $e \parallel e'$, se non vale né $e \to e'$ né $e' \to e$.

$e \to e'$ si legge «e precede causalmente e'»: l'esecuzione di e, e il suo risultato, *possono* aver influenzato e'. Solo per una coppia invio–ricezione l'influenza è certa; in generale si può dire soltanto che è possibile. Lamport la chiamava *happened before*, ma il nome inganna: conta la possibilità di un'influenza, non l'ordine nel tempo reale. Due eventi concorrenti, invece, non possono essersi influenzati in nessun verso, e il loro ordine nel tempo reale è irrilevante.

Una computazione distribuita, quindi, non è una sequenza di eventi ma un insieme parzialmente ordinato (*poset*): formalmente è la coppia $(H, \to)$. L'ordine è *parziale* perché le coppie di eventi concorrenti non sono ordinate.

1.3.2 Diagrammi spazio-tempo

Una computazione si rappresenta comodamente con un **diagramma spazio-tempo** (*space-time diagram*) come quello della Figura 1.2. Ogni processo è una linea orizzontale, con il tempo che scorre da sinistra a destra; gli eventi sono punti sulla linea, e ogni messaggio è una freccia che parte dall'evento di invio e arriva all'evento di ricezione. Gli eventi interni non hanno frecce.

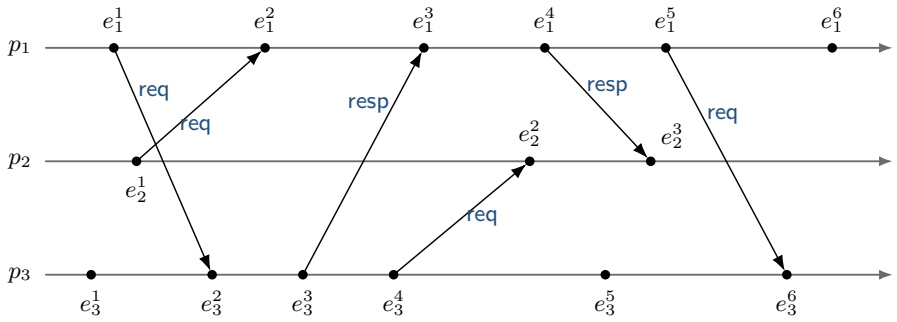


Figura 1.2. La computazione d'esempio: tre processi che si scambiano richieste (req) e risposte (resp).

Esempio (la computazione d'esempio). La computazione della Figura 1.2 tornerà in quasi tutti i capitoli. È un sistema client–server con chiamate di procedura remota: ogni processo può fare sia da client sia da server, e dopo aver inviato una richiesta (req) aspetta la risposta (resp). Per esempio p_1 invia una richiesta a p_3 (e_1^1); p_3 la riceve (e_3^2) e risponde (e_3^3), e la risposta arriva a p_1 in e_1^3. Nel frattempo p_1 ha ricevuto una richiesta da p_2 (e_1^2), a cui risponde in e_1^4. Gli eventi e_3^1, e_3^5 e e_1^6 sono interni.

Il diagramma rende facile leggere la precedenza causale: $e \to e'$ se e solo se si può andare da e a e' con un cammino che si muove verso destra lungo le linee dei processi e segue le frecce nel loro verso. Se un cammino così non c'è in nessuno dei due versi, i due eventi sono concorrenti.

Esempio (leggere la precedenza causale). Nella Figura 1.2:

- $e_2^1 \to e_3^6$, grazie al cammino $e_2^1 \to e_1^2 \to e_1^3 \to e_1^4 \to e_1^5 \to e_3^6$: la richiesta di p_2 arriva a p_1, si prosegue lungo p_1, e da e_1^5 parte la richiesta che p_3 riceve in e_3^6;
- $e_2^2 \parallel e_3^6$: dopo e_2^2 il processo p_2 non invia più messaggi, quindi da e_2^2 non si arriva a p_3; e e_3^6 è l'ultimo evento di p_3, quindi da lì non si arriva da nessuna parte;
- $e_1^1 \parallel e_2^1$: sono i primi eventi dei rispettivi processi e nessun messaggio li collega.

Capitolo 2

Stati globali, tagli e consistenza

Per valutare un predicato globale bisogna prima ricostruire uno stato globale. Questo capitolo definisce stati locali e globali e i tagli che li individuano, e mostra con un esempio come un monitor ingenuo possa ricostruire uno stato mai esistito e trarne conclusioni sbagliate. La nozione che separa gli stati sensati da quelli impossibili è la consistenza; gli stati consistenti, ordinati, formano il reticolo degli stati globali.

2.1 Stati globali, tagli e run

Indichiamo con σ_i^k lo **stato locale** di p_i subito dopo l'evento e_i^k, e con σ_i^0 il suo stato iniziale, prima di qualunque evento. Lo stato locale comprende i valori delle variabili del processo e, se servono, le sequenze dei messaggi inviati e ricevuti su ciascun canale. Lo **stato globale** è la n-upla degli stati locali, uno per processo: $\Sigma = (\sigma_1, \ldots, \sigma_n)$.

Non serve aggiungere allo stato globale lo stato dei canali, cioè i messaggi in viaggio: se gli stati locali registrano i messaggi inviati e ricevuti, i messaggi in viaggio da p_i a p_j sono quelli che p_i ha inviato e p_j non ha ancora ricevuto. Gli stati dei canali torneranno in modo esplicito con gli snapshot (capitolo 5).

> **Storia e stato.** Storia e stato descrivono la stessa computazione, ma rispondono a domande diverse. La storia dice *che cosa è successo e in che ordine*: è la sequenza degli eventi, come l'elenco delle mosse di una partita a scacchi. Lo stato dice *com'è il processo in un certo punto*: sono i valori delle sue variabili, come la posizione sulla scacchiera.
> Per esempio, p_1 gestisce un conto che parte da 0 euro: riceve da p_2 un bonifico di 50 euro (e_1^1), deposita 100 euro (e_1^2) e ne preleva 30 (e_1^3).
>
> - «Il saldo supera i 100 euro?» Lo stato σ_1^3 lo dice subito: il saldo è 120 euro. Nella storia questo numero non c'è, e per trovarlo bisogna rifare i conti dall'inizio, cioè ricostruire lo stato.
> - «Il bonifico è arrivato prima del prelievo?» La storia lo dice subito. Lo stato no, perché a 120 euro si arriva in molti modi, anche con un solo deposito.

Lo stato σ_i^k corrisponde al prefisso h_i^k dei primi k eventi solo nel senso che ne è il risultato, come dopo k mosse c'è una sola posizione: dalla storia, partendo dallo stato iniziale, si ricava lo stato, ma non il contrario.
Per questo servono tutti e due. I predicati parlano di valori e si valutano sugli stati; l'ordine si studia sugli eventi, su cui si definiscono precedenza causale, clock e consistenza dei tagli. In GPE la storia dice *dove* si può guardare, cioè quali tagli sono consistenti, e lo stato dice *che cosa* si vede lì, cioè i valori su cui valutare il predicato globale Φ.
Le sequenze dei messaggi inviati e ricevuti sono un pezzo di storia annotato nello stato. Servono perché nessun processo vede la storia: ognuno ha solo il proprio stato e i messaggi che riceve, e ciò che gli serve sapere del passato deve averlo annotato. Annotare non trasforma però lo stato nella storia: anche se p_i annotasse tutti i propri eventi, σ_i^k conterrebbe solo una copia del passato fino a quel punto, mentre la storia descrive l'intera computazione vista da fuori ed è il riferimento con cui si giudica se ciò che i processi ricostruiscono è corretto. In pratica si annota solo il riassunto che serve, come il totale inviato e ricevuto su ogni canale, o i clock dei capitoli 3 e 4.

Per indicare *quale* stato globale si considera basta dire fino a che punto è arrivato ciascun processo. È l'idea di taglio.

Definizione (taglio). Un **taglio** (*cut*) è un sottoinsieme C della storia globale formato da un prefisso della storia locale di ogni processo:

$$C = h_1^{c_1} \cup \cdots \cup h_n^{c_n}.$$

Si indica con la tupla $(c_1, \ldots, c_n)$ degli indici dell'ultimo evento incluso per ciascun processo. Gli ultimi eventi inclusi, $(e_1^{c_1}, \ldots, e_n^{c_n})$, formano la **frontiera** del taglio, e lo stato globale che gli corrisponde è $(\sigma_1^{c_1}, \ldots, \sigma_n^{c_n})$.

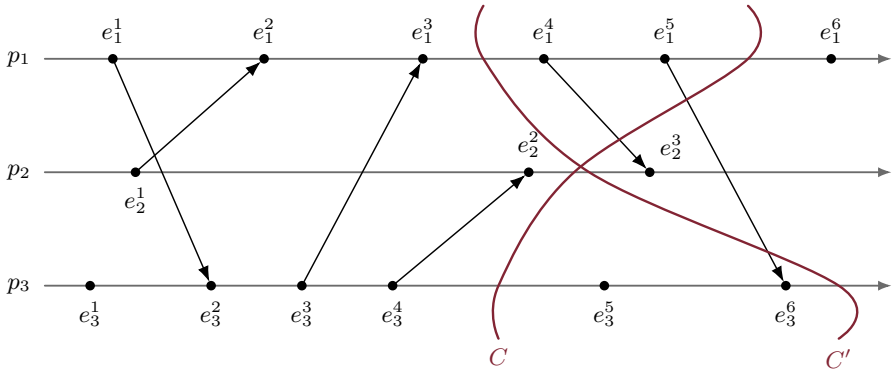


Figura 2.1. Due tagli della computazione d'esempio: $C = (5, 2, 4)$ e $C' = (3, 2, 6)$.

Un taglio non è quindi un sottoinsieme qualunque della storia globale: di ogni processo p_i prende i primi c_i eventi, senza buchi. Non può contenere, per esempio,

e_1^3 senza e_1^1 ed e_1^2.

Nel diagramma spazio-tempo un taglio è una linea che attraversa dall'alto in basso tutti i processi: gli eventi a sinistra della linea sono nel taglio, quelli a destra no. La Figura 2.1 mostra due tagli della computazione d'esempio, $C = (5, 2, 4)$ e $C' = (3, 2, 6)$.

Un taglio dice quali eventi sono avvenuti, non in che ordine. Per ragionare sulle esecuzioni serve anche un ordine totale: in un'esecuzione reale tutti gli eventi, anche di processi diversi, avvengono in un qualche ordine. Due eventi simultanei si possono ordinare in modo arbitrario, per esempio mettendo prima quello del processo con indice minore.

Definizione (run). Un **run** (esecuzione) di una computazione è un ordine totale di tutti gli eventi della storia globale che rispetta ogni storia locale: gli eventi di ciascun processo p_i compaiono nel run nello stesso ordine in cui compaiono in h_i.

Una computazione ha in generale molti run, e un run non corrisponde per forza a un'esecuzione possibile: la definizione non impedisce, per esempio, di mettere la ricezione di un messaggio prima del suo invio. A questo rimediano i run consistenti della Sezione 2.3.

2.2 Monitoraggio attivo e deadlock fantasma

Supponiamo che un solo processo, il **monitor** p_0, abbia il compito di valutare il predicato Φ. Può essere uno dei processi $p_1, \ldots, p_n$ oppure un processo esterno alla computazione, ma pur sempre parte del sistema e soggetto ai suoi ritardi. Gli eventi eseguiti per il monitoraggio non fanno parte della computazione e non cambiano la numerazione dei suoi eventi.

La strategia più semplice è attiva: p_0 chiede lo stato a tutti. Invia a ogni processo una richiesta; quando la riceve, p_i risponde con il proprio stato locale corrente; raccolte tutte le risposte, p_0 compone lo stato globale. I punti in cui le richieste arrivano ai processi definiscono un taglio, e lo stato costruito è quello di quel taglio. Il problema è che le richieste arrivano in momenti scollegati tra loro, quindi il taglio che ne risulta può essere qualunque.

Esempio (deadlock fantasma). Nella computazione d'esempio (Figura 1.2) ogni processo, dopo aver inviato una richiesta, resta bloccato finché non riceve la risposta, quindi il sistema può andare in deadlock. Per rilevarlo, ogni server tiene nel proprio stato locale l'elenco dei client da cui ha ricevuto richieste a cui non ha ancora risposto. Con queste informazioni si costruisce il **grafo delle attese** (*waits-for graph*, WFG): i nodi sono i processi, e c'è un arco da p_i a p_j se p_j ha ricevuto una richiesta da p_i e non ha ancora risposto, cioè se p_i sta aspettando p_j. Un ciclo nel grafo è un deadlock, perché ogni processo del ciclo aspetta il successivo. Il predicato da valutare è quindi Φ = «il WFG contiene un ciclo».
Supponiamo che le richieste di p_0 arrivino ai processi nei punti del taglio

$C' = (3, 2, 6)$ della Figura 2.1, e che p_0 riceva quindi σ_1^3, σ_2^2 e σ_3^6:

- in σ_3^6, p_3 ha ricevuto da p_1 la richiesta inviata in e_1^5 e non ha risposto: arco $p_1 \to p_3$;
- in σ_1^3, p_1 ha ricevuto la richiesta di p_2 (e_1^2) e non ha ancora risposto: arco $p_2 \to p_1$;
- in σ_2^2, p_2 ha ricevuto la richiesta di p_3 (e_2^2) e non ha risposto: arco $p_3 \to p_2$.

Il grafo contiene il ciclo $p_1 \to p_3 \to p_2 \to p_1$ (Figura 2.2) e p_0 dichiara un deadlock. Ma un osservatore esterno che vedesse l'intera computazione direbbe che il sistema non è mai in deadlock: p_1 risponde a p_2 (e_1^4) *prima* di inviare la nuova richiesta a p_3 (e_1^5). Il deadlock rilevato è un **deadlock fantasma** (*ghost deadlock*).

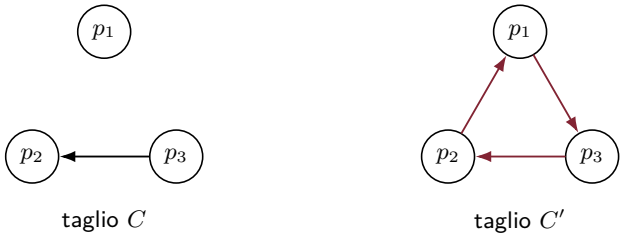


Figura 2.2. Il grafo delle attese costruito sullo stato del taglio C e su quello di C': solo il secondo contiene un ciclo.

Dov'è l'errore? Nello stato di C', p_3 risulta aver ricevuto una richiesta (e_3^6) che p_1 non ha ancora inviato (e_1^5 è fuori dal taglio): uno stato così non avrebbe mai potuto verificarsi. Se invece le richieste di p_0 arrivassero nei punti del taglio $C = (5, 2, 4)$, il grafo avrebbe il solo arco $p_3 \to p_2$ e nessun ciclo. Qui si vedono anche i ruoli di storia e stato: il grafo delle attese si costruisce dagli stati locali, ma l'errore si scopre guardando gli eventi.

2.3 Tagli e run consistenti

La precedenza causale è lo strumento giusto per distinguere i due tipi di taglio.

Definizione (taglio consistente). Un taglio C è **consistente** se, per ogni coppia di eventi e ed e',

$$(e \in C) \wedge (e' \to e) \;\Rightarrow\; e' \in C.$$

In parole: se un evento sta nel taglio, ci stanno anche tutti gli eventi che lo precedono causalmente, anche indirettamente. Un taglio consistente è «chiuso all'indietro» rispetto alla precedenza causale: non contiene un effetto senza le sue cause.

Dentro un singolo processo questa condizione è sempre soddisfatta, perché un taglio contiene prefissi delle storie locali. L'unico modo per violarla è un messaggio

ricevuto dentro il taglio ma *inviato* fuori. Nel diagramma il test è immediato: un taglio è consistente se ogni freccia che lo attraversa va da sinistra (dentro il taglio) verso destra (fuori); è inconsistente se una freccia lo attraversa da destra verso sinistra, cioè se c'è un messaggio ricevuto ma non ancora inviato. Nella Figura 2.1 il taglio C è consistente; C' no, perché la richiesta da e_1^5 a e_3^6 lo attraversa all'indietro.

Uno **stato globale consistente** è lo stato di un taglio consistente. La consistenza separa gli stati possibili da quelli impossibili.

- In uno stato inconsistente il sistema non si sarebbe mai potuto trovare: è il caso del deadlock fantasma e dei 200 euro del bonifico (Sezione 1.1).
- In uno stato consistente il sistema si sarebbe potuto trovare: un osservatore ideale, esterno al sistema, avrebbe potuto vederlo in qualche esecuzione. Non è detto, però, che ci si sia trovato davvero.

Il valore di un predicato ha quindi senso solo negli stati globali consistenti.

Esempio (uno stato possibile ma mai attraversato). Una computazione ha due soli eventi, e su p_1 ed f su p_2, senza messaggi tra loro: sono concorrenti. Supponiamo che in realtà e avvenga prima di f: l'esecuzione passa per lo stato in cui e è avvenuto ed f non ancora. Anche lo stato opposto, con f avvenuto ed e non ancora, è consistente: nessun messaggio lo vieta, e se f fosse stato un po' più veloce il sistema ci sarebbe passato. In realtà non ci è mai passato, ma nessuno dentro il sistema può accorgersene: senza un orologio comune, l'ordine reale di due eventi concorrenti non si osserva.

Nella Sezione 2.4 l'idea diventa generale: i run consistenti sono i cammini nel reticolo degli stati consistenti, e l'esecuzione reale è solo uno di essi. Lo snapshot del capitolo 5 costruirà proprio uno stato consistente che l'esecuzione reale può non aver attraversato (Sezione 5.6).

I tagli consistenti hanno un ruolo centrale. In un programma sequenziale un istante di tempo individua un punto preciso dell'esecuzione; in una computazione distribuita questo ruolo lo ha la frontiera di un taglio consistente. Anche «prima» e «dopo» vanno intesi rispetto ai tagli consistenti: un evento è prima di un taglio se sta a sinistra della sua frontiera, dopo se sta a destra.

La stessa idea si applica ai run.

Definizione (run consistente). Un run R è **consistente** se, per ogni coppia di eventi, $e \to e'$ implica che e compare prima di e' in R.

La condizione è diversa da quella dei tagli. Un taglio è un insieme, e chiede che le cause *ci siano*; un run contiene sempre tutti gli eventi, e chiede che le cause *vengano prima*. Le due cose si collegano così: un run è consistente esattamente quando, fermandolo in qualunque punto, si ottiene un taglio consistente. In pratica, siccome le storie locali sono già rispettate, basta che ogni invio compaia prima della propria ricezione.

Un run $R = e^1 e^2 \dots$ si può vedere anche come una sequenza di stati globali $\Sigma^0 \Sigma^1 \Sigma^2 \dots$, dove $\Sigma^0 = (\sigma_1^0, \dots, \sigma_n^0)$ è lo stato iniziale e ogni Σ^i si ottiene da Σ^{i-1}

eseguendo il singolo evento e^i; se il run è consistente, sono tutti consistenti. Due stati consecutivi sono legati dalla relazione *leads to* («porta a»), e Σ' è **raggiungibile** da Σ, in simboli $\Sigma \rightsquigarrow \Sigma'$, se esiste un run consistente che passa prima per Σ e poi per Σ'.

2.4 Il reticolo degli stati globali

L'insieme degli stati globali consistenti di una computazione, con la relazione *leads to*, forma un **reticolo** (*lattice*). È un modo di disegnare tutti gli stati in cui il sistema si può trovare e tutti i modi di passare dall'uno all'altro.

La notazione. Si scrive $\Sigma^{k_1 \cdots k_n}$ per lo stato globale $(\sigma_1^{k_1}, \ldots, \sigma_n^{k_n})$, quello in cui p_1 ha eseguito esattamente k_1 eventi, p_2 esattamente k_2, e così via: è lo stato del taglio $(k_1, \ldots, k_n)$. Con due processi si scrive Σ^{ab}, e le due cifre in apice sono due contatori separati, non un numero: Σ^{21} si legge «p_1 ha eseguito due eventi, p_2 uno», e Σ^{00} è lo stato iniziale. Il **livello** di uno stato è la somma $k_1 + \cdots + k_n$, cioè il numero totale di eventi eseguiti: Σ^{21} sta al livello 3.

> **Due notazioni da non confondere.** Nella Sezione 2.3 gli stati attraversati da un run sono scritti $\Sigma^0\Sigma^1\Sigma^2\ldots$: lì l'unico indice conta gli eventi eseguiti *lungo quel run*, di qualunque processo siano. In $\Sigma^{k_1 \cdots k_n}$, invece, c'è un indice per ogni processo. Per esempio, se un run comincia con $e_2^1\, e_1^1\, e_1^2$, il suo Σ^3 è Σ^{21}.

Come si costruisce. Prendiamo la computazione di due processi della Figura 2.3, a destra: p_1 ha sei eventi, p_2 cinque, e ci sono tre messaggi. Il reticolo si costruisce dall'alto.

1. In cima si mette lo stato iniziale Σ^{00}.
2. Da ogni stato si scende di un passo eseguendo un solo evento in più: il prossimo di p_1, e allora nella figura si scende a sinistra, da Σ^{ab} a $\Sigma^{a+1,b}$; oppure il prossimo di p_2, e si scende a destra, verso $\Sigma^{a,b+1}$.
3. Un passo è possibile solo se lo stato che si ottiene è consistente: un evento interno o un invio si può eseguire sempre, una ricezione solo se l'invio corrispondente è già stato eseguito.
4. Gli stati dello stesso livello stanno sulla stessa riga.

Con n processi il reticolo ha n assi, uno per processo, e già con tre processi è difficile da disegnare, ma la costruzione è la stessa.

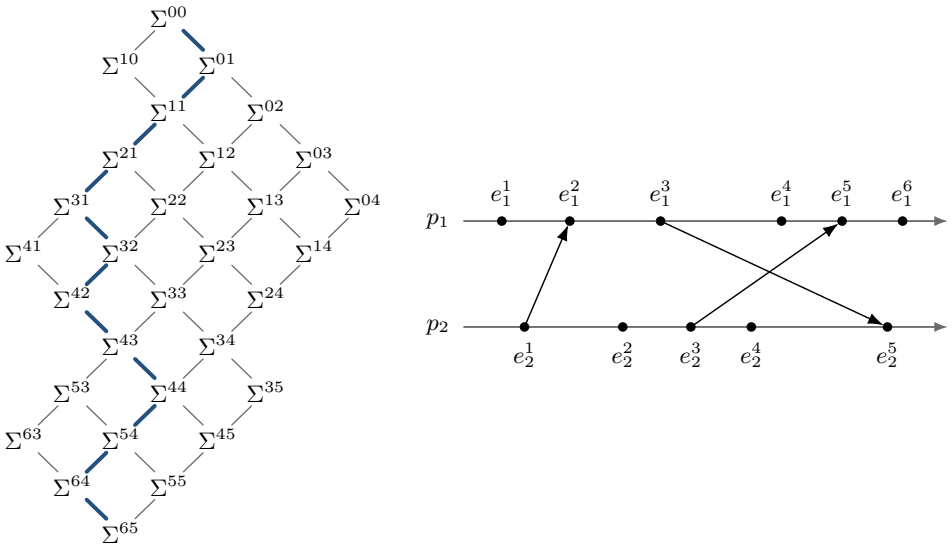


Figura 2.3. Una computazione di due processi (a destra) e il reticolo dei suoi stati globali consistenti (a sinistra). In blu un cammino, cioè un run consistente.

La Figura 2.4 mostra i primi livelli, con l'evento eseguito a ogni passo. Da Σ^{00} si può eseguire e_1^1 oppure e_2^1. Da Σ^{10}, invece, non si può scendere a sinistra: il prossimo evento di p_1 è e_1^2, che riceve il messaggio inviato in e_2^1, non ancora eseguito. Lo stato Σ^{20}, quindi, non c'è. Da Σ^{11} lo stesso evento e_1^2 è possibile, perché ora e_2^1 è avvenuto, e si arriva a Σ^{21}.

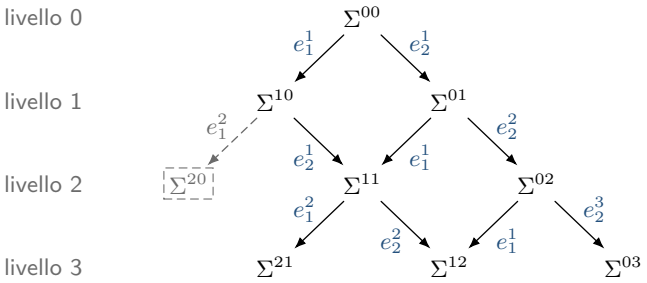


Figura 2.4. I primi livelli del reticolo della Figura 2.3, con l'evento eseguito a ogni passo. Da Σ^{10} non si può eseguire e_1^2, che riceve un messaggio non ancora inviato: Σ^{20} non è consistente.

Quali stati mancano lo dicono i messaggi, perché ognuno impone che la sua ricezione non avvenga prima dell'invio. Nella computazione della figura i vincoli su Σ^{ab} sono tre:

- per il messaggio da e_2^1 a e_1^2: se $a \geq 2$, allora $b \geq 1$;
- per il messaggio da e_2^3 a e_1^5: se $a \geq 5$, allora $b \geq 3$;
- per il messaggio da e_1^3 a e_2^5: se $b \geq 5$, allora $a \geq 3$.

Nel reticolo completo della Figura 2.3 mancano quindi gli stati con $a \geq 2$ e $b = 0$, come Σ^{20}; quelli con $a \geq 5$ e $b \leq 2$, come Σ^{51}; e quelli con $b = 5$ e $a \leq 2$, come Σ^{05}. Tutti gli altri ci sono.

Cammini e run. Un **cammino** nel reticolo è una sequenza di stati in cui ogni stato ha un evento in più del precedente: a ogni passo si sceglie quale processo avanza. Ogni cammino dallo stato iniziale a quello finale corrisponde a un run consistente, e ogni run consistente è un cammino. Il cammino evidenziato in blu nella Figura 2.3, per esempio, passa per gli stati

$$\Sigma^{00}\,\Sigma^{01}\,\Sigma^{11}\,\Sigma^{21}\,\Sigma^{31}\,\Sigma^{32}\,\Sigma^{42}\,\Sigma^{43}\,\Sigma^{44}\,\Sigma^{54}\,\Sigma^{64}\,\Sigma^{65}$$

e corrisponde al run

$$e_2^1\,e_1^1\,e_1^2\,e_1^3\,e_2^2\,e_1^4\,e_2^3\,e_2^4\,e_1^5\,e_1^6\,e_2^5.$$

Ogni passo a sinistra è un evento di p_1, ogni passo a destra un evento di p_2. Cammini diversi sono ordini diversi in cui si possono eseguire gli eventi concorrenti, e descrivono tutti la stessa computazione. L'esecuzione reale è uno di questi cammini, ma dall'interno del sistema non si può sapere quale (Sezione 2.3).

Raggiungibilità. Nel reticolo, $\Sigma \rightsquigarrow \Sigma'$ vuol dire che da Σ si può scendere fino a Σ' seguendo gli archi. Succede esattamente quando in Σ' ogni processo ha eseguito almeno tanti eventi quanti in Σ. Per esempio $\Sigma^{21} \rightsquigarrow \Sigma^{23}$, passando per Σ^{22}; invece tra Σ^{31} e Σ^{23} nessuno dei due è raggiungibile dall'altro, perché il primo ha più eventi di p_1 e il secondo più eventi di p_2.

Tre proprietà dei tagli consistenti aiutano a ragionare sul reticolo.

- *Intersezione e unione di due tagli consistenti sono tagli consistenti.* Se un evento sta in entrambi i tagli, le sue cause stanno in entrambi; se sta in almeno uno dei due, le sue cause stanno almeno in quello. È questo a fare dell'insieme un reticolo: per due stati consistenti esistono sempre il più grande stato da cui entrambi sono raggiungibili (quello dell'intersezione, il minimo componente per componente) e il più piccolo stato raggiungibile da entrambi (quello dell'unione, il massimo). Per Σ^{31} e Σ^{23} sono Σ^{21} e Σ^{33}.

- *Ogni stato globale consistente è raggiungibile dallo stato iniziale con un run consistente.* Basta eseguire gli eventi del taglio uno alla volta, scegliendo ogni volta un evento le cui cause sono già state eseguite, e poi allo stesso modo tutti gli altri. Un evento così esiste sempre: tra quelli del taglio ancora da eseguire ce n'è uno senza cause tra gli altri rimasti, perché la precedenza causale non ha cicli, e le sue cause stanno nel taglio, perché il taglio è consistente.

- *Se $C \subseteq C'$ sono due tagli consistenti, lo stato di C' è raggiungibile da quello di C.* Si prende un run consistente che arriva allo stato di C e lo si prolunga con gli eventi di $C' \setminus C$, sempre in un ordine che rispetta la precedenza causale.

L'ultima proprietà tornerà utile con gli snapshot (Sezione 5.6).

Capitolo 3

Osservare una computazione: orologio reale e clock logici

Nel capitolo precedente il monitor interrogava attivamente i processi e otteneva un taglio qualunque. Qui si prova la strategia opposta: il monitor non chiede nulla, sono i processi a informarlo di ogni evento, e il suo compito diventa mettere in ordine le notifiche in modo che la sequenza osservata sia consistente. Si parte da ipotesi forti, un orologio globale e ritardi limitati, e le si toglie una alla volta. Lo strumento centrale del capitolo sono i clock logici di Lamport.

3.1 Monitoraggio passivo e osservazioni

Nella strategia passiva il monitor p_0 non invia messaggi. I processi della computazione vengono modificati di poco: ogni volta che eseguono un evento, lo notificano a p_0 con un messaggio che lo descrive. L'invio della notifica non è un evento nuovo, ma coincide con l'evento notificato: così, come nella strategia attiva (Sezione 2.2), il monitoraggio non aggiunge eventi alla computazione e non ne cambia la numerazione. In pratica basta notificare gli eventi **rilevanti** per Φ, cioè quelli che possono cambiarne il valore: per il deadlock della computazione d'esempio sono gli invii e le ricezioni di richieste e risposte, gli unici che modificano il grafo delle attese. Il monitor costruisce così un'**osservazione** della computazione: la sequenza degli eventi nell'ordine in cui arrivano le notifiche.

Le osservazioni hanno due caratteristiche importanti.

- Poiché i ritardi variano, la stessa esecuzione può produrre osservazioni diverse su monitor diversi. È il cosiddetto **effetto relativistico** del calcolo distribuito: non esiste un punto di vista privilegiato.
- Un'osservazione può corrispondere a un run consistente, a un run inconsistente o a nessun run, se le notifiche di uno stesso processo arrivano in disordine. Si dice **consistente** un'osservazione che corrisponde a un run consistente.

Esempio (tre osservazioni della stessa esecuzione). Supponiamo che la computazione d'esempio (Figura 1.2) venga notificata a p_0. Con canali non FIFO

qualunque ordine di arrivo delle notifiche è possibile.

- Se la notifica di e_3^4 arriva prima di quella di e_3^3, la sequenza osservata non rispetta nemmeno la storia locale di p_3: non è un run.
- Se le notifiche di e_1^4 ed e_1^5 tardano mentre quella di e_3^6 arriva presto, p_0 vede p_3 ricevere una richiesta che p_1 non ha ancora inviato: l'osservazione è un run inconsistente. Se a quel punto p_0 valuta Φ, si trova proprio nello stato del taglio C', cioè $(\sigma_1^3, \sigma_2^2, \sigma_3^6)$, e ritrova il deadlock fantasma.
- Se le notifiche arrivano in un ordine che rispetta la precedenza causale, l'osservazione è consistente, e tutti gli stati che p_0 attraversa sono consistenti: per esempio quello del taglio C.

3.2 Regole di consegna e consegna FIFO

Per rimettere in ordine i messaggi, chi li riceve non li passa subito all'applicazione, cioè al proprio programma, ma decide quando farlo seguendo una **regola di consegna** (*delivery rule*). Ogni processo che riceve messaggi ha quindi due livelli, come la portineria e la scrivania di un ufficio: il *livello di consegna*, che riceve i messaggi dal canale e li tiene in attesa, e l'*applicazione*, che li usa. Le due operazioni sono *receive*, con cui il messaggio arriva dal canale, e *deliver*, con cui viene consegnato all'applicazione: avvengono entrambe nello stesso processo, e consegnare non vuol dire spedire a qualcun altro. La ricezione resta nascosta dentro la regola e non compare nella storia locale: per l'applicazione conta solo la consegna. Per il monitor (Figura 3.1) le notifiche arrivano al livello di consegna di p_0, che le passa al programma di p_0 nell'ordine stabilito dalla regola. D'ora in poi, quindi, l'osservazione è la sequenza delle notifiche *consegnate*, non di quelle ricevute.

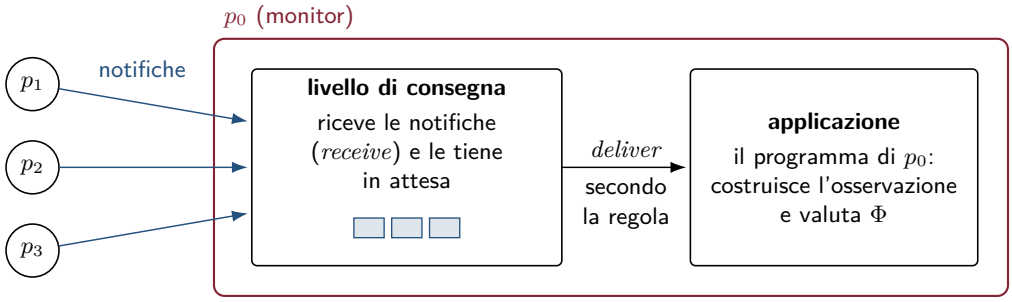


Figura 3.1. Il monitor nella strategia passiva. Le notifiche (in blu) arrivano al livello di consegna di p_0 e aspettano; la regola di consegna le passa, nell'ordine giusto, all'applicazione di p_0, che costruisce l'osservazione e valuta Φ. I due livelli fanno parte dello stesso processo.

Definizione (consegna FIFO). La comunicazione da p_i a p_j rispetta la **consegna FIFO** (*First In, First Out*) se, per ogni coppia di messaggi m ed m',

$$send_i(m) \to send_i(m') \;\Rightarrow\; deliver_j(m) \to deliver_j(m'),$$

dove il pedice indica il processo che esegue l'evento.

In parole: un messaggio non può sorpassare un messaggio inviato prima dallo stesso mittente. La consegna FIFO si realizza facilmente anche su canali che non la garantiscono: il mittente numera i messaggi che invia a ciascun destinatario, e il destinatario li consegna in ordine di numero, tenendo da parte quelli arrivati in anticipo. Se per esempio arrivano prima i messaggi numero 2 e 3, il destinatario li trattiene finché non arriva l'1, e poi consegna 1, 2 e 3.

Con canali FIFO tra i processi e il monitor, le notifiche di ogni processo arrivano nell'ordine giusto e ogni osservazione è un run: il primo caso dell'esempio non può più capitare. La consistenza però non è garantita: il secondo caso riguarda notifiche di processi *diversi*, p_1 e p_3, che la consegna FIFO non ordina tra loro. Serve un meccanismo che ordini le notifiche secondo la precedenza causale.

3.3 Orologio reale e clock condition

Supponiamo che tutti i processi possano leggere un orologio reale globale e che ogni messaggio arrivi entro un ritardo massimo noto δ. Non è più un sistema asincrono, ma è un buon punto di partenza. Sia $RC(e)$ il valore dell'orologio quando viene eseguito l'evento e: ogni notifica porta con sé come **timestamp** (marca temporale) il valore $RC(e)$ dell'evento notificato, e il monitor segue questa regola.

Definizione (regola DR1). Al tempo t, consegna tutti i messaggi ricevuti con timestamp fino a $t - \delta$, in ordine di timestamp crescente.

Perché funziona? Un messaggio con timestamp non superiore a $t - \delta$ è stato inviato almeno δ prima di t, quindi al tempo t è sicuramente già arrivato. Per esempio, con $\delta = 3$, al tempo 10 il monitor consegna i messaggi con timestamp fino a 7: uno con timestamp 7 è partito al tempo 7 ed è arrivato entro il 10, quindi nessun messaggio con timestamp fino a 7 può ancora arrivare. Quelli con timestamp 8, 9 e 10 aspettano: un messaggio con timestamp 8 può arrivare fino al tempo 11, e se il monitor avesse già consegnato un 9 l'ordine sarebbe sbagliato. L'osservazione ordina quindi gli eventi per timestamp crescente: e viene osservato prima di e' se e solo se $RC(e) < RC(e')$ (a parità di timestamp decide l'indice del processo). Allora l'osservazione è consistente se e solo se i timestamp rispettano la precedenza causale.

Definizione (clock condition). Una funzione T che associa un valore a ogni evento soddisfa la **clock condition** se

$$e \to e' \;\Rightarrow\; T(e) < T(e').$$

L'orologio reale la soddisfa senza fatica, perché una causa avviene sempre prima del suo effetto. L'osservazione chiave è che la clock condition è l'*unica* proprietà dell'orologio reale che serve: qualunque meccanismo di timestamp che la soddisfi si può usare al posto del tempo reale. E un meccanismo così esiste anche in un sistema asincrono.

3.4 Clock logici di Lamport

Il clock logico di Lamport è un contatore che ogni processo mantiene per conto proprio e che fa da orologio senza alcuna sincronizzazione tra i processi.

Definizione (clock logico di Lamport). Ogni processo p_i mantiene un contatore LC, il suo **clock logico** (*logical clock*), inizialmente 0. Ogni messaggio m porta un timestamp $TS(m)$, il valore del clock del mittente nell'evento di invio. A ogni nuovo evento e_i, p_i aggiorna il clock così:

$$LC(e_i) = \begin{cases} LC + 1 & \text{se } e_i \text{ è un evento interno o un invio,} \\ \max\{LC, TS(m)\} + 1 & \text{se } e_i = \mathit{receive}(m). \end{cases}$$

Alla ricezione il clock salta oltre il timestamp del messaggio: così il destinatario non resta indietro rispetto al passato del mittente.

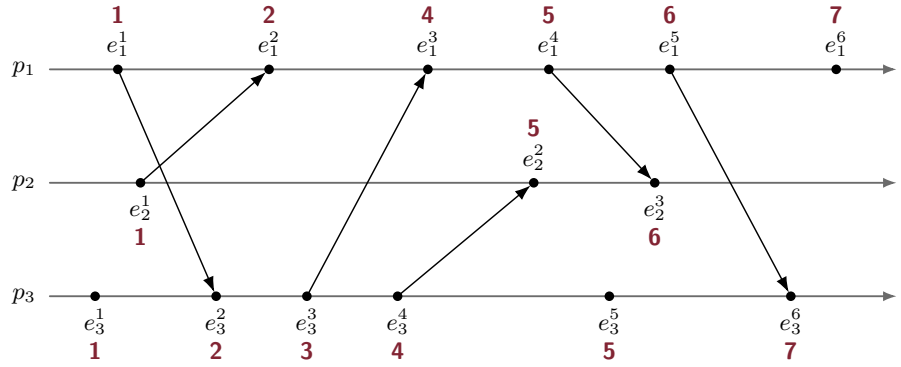


Figura 3.2. I clock logici della computazione d'esempio: accanto a ogni evento, in rosso, il valore di LC.

Esempio (clock logici passo passo). Applichiamo le regole alla computazione d'esempio (Figura 3.2). All'inizio tutti i clock valgono 0.

- I primi eventi e_1^1, e_2^1 ed e_3^1 sono invii o eventi interni, e il loro clock vale 1. Le richieste inviate in e_1^1 ed e_2^1 portano timestamp 1.
- e_1^2 riceve la richiesta di p_2, con timestamp 1: $\max\{1, 1\} + 1 = 2$. Allo stesso modo e_3^2, che riceve la richiesta di p_1, vale 2.
- p_3 invia la risposta, $LC(e_3^3) = 3$, e poi una richiesta a p_2, $LC(e_3^4) = 4$.
- e_1^3 riceve la risposta di p_3, con timestamp 3: $\max\{2, 3\} + 1 = 4$. Il clock di p_1 salta da 2 a 4 per superare il passato di p_3.
- e_2^2 riceve la richiesta di p_3, con timestamp 4: $\max\{1, 4\} + 1 = 5$.
- p_1 prosegue con la risposta a p_2, $LC(e_1^4) = 5$, la richiesta a p_3, $LC(e_1^5) = 6$, e l'evento interno e_1^6, che vale 7.
- e_2^3 riceve la risposta di p_1, con timestamp 5: $\max\{5, 5\} + 1 = 6$.

- e_3^5 è interno e vale 5; e_3^6 riceve la richiesta inviata in e_1^5, con timestamp 6: $\max\{5,6\}+1=7$. Il clock di p_3 passa da 5 a 7 e non assume mai il valore 6.

I clock logici crescono lungo ogni catena causale: a ogni passo dentro un processo il clock aumenta di uno, e a ogni messaggio il clock del destinatario supera il timestamp del mittente. Ne segue la proprietà fondamentale.

I clock logici soddisfano la clock condition. Per ogni coppia di eventi

$$e \to e' \;\Rightarrow\; LC(e) < LC(e').$$

La clock condition vale in un verso solo: un clock più piccolo non indica per forza una causa, e dai soli valori dei clock non si capisce se due eventi siano legati o concorrenti. Nella Figura 3.2, $LC(e_2^2)=5<LC(e_3^6)=7$, ma i due eventi sono concorrenti, come si è visto nel capitolo 1. Da $LC(e)<LC(e')$ si deduce soltanto che e' non precede causalmente e.

Ordinando tutti gli eventi per clock logico crescente, e risolvendo le parità con l'indice del processo, si ottiene un ordine totale che rispetta la precedenza causale, cioè un run consistente. Per la computazione d'esempio è

$$e_1^1\, e_2^1\, e_3^1\, e_1^2\, e_3^2\, e_3^3\, e_1^3\, e_3^4\, e_1^4\, e_2^2\, e_3^5\, e_1^5\, e_2^3\, e_1^6\, e_3^6.$$

3.5 Consegnare le notifiche in ordine di clock logico

Torniamo al monitor. Se i processi usano i clock logici come timestamp delle notifiche e p_0 le consegna in ordine di timestamp crescente, l'osservazione è consistente, perché i clock logici soddisfano la clock condition. Resta però un problema serio: senza un limite ai ritardi, p_0 non può mai essere sicuro che più tardi non arrivi una notifica con timestamp minore di quelle che ha in mano. Per prudenza non consegnerebbe mai nulla: la regola non ha *liveness*, cioè non garantisce che prima o poi qualcosa venga consegnato. Manca la capacità di accorgersi dei «buchi».

Definizione (gap detection). Un meccanismo di clock ha la proprietà di **gap detection** se, dati due eventi e ed e' con $LC(e)<LC(e')$, permette di stabilire se esiste un altro evento e'' con $LC(e)<LC(e'')<LC(e')$.

I clock logici non hanno questa proprietà: i valori possono saltare, come il clock di p_1 che passa da 2 a 4, e guardando i soli timestamp non si sa se da qualche parte ci sia un evento con valore 3 di cui non è ancora arrivata la notifica. Nemmeno l'orologio reale ha la gap detection, ma con il limite δ sui ritardi il controllo diventa facile: al tempo t sono arrivati tutti i messaggi con timestamp minore di $t-\delta$.

Con i clock logici serve un'informazione in più, e la fornisce la consegna FIFO. Supponiamo che i canali da ogni processo a p_0 siano FIFO. Poiché il clock di ogni processo cresce e i messaggi di uno stesso mittente arrivano in ordine, quando p_0 riceve da p_i un messaggio con un certo timestamp sa che da p_i non arriveranno

più messaggi con timestamp minore o uguale. Un messaggio m ricevuto da p_0 si dice **stabile** se non può più arrivare un messaggio con timestamp minore di $TS(m)$. Con canali FIFO, m è stabile non appena p_0 ha ricevuto da *ogni* altro processo almeno un messaggio con timestamp maggiore di $TS(m)$. Per esempio, p_0 ha una notifica di p_1 con timestamp 4, e ha già ricevuto un 6 da p_2 e un 5 da p_3: né p_2 né p_3 manderanno più niente sotto il 4, quindi la notifica è stabile. Se da p_3 avesse ricevuto solo un 2, da p_3 potrebbe ancora arrivare un 3, e p_0 dovrebbe aspettare.

Definizione (regola DR2). Consegna tutti i messaggi ricevuti che sono stabili per p_0, in ordine di timestamp crescente.

I due ingredienti hanno ruoli diversi: i clock logici danno l'ordine, perché una causa ha sempre un timestamp più piccolo dei suoi effetti, e la consegna FIFO dà la certezza che non arriverà più niente di più piccolo. La regola si può ancora bloccare se un processo smette di inviare notifiche, perché p_0 non riceverebbe mai da lui un timestamp più grande. Si rimedia facendo chiedere a p_0, periodicamente, una conferma a tutti i processi: le risposte hanno timestamp più grandi e rendono stabili i messaggi precedenti.

DR2 funziona, ma a volte aspetta senza bisogno (Sezione 4.2); il capitolo 4 mostra come evitarlo.

Capitolo 4

Consegna causale e vector clock

Nel capitolo precedente il monitor p_0 metteva in ordine le notifiche dei processi con i clock logici, così da osservare un run consistente, ma a volte aspettava senza bisogno. Questo capitolo introduce la consegna causale, cioè l'ordine di cui il monitor ha davvero bisogno, e i *vector clock* (clock vettoriali): timestamp da cui si capisce sempre se due eventi sono legati causalmente o concorrenti, e con cui la consegna causale si realizza senza attese inutili.

4.1 Consegna causale

Un messaggio viene *ricevuto* quando arriva dal canale, e *consegnato* quando il processo lo passa all'applicazione (Sezione 3.2). La consegna FIFO mette in ordine i messaggi di uno stesso mittente; con mittenti diversi non basta.

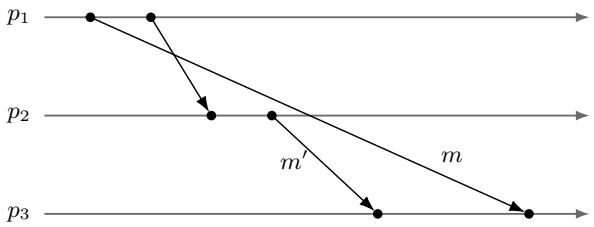


Figura 4.1. Consegna FIFO ma non causale: l'invio di m precede causalmente quello di m', ma p_3 riceve m' per primo.

Nella Figura 4.1, p_1 invia m a p_3 e poi un messaggio a p_2, che dopo averlo ricevuto invia m' a p_3. L'invio di m precede causalmente quello di m': c'è un cammino che parte dall'invio di m, segue p_1 fino all'invio a p_2, scende lungo la freccia e prosegue su p_2 fino all'invio di m'. Quindi m' *potrebbe* dipendere da m. Se invece p_1 scrivesse a p_2 prima di inviare m, il cammino non ci sarebbe, e i due invii sarebbero concorrenti.

Eppure m' arriva a p_3 per primo. La consegna FIFO è rispettata, perché su ogni canale viaggia un solo messaggio; ma se p_3 consegna subito m', vede l'effetto prima della causa, come chi in una chat di gruppo leggesse la risposta prima della domanda. La consegna causale lo vieta: se l'invio di un messaggio precede causalmente l'invio di un altro, e i due vanno allo stesso destinatario, il primo va consegnato prima.

Definizione (consegna causale). Una regola di consegna rispetta la **consegna causale** (*causal delivery*, CD) se, per ogni coppia di messaggi m ed m' diretti allo stesso processo p_k,

$$send_i(m) \rightarrow send_j(m') \;\Rightarrow\; deliver_k(m) \rightarrow deliver_k(m'),$$

dove il pedice indica il processo che esegue l'evento: p_i invia m, p_j invia m' e p_k consegna entrambi.

Con lo stesso mittente ($i = j$) si ritrova la consegna FIFO, perché due invii dello stesso processo sono sempre legati causalmente. La consegna causale è quindi una FIFO più forte: ordina anche i messaggi di mittenti diversi, ma solo se i loro invii sono legati causalmente; quelli con invii concorrenti si possono consegnare in qualunque ordine.

È esattamente ciò che serve al monitor. La notifica di un evento parte in quell'evento, quindi se $e \rightarrow e'$ l'invio della notifica di e precede causalmente quello della notifica di e': con la consegna causale il monitor vede sempre le cause prima degli effetti, e tutte le sue osservazioni sono consistenti.

La consegna causale dice *che cosa* garantire, non *come*. DR1 e DR2 del capitolo precedente la realizzano già, con un orologio globale e ritardi limitati oppure con canali FIFO e clock logici, ma sono troppo prudenti: ordinano per timestamp anche le notifiche di eventi concorrenti, e DR2 aspetta un timestamp più grande da tutti i processi. Il resto del capitolo costruisce una realizzazione che impone solo l'ordine richiesto, in un sistema asincrono: serve un timestamp che distingua le cause dagli eventi concorrenti (Sezione 4.2), che si ottiene dalle storie causali (Sezione 4.3) riassunte nei vector clock (Sezioni 4.4 e 4.5), usati dalla regola DR3 (Sezione 4.6).

4.2 La strong clock condition

DR1 e DR2 consegnano in ordine di timestamp crescente, quindi trattano ogni notifica con un timestamp più piccolo come una possibile causa. Con i clock logici della Figura 3.2, $LC(e_2^2) = 5$ e $LC(e_3^6) = 7$: se a p_0 arriva prima la notifica di e_3^6, deve aspettare quella di e_2^2, che potrebbe essere una sua causa. In realtà i due eventi sono concorrenti (capitolo 1), e l'attesa è inutile. Il valore 5, da solo, non dice se e_2^2 è una causa di e_3^6 o un evento concorrente: serve un timestamp da cui si capisca *esattamente* quando due eventi sono legati.

Definizione (strong clock condition). Una funzione T che associa un timestamp a ogni evento soddisfa la **strong clock condition** se, per ogni coppia di eventi,

$$e \rightarrow e' \iff T(e) < T(e').$$

Il verso da sinistra a destra è la clock condition del capitolo 3; quello nuovo dice che un timestamp più piccolo indica davvero una causa. Così il confronto tra due timestamp dice sempre come stanno due eventi: se uno dei due è più piccolo, quell'evento precede l'altro; se nessuno dei due lo è, sono concorrenti. Si dice che un

timestamp così **caratterizza** la precedenza causale: dai soli timestamp si ricostruisce tutta la relazione $\rightarrow$.

Un solo numero non basta: due numeri diversi sono sempre confrontabili, mentre i timestamp di due eventi concorrenti non devono esserlo. Servono timestamp con più dimensioni, che possano risultare incomparabili: è l'idea dei vector clock, a cui si arriva passando per le storie causali.

4.3 Storie causali

Il modo più diretto per soddisfare la strong clock condition è usare come timestamp di un evento tutto il suo passato, cioè l'insieme degli eventi che possono averlo influenzato.

Definizione (storia causale). La **storia causale** (*causal history*) di un evento e è l'insieme formato da e e da tutti gli eventi che lo precedono causalmente:

$$\theta(e) = \{\, e' \in H \mid e' \rightarrow e \,\} \cup \{e\},$$

dove θ è la lettera greca *theta* e H è la storia globale, cioè l'insieme di tutti gli eventi.

Nella Figura 4.2 è evidenziata la storia causale di e_1^4:

$$\theta(e_1^4) = \{e_1^1, e_1^2, e_1^3, e_1^4, e_2^1, e_3^1, e_3^2, e_3^3\}.$$

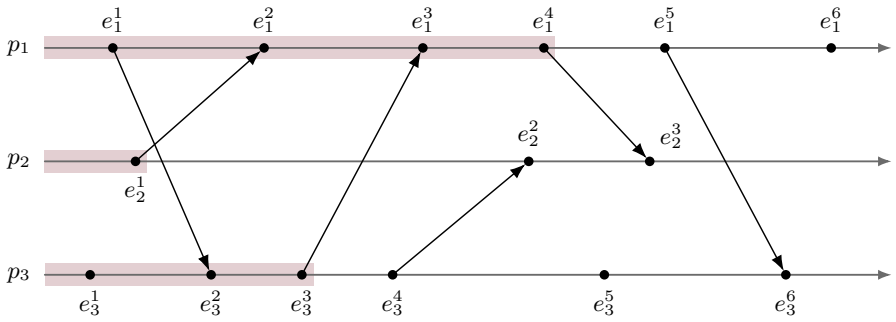


Figura 4.2. La storia causale di e_1^4: i tratti evidenziati delle storie locali.

Le storie causali si confrontano per contenimento: $e \rightarrow e'$ se e solo se $\theta(e) \subset \theta(e')$, e se nessuna delle due contiene l'altra gli eventi sono concorrenti. La strong clock condition è soddisfatta. Il difetto è pratico: le storie causali crescono senza limite, e i messaggi dovrebbero trasportare insiemi sempre più grandi.

4.4 Vector clock

Una storia causale si può riassumere con pochi numeri. Su ogni processo contiene sempre i primi eventi, senza buchi: se contiene un evento di p_3, contiene anche quelli

precedenti di p_3, perché precedono causalmente anche loro. Di p_3, per esempio, la storia causale di e_1^4 contiene e_3^1, e_3^2 ed e_3^3. Basta quindi dire, per ogni processo, quanti eventi contiene: per e_1^4 sono 4 di p_1, 1 di p_2 e 3 di p_3, cioè il vettore $(4, 1, 3)$.

Definizione (vector clock). Il **vector clock** di un evento e è il vettore $VC(e)$ di n numeri naturali, uno per processo: la componente i, indicata con $VC(e)[i]$, è il numero di eventi di p_i che stanno nella storia causale di e.
Si calcola mentre la computazione procede. Ogni processo p_i tiene un vettore VC, all'inizio tutto a zero, e ogni messaggio m porta come timestamp $TS(m)$ il vector clock dell'evento in cui è stato inviato. A ogni evento, p_i aggiorna il vettore così:

- se l'evento è interno o un invio, incrementa la propria componente: $VC[i] := VC[i] + 1$;
- se l'evento è la ricezione di un messaggio m, prima prende il massimo componente per componente tra il proprio vettore e il timestamp, $VC := \max\{VC, TS(m)\}$, e poi incrementa la propria componente.

Qui VC senza argomento è il vettore corrente del processo, che dopo un evento e vale $VC(e)$; $:=$ è un assegnamento, e il massimo si prende posizione per posizione, per esempio $\max\{(2, 1, 0), (1, 0, 3)\} = (2, 1, 3)$: chi riceve unisce ciò che sapeva con ciò che sapeva il mittente. Per un evento e di p_i, quindi, $VC(e)[i]$ è la posizione di e tra gli eventi di p_i, e ogni altra componente $VC(e)[j]$ dice quanti eventi di p_j precedono causalmente e, cioè quanto p_i «sa» di p_j in quel momento.

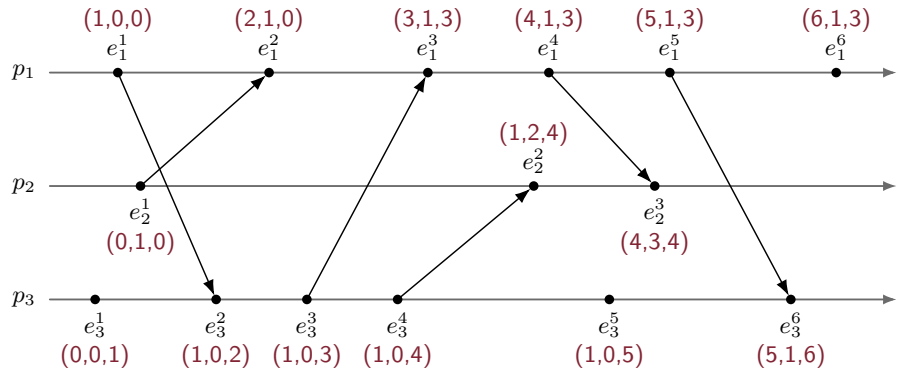


Figura 4.3. I vector clock della computazione d'esempio: accanto a ogni evento, in rosso, il suo vector clock.

Esempio (vector clock passo passo). Applichiamo le regole alla computazione d'esempio (Figura 4.3); i vettori hanno tre componenti, una per processo.

- e_1^1, e_2^1 ed e_3^1 sono i primi eventi dei rispettivi processi, con vector clock $(1, 0, 0)$, $(0, 1, 0)$ e $(0, 0, 1)$.
- e_1^2 riceve la richiesta di p_2, con timestamp $(0, 1, 0)$: il massimo con $(1, 0, 0)$ è $(1, 1, 0)$, e incrementando la componente di p_1 si ottiene $(2, 1, 0)$.

- e_3^2 riceve la richiesta di p_1, con timestamp $(1,0,0)$: dal massimo con $(0,0,1)$, cioè $(1,0,1)$, si ottiene $(1,0,2)$. Seguono due invii, con vector clock $(1,0,3)$ e $(1,0,4)$.
- e_1^3 riceve la risposta di p_3, con timestamp $(1,0,3)$: dal massimo con $(2,1,0)$, cioè $(2,1,3)$, si ottiene $(3,1,3)$. Gli eventi successivi di p_1 non ricevono nulla e incrementano solo la prima componente: $(4,1,3)$, $(5,1,3)$, $(6,1,3)$.
- e_2^2 riceve la richiesta di p_3, con timestamp $(1,0,4)$, e vale $(1,2,4)$; e_2^3 riceve la risposta di p_1, con timestamp $(4,1,3)$: il massimo con $(1,2,4)$ è $(4,2,4)$, quindi $(4,3,4)$.
- e_3^5 è interno, $(1,0,5)$; e_3^6 riceve la richiesta di p_1, con timestamp $(5,1,3)$, e vale $(5,1,6)$.

Per esempio $VC(e_2^3) = (4,3,4)$ dice che e_2^3 è il terzo evento di p_2 e che lo precedono causalmente quattro eventi di p_1 (e_1^1–e_1^4) e quattro di p_3 (e_3^1–e_3^4).

4.5 Confrontare i vector clock

Due vettori V e V' di n numeri si confrontano posizione per posizione: $V \leq V'$ se ogni componente di V è minore o uguale a quella di V', e $V < V'$ se in più i due vettori sono diversi. Per esempio $(1,0,0) < (2,1,0)$. A differenza dei numeri, due vettori possono essere **incomparabili**: tra $(2,1,0)$ e $(1,0,4)$ nessuno è minore dell'altro, perché il primo è più grande nella prima componente e il secondo nella terza. È proprio ciò che serve per rappresentare la concorrenza. Ecco che cosa si capisce dai vector clock, con i valori della Figura 4.3.

Un evento precede l'altro? Sì, se e solo se il suo vector clock è più piccolo: i vector clock soddisfano la strong clock condition,

$$e \to e' \iff VC(e) < VC(e').$$

Infatti e precede e' se e solo se il passato di e sta tutto nel passato di e', cioè se su ogni processo e' conosce almeno tanti eventi quanti ne conosce e. Per esempio $VC(e_2^1) = (0,1,0) < (5,1,6) = VC(e_3^6)$, quindi $e_2^1 \to e_3^6$.

Due eventi sono concorrenti? Sì, se e solo se i loro vector clock sono incomparabili, come $VC(e_2^2) = (1,2,4)$ e $VC(e_3^6) = (5,1,6)$.

Se si sa di quali processi sono i due eventi, basta una componente. Se e è il k-esimo evento di p_i, cioè $VC(e)[i] = k$, ed e' è un evento di un altro processo, e precede e' se e solo se e' conosce almeno i primi k eventi di p_i:

$$e \to e' \iff VC(e')[i] \geq VC(e)[i].$$

Per esempio e_1^4 è il quarto evento di p_1 e $VC(e_2^3)[1] = 4$, quindi $e_1^4 \to e_2^3$; invece e_2^2 è il secondo evento di p_2, ma e_3^6 ne conosce uno solo, quindi $e_2^2 \not\to e_3^6$.

Un taglio è consistente? Basta guardare la frontiera, cioè l'ultimo evento di ogni processo nel taglio. Sia $(c_1, \ldots, c_n)$ il taglio, dove c_i è il numero di eventi di p_i che contiene. Il taglio è consistente se e solo se nessun evento della frontiera conosce, di un processo, più eventi di quanti il taglio ne contenga, cioè se il vector clock di ogni evento della frontiera è minore o uguale alla tupla del taglio:

$$VC(e_j^{c_j}) \leq (c_1, \ldots, c_n) \qquad \text{per ogni processo } p_j,$$

dove $e_j^{c_j}$ è l'ultimo evento di p_j nel taglio (si considerano solo i processi con almeno un evento nel taglio). Se un evento della frontiera conoscesse più eventi di un processo di quanti il taglio ne contiene, dipenderebbe da un evento rimasto fuori. Per i due tagli della Figura 2.1:

- per $C = (5, 2, 4)$ i vettori della frontiera sono $(5, 1, 3)$, $(1, 2, 4)$ e $(1, 0, 4)$, tutti minori o uguali a $(5, 2, 4)$: il taglio è consistente;
- per $C' = (3, 2, 6)$ il vettore di e_3^6, $(5, 1, 6)$, supera il 3 nella prima componente: e_3^6 conosce cinque eventi di p_1, ma il taglio ne contiene tre, quindi C' è inconsistente.

Eventi come e_1^3 ed e_3^6, che non possono stare insieme sulla frontiera di un taglio consistente, si dicono **inconsistenti a coppie** (*pairwise inconsistent*).

Manca qualcosa in mezzo? Prima di consegnare una notifica, il monitor deve sapere se gli manca ancora qualcosa da consegnare prima. Con i clock logici questa domanda, la gap detection (Sezione 3.5), non ha risposta, perché i valori saltano: chi ha consegnato fino al 5 e riceve un 7 non sa se esiste un 6 ancora in viaggio. I vector clock rispondono a una domanda più modesta, che però basta: la **weak gap detection**.

$VC(e)[k] = c$ vuol dire che e conosce i primi c eventi di p_k. Se e ne conosce meno di e', gli eventi di p_k in più stanno nel passato di e' ma non in quello di e:

$$VC(e)[k] < VC(e')[k] \implies \text{esiste un evento } f \text{ di } p_k \text{ con } f \not\to e \text{ e } f \to e'.$$

La proprietà è debole perché «e non conosce f» non vuol dire «f viene dopo e». Con $VC(e_1^2) = (2, 1, 0)$ e $VC(e_2^3) = (4, 3, 4)$:

- nella terza componente $0 < 4$, ma un evento trovato così, come e_3^1, è concorrente a e_1^2: non sta *tra* i due;
- nella prima componente $2 < 4$, e gli eventi trovati, e_1^3 ed e_1^4, sono dello stesso processo di e_1^2, quindi vengono dopo: stanno proprio tra i due.

Il monitor della Sezione 4.6 è sempre nel secondo caso, perché per ogni processo p_k confronta la notifica arrivata con l'ultima notifica di p_k già consegnata, cioè con un evento di p_k stesso. Se la notifica arrivata conosce le prime 3 notifiche di p_1 e il monitor ne ha consegnate 2, manca la terza, che va consegnata prima: il monitor sa *che* manca qualcosa, *di chi* è e *quanto* manca.

4.6 Consegna causale con i vector clock

Torniamo al monitor. Ogni notifica porta come timestamp il vector clock dell'evento notificato, e i processi incrementano la propria componente solo negli eventi che

notificano (o, che è lo stesso, notificano tutti i loro eventi): così le componenti contano notifiche, e il monitor, che vede solo quelle, le può confrontare con quelle che ha ricevuto. Per una notifica m inviata da p_j:

- $TS(m)[j]$ è la posizione di m tra le notifiche di p_j (m è la prima, la seconda, ...);
- $TS(m)[k]$, per ogni altro processo p_k, è il numero di notifiche di p_k che precedono causalmente m, cioè quante ne conosceva p_j quando ha inviato m.

Il monitor tiene da parte le notifiche ricevute ma non ancora consegnate, e un contatore per ogni processo: $D[k]$, inizialmente 0, è il numero di notifiche di p_k già consegnate.

Il monitor può consegnare una notifica m di p_j quando nessun'altra notifica, arrivata o ancora in viaggio, deve venire prima. Per esserne sicuro risponde a due domande.

1. *È la prossima notifica di p_j?* Le notifiche di p_j vanno consegnate nell'ordine di invio: se m è la terza, il monitor deve averne consegnate esattamente due.
2. *Ha già consegnato tutto ciò che p_j conosceva?* Le notifiche degli altri processi che p_j conosceva quando ha inviato m precedono causalmente m, quindi vanno consegnate prima. Se ne manca qualcuna, la weak gap detection lo rivela, con il buco su p_k.

Se una risposta è no, la notifica aspetta, e il monitor la riesamina ogni volta che consegna qualcos'altro.

Definizione (regola DR3, consegna causale). Il monitor consegna la notifica m di p_j non appena

$$D[j] = TS(m)[j] - 1 \qquad \text{e} \qquad D[k] \geq TS(m)[k] \ \text{ per ogni } k \neq j,$$

e dopo la consegna pone $D[j] := TS(m)[j]$.

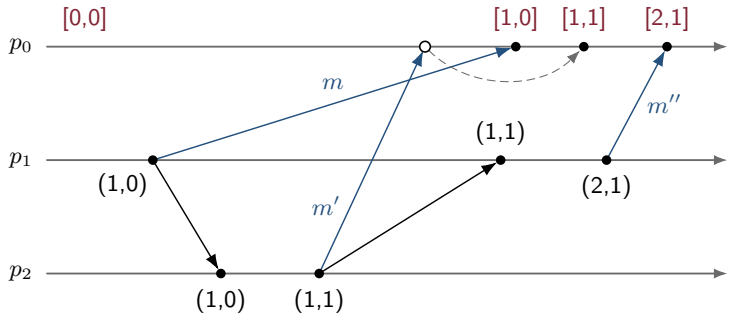


Figura 4.4. La regola DR3 al lavoro. Su p_1 e p_2 i vector clock, su p_0 (in rosso) i valori di D; in blu le notifiche. m' arriva prima di m (cerchio vuoto) e viene consegnata solo dopo di essa.

Esempio (la regola DR3 al lavoro). Nella Figura 4.4 ci sono due processi, p_1 e p_2, e il monitor p_0. I vector clock hanno due componenti, una per processo, e si incrementano solo negli eventi notificati.

- p_1 esegue un evento notificato, con vector clock $(1,0)$, in cui invia un messaggio a p_2; la sua notifica è m, la prima di p_1.
- p_2 riceve il messaggio. L'evento non è notificato, quindi il vector clock diventa $\max\{(0,0),(1,0)\} = (1,0)$ senza incremento. Poi p_2 esegue un evento notificato, con vector clock $(1,1)$, in cui invia un messaggio a p_1; la sua notifica è m', la prima di p_2.
- m' arriva a p_0 per prima, con $TS(m') = (1,1)$ e $D = [0,0]$. È la prossima notifica di p_2 ($D[2] = 0 = 1-1$), ma p_2 conosceva una notifica di p_1 che il monitor non ha ancora consegnato ($D[1] = 0 < 1$): m' resta in attesa.
- Arriva m, con $TS(m) = (1,0)$: è la prossima notifica di p_1 ($D[1] = 0 = 1-1$) e p_1 non conosceva notifiche di p_2 ($D[2] = 0 \geq 0$), quindi m viene consegnata e D diventa $[1,0]$. Ora anche m' supera la seconda domanda ($D[1] = 1 \geq 1$): viene consegnata e D diventa $[1,1]$.
- p_1 riceve il messaggio di p_2 (vector clock $(1,1)$) e poi esegue un evento notificato, $(2,1)$, con notifica m''. Quando m'' arriva è la prossima notifica di p_1 ($D[1] = 1 = 2-1$) e il monitor ha già consegnato la notifica di p_2 che p_1 conosceva ($D[2] = 1 \geq 1$): viene consegnata subito, e D diventa $[2,1]$.

Con DR3 si ha una soluzione completa di GPE con monitoraggio passivo in un sistema asincrono: i processi notificano gli eventi rilevanti, p_0 li consegna in ordine causale e osserva un run consistente, in cui Φ si può valutare in qualunque stato. La stessa regola funziona in ogni processo, non solo nel monitor: se tutti comunicano con *broadcast*, cioè inviando ogni messaggio a tutti, DR3 dà il *causal broadcast*, una primitiva molto usata.

Canali nascosti. La precedenza causale vede solo la comunicazione che passa per i canali del sistema (un **sistema chiuso**); un **canale nascosto** (*hidden channel*), esterno al sistema, le sfugge. Per esempio p_1 rileva la rottura di un tubo del vapore e lo notifica a p_3, che controlla il riscaldamento; poco dopo p_2, più a valle, rileva un calo di pressione e notifica anche quello. Per il sistema le due notifiche sono concorrenti: p_3 può vedere prima il calo di pressione, aumentare il calore e concludere a torto che il tubo si è rotto per il calore. Il legame tra le due cose passa per il tubo, fuori dal sistema; in casi così bisogna ordinare le osservazioni con il tempo reale.

Capitolo 5

Snapshot distribuiti

Nel capitolo 2 il monitor costruiva uno stato globale chiedendo a ogni processo il proprio stato, e il risultato poteva essere inconsistente. Un protocollo di **snapshot** (istantanea) fa la stessa cosa, ma garantisce uno stato globale consistente, senza fermare la computazione. Si parte da un protocollo che usa un orologio reale, lo si trasforma nel protocollo di Chandy e Lamport, che funziona in un sistema asincrono, e infine si vede che cosa garantisce lo stato costruito. In tutto il capitolo i canali sono FIFO.

5.1 Che cosa registra uno snapshot

Uno snapshot è una fotografia del sistema, e deve contenere due cose: lo stato locale di ogni processo e i messaggi che in quel momento sono in viaggio.

> **Definizione (stato di un canale).** Lo **stato del canale** da p_i a p_j, indicato con $\chi_{i,j}$ (con la lettera greca *chi*), è l'insieme dei messaggi in viaggio su quel canale: quelli che p_i ha inviato a p_j e che p_j non ha ancora ricevuto.

I canali su cui un processo riceve sono i suoi **canali entranti**, quelli su cui invia i suoi **canali uscenti**. Ogni processo registra il proprio stato locale e lo stato dei propri canali entranti: così ogni canale viene registrato una volta sola, da chi sta all'estremità di arrivo. Le difficoltà sono due: i processi non possono registrare tutti nello stesso istante, e i messaggi in viaggio non li vede nessuno.

5.2 Primo protocollo: un orologio reale

Supponiamo per ora che tutti i processi leggano lo stesso orologio reale RC, e che i ritardi dei messaggi e le velocità dei processi abbiano limiti noti. Allora la soluzione è ovvia: tutti registrano il proprio stato nello stesso istante, come in una foto di gruppo. Il monitor p_0 sceglie l'istante dello snapshot, t_{ss} (*time* dello *snapshot*), abbastanza avanti nel tempo perché il suo annuncio arrivi a tutti prima di t_{ss}. Ogni messaggio porta come timestamp l'istante in cui è stato inviato.

Definizione (Protocollo 1, con orologio reale).

1. p_0 invia a tutti i processi il messaggio «take snapshot at t_{ss}».
2. Quando l'orologio segna t_{ss}, ogni processo p_i, prima di qualunque altro evento, registra il proprio stato locale σ_i, invia un messaggio vuoto su ogni canale uscente e comincia a registrare i messaggi che arrivano su ogni canale entrante.
3. La prima volta che riceve da p_j un messaggio con timestamp maggiore o uguale a t_{ss}, p_i smette di registrare quel canale: $\chi_{j,i}$ è l'insieme dei messaggi registrati.

Sul canale da p_j a p_i, all'istante t_{ss}, sono in viaggio i messaggi inviati prima di t_{ss} e ricevuti dopo. p_i registra proprio quelli: comincia a registrare in t_{ss} e si ferma al primo messaggio inviato da t_{ss} in poi, perché il canale è FIFO e a quel punto tutti i messaggi inviati prima sono già arrivati. I messaggi vuoti servono solo a far arrivare quel primo messaggio anche su un canale su cui p_j non avrebbe inviato più nulla.

Lo stato costruito è consistente, perché il sistema ci si è trovato davvero, nell'istante t_{ss}. Conta però capire che cosa lo garantisce. Il taglio dello snapshot contiene gli eventi avvenuti prima di t_{ss}. Se e è nel taglio ed e' è una sua causa, allora $RC(e') < RC(e) < t_{ss}$, dove $RC(e)$ è l'istante in cui avviene e, e la prima disuguaglianza è la clock condition: anche e' è nel taglio, che quindi contiene le cause di ogni suo evento.

5.3 Dall'orologio reale ai marker

Il Protocollo 1 usa dell'orologio reale soltanto la clock condition, come la regola DR1 (Sezione 3.3), e la clock condition vale anche per i clock logici. Da qui si arriva al protocollo di Chandy–Lamport in tre passi, riassunti nella Tabella 5.1.

1. Clock logici al posto dell'orologio (Protocollo 2). Le regole sono quelle del Protocollo 1, con il clock logico LC di ogni processo al posto dell'orologio reale. Cambia la scelta dell'istante: in un sistema asincrono p_0 non può calcolare un valore «abbastanza avanti». Si usa allora un valore ω (*omega*) così grande che nessun clock lo raggiunge con gli eventi normali della computazione. p_0 invia l'annuncio «take snapshot at ω», porta il proprio clock a ω, registra il proprio stato e invia i messaggi vuoti, con timestamp ω. Quando un altro processo riceve un messaggio con timestamp ω, il suo clock salta oltre ω: il processo registra il proprio stato subito prima di quella ricezione e invia a sua volta i messaggi vuoti. Così la registrazione si propaga a tutti. Resta un'ipotesi scomoda: perché un valore come ω esista, servono comunque limiti alle velocità e ai ritardi.

2. Niente annuncio. Nel Protocollo 2 un processo non fa nulla tra l'arrivo dell'annuncio e l'arrivo del primo messaggio vuoto, ed è quel messaggio a farlo registrare. L'annuncio quindi non serve: un processo registra quando riceve il primo

messaggio vuoto. Perché non si confonda con altri messaggi, gli si dà un contenuto riconoscibile, «take snapshot», e lo si chiama **marker** (segnalino).

3. Niente timestamp. Su ogni canale il marker è il primo messaggio che il mittente invia dopo essersi registrato, e il canale è FIFO. Quindi i messaggi che arrivano prima del marker sono stati inviati prima della registrazione del mittente, e quelli che arrivano dopo sono stati inviati dopo: il marker *separa* i due gruppi, e fa da solo il lavoro del timestamp. Spariscono così i timestamp, i clock logici e ω, e resta il protocollo di Chandy–Lamport, che non fa ipotesi oltre ai canali FIFO.

Tabella 5.1. I tre protocolli di snapshot a confronto.

	Protocollo 1	Protocollo 2	Chandy–Lamport
Quando p_i registra il proprio stato	quando l'orologio reale segna t_{ss}	quando il suo clock logico raggiunge ω	alla prima ricezione di un marker
Quando smette di registrare il canale da p_j	al primo messaggio di p_j con timestamp $\geq t_{ss}$	al primo messaggio di p_j con timestamp $\geq \omega$	quando arriva il marker di p_j
Ipotesi, oltre ai canali FIFO	orologio comune; ritardi e velocità limitati	un valore ω che nessun clock raggiunge da solo	nessuna

5.4 Il protocollo di Chandy–Lamport

Vediamo il protocollo all'opera su due conti bancari, come nel bonifico della Sezione 1.1. p_1 ha 100 euro, p_2 ne ha 50, e si mandano bonifici, quindi in tutto i soldi sono sempre 150. Ogni euro sta in uno di tre posti: sul conto di p_1, sul conto di p_2 o in un bonifico in viaggio. Uno snapshot è giusto se conta ogni euro una volta sola, né due volte né nessuna. Chiamiamo *foto* di un processo lo stato locale che registra, qui il suo saldo, e *lista* di un canale l'elenco dei messaggi che il destinatario registra su quel canale: alla fine è lo stato del canale.

Seguiamo il canale da p_1 a p_2, l'unico su cui viaggiano soldi: sugli altri canali passano solo marker, e le loro liste restano vuote. p_1 invia due bonifici, A da 10 euro prima della sua foto e B da 20 euro dopo. Nella prima esecuzione, nella Tabella 5.2, il marker che p_0 invia a p_2 arriva presto, prima di A.

Tabella 5.2. Prima esecuzione: il marker di p_0 arriva a p_2 prima di A.

		Saldi		
	Che cosa succede	p_1	p_2	Che cosa si registra
1	p_1 invia A (10 euro)	90	50	
2	p_0 invia i marker a p_1 e a p_2	90	50	
3	il marker di p_0 arriva a p_1	90	50	p_1 fa la foto, 90, e invia subito il suo marker a p_2
4	p_1 invia B (20 euro)	70	50	
5	il marker di p_0 arriva a p_2	70	50	p_2 fa la foto, 50, invia il suo marker a p_1 e apre la lista del canale da p_1
6	A arriva a p_2	70	60	A entra nella lista
7	il marker di p_1 arriva a p_2	70	60	p_2 chiude la lista: $\{A\}$
8	B arriva a p_2	70	80	la lista è chiusa: B non entra

Lo snapshot dà $90 + 50 + 10 = 150$: ogni bonifico è contato una volta. Per capire perché, bisogna guardare dove sta ciascuno nelle due foto.

- A nella foto di p_1 è già uscito, perché i 90 non lo contengono, e in quella di p_2 non è ancora entrato. Nello snapshot è in viaggio: senza la lista quei 10 euro sparirebbero, e il totale sarebbe 140.
- B nella foto di p_1 c'è ancora, perché p_1 l'ha inviato dopo la foto: è dentro i 90. Se entrasse nella lista sarebbe contato due volte, e il totale sarebbe 170.

p_2 non sa quando p_1 ha fatto la foto, quindi da solo non distingue A da B. Glielo dice il marker, come nel passo 3 della Sezione 5.3: p_1 lo invia subito dopo la foto, e il canale è FIFO, quindi A, partito prima, arriva prima del marker, e B, partito dopo, arriva dopo. Così p_2 scrive nella lista:

- a partire dalla propria foto, perché ciò che è arrivato prima è già contato nella foto di p_2;
- fino al marker di p_1, perché ciò che arriva dopo è ancora nella foto di p_1.

Per lo stesso motivo p_1 deve inviare il marker subito dopo la foto, prima di qualunque altro messaggio: se inviasse B prima del marker, B arriverebbe prima del confine, entrerebbe nella lista e sarebbe contato due volte.

Il sistema è asincrono, quindi il marker che p_0 invia a p_2 può metterci quanto vuole. Nella seconda esecuzione, nella Tabella 5.3, quel marker arriva a p_2 dopo B. Può succedere perché i due marker diretti a p_2 viaggiano su canali diversi, uno da p_0 e uno da p_1, e il FIFO ordina solo i messaggi di uno stesso canale: garantisce che il marker di p_1 arrivi dopo A e prima di B, non che arrivi dopo quello di p_0.

Tabella 5.3. Seconda esecuzione: le righe 1–4 sono quelle della Tabella 5.2, ma il marker di p_0 arriva a p_2 dopo B.

		Saldi		
	Che cosa succede	p_1	p_2	Che cosa si registra
1	p_1 invia A (10 euro)	90	50	
2	p_0 invia i marker a p_1 e a p_2	90	50	
3	il marker di p_0 arriva a p_1	90	50	p_1 fa la foto, 90, e invia subito il suo marker a p_2
4	p_1 invia B (20 euro)	70	50	
5	A arriva a p_2	70	60	nessun marker ancora: A entra nel saldo come un bonifico qualsiasi
6	il marker di p_1 arriva a p_2	70	60	è il primo marker per p_2: fa la foto, 60, e invia il suo marker a p_1; la lista del canale da p_1 resta vuota
7	B arriva a p_2	70	80	arriva dopo il marker di p_1: non entra in nessuna lista
8	il marker di p_0 arriva a p_2	70	80	non è il primo: p_2 non rifà la foto

Lo snapshot dà $90+60=150$. Se p_2 aspettasse il marker di p_0, alla riga 8 farebbe la foto con 80, e il totale sarebbe 170: B sarebbe contato sia nei 90 di p_1 sia negli 80 di p_2. È il messaggio ricevuto ma non ancora inviato che rende inconsistente un taglio (Sezione 2.3): B è già arrivato nella foto di p_2 e non è ancora partito in quella di p_1. Per questo un processo fa la foto al *primo* marker che riceve, da chiunque arrivi. Dietro al marker di p_1 arrivano solo bonifici che sono ancora nella foto di p_1: facendo la foto al più tardi all'arrivo di quel marker, p_2 non ne riceve nessuno prima della foto. La lista del canale da p_1 resta vuota perché la foto di p_2 e l'arrivo del marker di p_1 sono lo stesso momento, e in mezzo non arriva niente.

Rispetto alla prima esecuzione cambia solo chi manda a p_2 il primo marker, e quindi dove finisce A: là nella lista, qui nella foto di p_2. In entrambe ogni euro è contato una volta, e B sta nei 90 di p_1. In generale il protocollo è il seguente.

Definizione (protocollo di Chandy–Lamport). p_0 avvia il protocollo inviando un marker a sé stesso. Quando un processo p_i riceve un marker dal canale di p_j:

- se è il *primo* marker che riceve, fa la foto, cioè registra il proprio stato locale σ_i, e invia subito un marker su ogni canale uscente, senza eventi della computazione in mezzo. La lista del canale da p_j resta vuota ($\chi_{j,i} = \emptyset$), e su ciascuno degli altri canali entranti p_i apre una lista, in cui scrive i messaggi che arrivano;
- altrimenti chiude la lista del canale da p_j: $\chi_{j,i}$ è l'insieme dei messaggi che contiene.

Indichiamo con e_i^* il momento in cui p_i fa la foto: divide gli eventi di p_i in quelli che vengono prima della foto e quelli che vengono dopo. Nelle figure è un quadratino.

Un processo inoltra il marker solo alla prima ricezione, e la rete è fortemente connessa: il marker raggiunge tutti e attraversa ogni canale esattamente una volta. Quando p_i ha ricevuto il marker da tutti i canali entranti ha la propria foto e le liste di tutti i canali entranti, e per lui il protocollo è finito: resta solo da inviarle a p_0. Il monitor ha quindi due compiti, avviare lo snapshot e raccoglierne i pezzi. Il marker che invia a sé stesso gli fa seguire la stessa regola degli altri processi, cioè inviare un marker su ogni canale uscente, ma il suo stato non fa parte dello snapshot, perché non partecipa alla computazione.

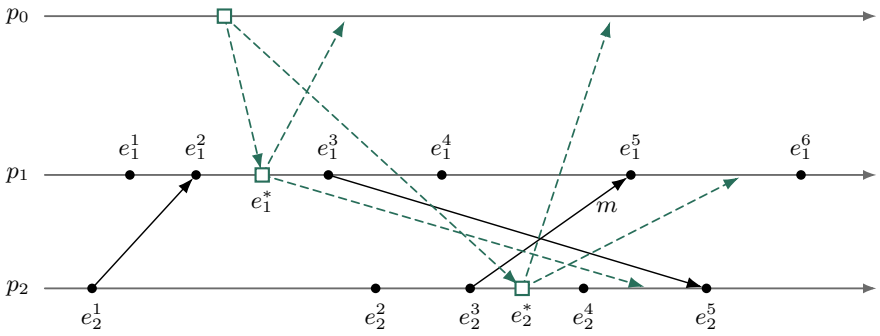


Figura 5.1. Il protocollo di Chandy–Lamport sulla computazione della Figura 2.3. Tratteggiati i marker; i quadratini segnano le foto (e_1^* ed e_2^*).

Esempio (una computazione a due processi). Nella Figura 5.1 il protocollo viene eseguito sulla computazione di due processi della Figura 2.3. Il marker di p_0 arriva a p_1 dopo e_1^2, e p_1 fa la foto σ_1^2; arriva a p_2 solo dopo e_2^3, e p_2 fa la foto σ_2^3. Le liste dei canali da p_0 restano vuote, perché da lì arrivano i primi marker. I tre messaggi della computazione fanno le parti dei bonifici:

- il messaggio di e_2^1 arriva a p_1 prima della foto di p_1, come A nella seconda esecuzione: è già contato in σ_1^2;
- m parte da p_2 prima della foto di p_2, e arriva a p_1 dopo la foto di p_1 e prima del marker di p_2, come A nella prima esecuzione: entra nella lista, e $\chi_{2,1} = \{m\}$;
- il messaggio di e_1^3 parte da p_1 dopo la foto di p_1, come B: arriva a p_2 dopo il marker di p_1, quando la lista è già chiusa, e $\chi_{1,2} = \emptyset$.

Lo snapshot è lo stato globale Σ^{23} (due eventi di p_1, tre di p_2), con m in viaggio da p_2 a p_1.

5.5 Perché ogni messaggio è contato una volta

Nei due conti ogni euro è contato una volta. Lo stesso vale per ogni messaggio da p_j a p_i, che sta sempre in uno di tre casi:

- *partito prima della foto di p_j e arrivato prima di quella di p_i*: è contato nella foto di p_i, e non nella lista, che p_i apre solo con la foto. È A nella seconda esecuzione;

- *partito prima della foto di p_j e arrivato dopo quella di p_i*: è in viaggio, ed è nella lista, perché arriva dopo la foto di p_i e prima del marker di p_j. È A nella prima esecuzione;
- *partito dopo la foto di p_j*: nello snapshot non è ancora partito, e infatti non compare. Viaggia dietro al marker di p_j, quindi quando arriva p_i ha già fatto la foto, al più tardi a quel marker, e ha già chiuso la lista del canale. È B.

Le liste contengono quindi esattamente i messaggi in viaggio. Il terzo caso dice anche che lo snapshot è consistente: un messaggio inviato dopo la foto del mittente è sempre ricevuto dopo quella del destinatario, quindi lo snapshot non contiene mai un messaggio ricevuto ma non ancora inviato, come B nel totale di 170. Servono però i canali FIFO: se B potesse superare il marker di p_1, nella seconda esecuzione arriverebbe a p_2 prima di ogni marker, e p_2 lo conterebbe nella foto.

5.6 Proprietà dello snapshot

Lo stato costruito è consistente, ma il sistema potrebbe non esserci mai passato. Nella Figura 5.1, quando p_1 fa la foto p_2 ha eseguito un solo evento, e quando p_2 fa la foto p_1 è già arrivato a e_1^4: nel run reale, il cammino blu della Figura 5.2, lo stato Σ^{23} non compare. Lo snapshot però non è uno stato consistente qualunque. Siano:

- Σ^a (a come avvio) lo stato del sistema quando il protocollo parte;
- Σ^s (s come snapshot) lo stato costruito dal protocollo;
- Σ^f (f come fine) lo stato del sistema quando il protocollo termina, cioè quando arriva l'ultimo marker.

Allora

$$\Sigma^a \rightsquigarrow \Sigma^s \rightsquigarrow \Sigma^f,$$

cioè esiste un run consistente che passa, nell'ordine, per l'avvio, per lo snapshot e per la fine: se non è quello reale, è uno che il sistema avrebbe potuto seguire.

Perché. Un evento di p_i è **pre-registrazione** (*prerecording*) se viene prima della foto, cioè di e_i^*, **post-registrazione** (*postrecording*) se viene dopo. Σ^s è lo stato in cui si arriva eseguendo tutti gli eventi pre-registrazione e nessun altro. Un evento post-registrazione non può precedere causalmente un evento pre-registrazione: servirebbe un messaggio inviato dopo la foto del mittente e ricevuto prima di quella del destinatario, e per la Sezione 5.5 non ce ne sono. Quindi, nel run reale, si possono spostare tutti gli eventi pre-registrazione davanti a quelli post-registrazione senza mai mettere un effetto prima della sua causa, e il run che si ottiene è consistente. Passa per Σ^a, perché gli eventi avvenuti prima dell'avvio sono tutti pre-registrazione e restano in testa; per Σ^s, subito dopo l'ultimo evento pre-registrazione; per Σ^f, perché gli eventi avvenuti dopo la fine sono tutti post-registrazione e restano in coda.

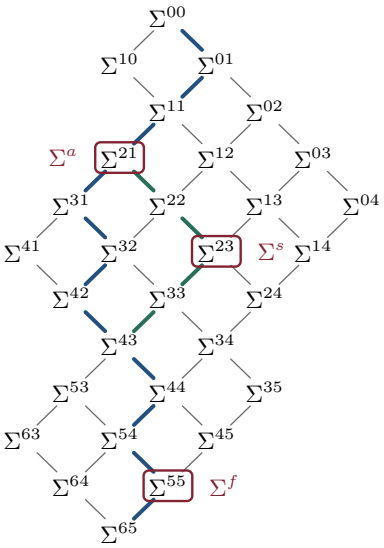


Figura 5.2. Lo snapshot nel reticolo della Figura 2.3. Il run reale (blu) passa per l'avvio $\Sigma^a = \Sigma^{21}$ e la fine $\Sigma^f = \Sigma^{55}$, ma non per lo stato costruito $\Sigma^s = \Sigma^{23}$. Il run riordinato (verde) se ne discosta solo tra Σ^{21} e Σ^{43}, e passa per tutti e tre.

Nella Figura 5.1 gli eventi pre-registrazione sono e_1^1, e_1^2 ed e_2^1, e_2^2, e_2^3. Nel run reale gli eventi post-registrazione e_1^3 ed e_1^4 vengono prima di alcuni eventi pre-registrazione di p_2; anticipando questi ultimi si ottiene il run

$$e_2^1\, e_1^1\, e_1^2\, e_2^2\, e_2^3 \mid e_1^3\, e_1^4\, e_2^4\, e_1^5\, e_2^5\, e_1^6,$$

dove la sbarra separa i due gruppi. È il cammino verde della Figura 5.2, e passa per $\Sigma^a = \Sigma^{21}$, per $\Sigma^s = \Sigma^{23}$ e per $\Sigma^f = \Sigma^{55}$.

A che cosa serve. Lo snapshot descrive uno stato che forse non è mai esistito, e che comunque è già passato. Serve per le proprietà **stabili**, quelle che, una volta diventate vere, restano vere, come il deadlock o la terminazione. Per una proprietà così, se vale in Σ^s vale anche in Σ^f, cioè adesso; se non vale in Σ^s, non valeva nemmeno in Σ^a, quando il protocollo è partito. Le proprietà stabili si riprenderanno più avanti.

Capitolo 6

Guasti e commit atomico

Finora i processi non si guastavano e ogni messaggio arrivava, e il problema era osservare il sistema. Da qui in poi siti e collegamenti si possono guastare, e il problema diventa far prendere a un gruppo di processi *la stessa decisione*. Il caso tipico è la conclusione di una transazione distribuita, che va confermata in tutti i siti in cui ha lavorato oppure annullata in tutti. Questo capitolo descrive il problema, i guasti possibili e i limiti di ogni soluzione; il capitolo 7 presenta il protocollo più usato. Il contesto è quello delle basi di dati, ma le idee valgono per qualunque sistema distribuito.

6.1 Transazioni distribuite

Una **transazione** è una sequenza di letture e scritture da eseguire come un'operazione sola: o hanno effetto tutte, o nessuna. Termina con il **Commit** (conferma), che rende definitivi i suoi effetti, oppure con l'**Abort** (annullamento), che li cancella come se non fosse mai stata eseguita.

In una base di dati distribuita i dati stanno su più **siti**, cioè su computer collegati da una rete, e supponiamo che ogni dato stia in un solo sito. Una transazione distribuita T nasce in un sito, il suo *home site*, dove il *transaction manager* (TM) inoltra ogni lettura o scrittura del dato x al sito in cui sta x: per letture e scritture non cambia niente rispetto a una base di dati centralizzata. Il Commit e l'Abort, invece, riguardano tutti i siti in cui T ha lavorato, e ognuno li deve eseguire. Ne nascono due difficoltà:

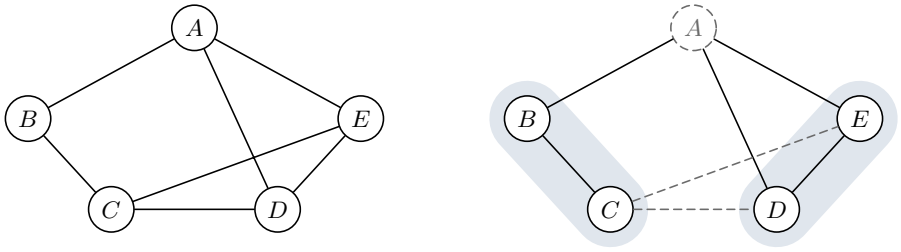
- *non basta inviare il Commit*: T è confermata quando ogni sito lo esegue, e un sito potrebbe rifiutarsi, per esempio perché il suo controllo della concorrenza ha annullato T; allora T va annullata ovunque;
- *i guasti sono parziali*: in un sistema centralizzato un guasto ferma tutto, mentre in un sistema distribuito alcuni siti possono funzionare e altri no.

Il problema è la **terminazione consistente**: la conclusione di T deve essere la stessa in tutti i siti coinvolti, anche quando alcuni si guastano. Un algoritmo che la garantisce è un **protocollo di commit atomico** (*atomic commitment protocol*, ACP), e lo si vuole resistente al maggior numero possibile di guasti.

Esempio (bonifico tra due banche). T sposta 100 euro dal conto x, nel sito A, al conto y, nel sito B. Se A esegue il Commit e B l'Abort, per esempio perché B si è guastato prima di salvare la scrittura, i 100 euro spariscono; nel caso opposto vengono creati dal nulla. Nella Sezione 1.1 bisognava osservare il bonifico senza sbagliare; qui i due siti devono prendere la stessa decisione.

6.2 Il modello dei guasti

Un sistema distribuito è fatto di **siti**, che elaborano, e di **collegamenti** (*link*) bidirezionali, che trasportano i messaggi: è un grafo connesso, come nella Figura 6.1(a). L'instradamento dei messaggi è un servizio della rete, e non ci interessa.



(a) la rete (b) una partizione

Figura 6.1. (a) Una rete di cinque siti. (b) Una partizione della stessa rete: il sito A è guasto e i collegamenti (C, D) e (C, E) sono interrotti (tratteggiati), quindi i siti operativi si dividono in due componenti, $\{B, C\}$ e $\{D, E\}$ (in azzurro).

6.2.1 Guasti dei siti

Un sito che si guasta si ferma di colpo e perde il contenuto della **memoria volatile** (la memoria centrale); quello della **memoria stabile**, come i dischi, resta. Quando riparte, una procedura di recupero lo riporta in uno stato corretto. Si assume che un sito sia sempre **operativo**, e allora funzioni correttamente, oppure **guasto** (*down*), e allora non faccia niente: non esegue mai azioni sbagliate. È il modello **fail-stop**, un'idealizzazione a cui ci si avvicina con test accurati e componenti ridondanti.

Si ha un **guasto parziale** quando alcuni siti sono operativi e altri no, un **guasto totale** quando sono guasti tutti. I guasti parziali sono insidiosi perché un sito operativo può non sapere che cosa avesse deciso un sito guasto, e questo può costringerlo ad aspettare (Sezione 6.4).

6.2.2 Guasti di comunicazione

Messaggi alterati o persi e collegamenti interrotti per poco li gestisce la rete, con codici che rilevano gli errori, ritrasmissioni e percorsi alternativi. Non c'è rimedio, però, se tutti i percorsi tra due siti passano per un sito guasto o per un collegamento interrotto: è una **partizione della rete** (*network partition*). I siti operativi si dividono in **componenti**: due siti della stessa componente comunicano, due di componenti diverse no. Nella Figura 6.1(b) il guasto di A e dei collegamenti (C, D) e (C, E) produce le componenti $\{B, C\}$ e $\{D, E\}$, che si riuniscono appena A riparte o

un collegamento torna a funzionare. In generale si ha un **guasto di comunicazione** quando due siti non guasti non riescono a comunicare; quando invece comunicano, i messaggi arrivano integri.

Un messaggio che non si può consegnare, perché il destinatario è guasto o sta in un'altra componente, si perde, e nessuno lo viene a sapere: farlo conservare dalla rete sposterebbe soltanto il problema, e un avviso di mancata consegna non sarebbe affidabile, perché la rete non distingue un destinatario che non ha ricevuto da uno che ha ricevuto e si è guastato subito dopo.

6.2.3 Rilevare i guasti con i timeout

Per chi aspetta, un sito guasto e una partizione sono indistinguibili: in entrambi i casi non arrivano risposte. Per accorgersene si usa un **timeout**: A invia un messaggio a B e ne aspetta la risposta per un **periodo di timeout**, che deve coprire l'andata, l'elaborazione e il ritorno; se il periodo scade, A conclude di non poter comunicare con B. Stabilire il periodo è difficile, perché dipende dal carico, dalla rete e dagli orologi, e in pratica si sceglie un valore che funziona quasi sempre.

Se il periodo è troppo corto, A può credere di non comunicare con B anche quando potrebbe: è un ***timeout failure*** (o *performance failure*), che conta come un guasto di comunicazione ma crea situazioni che una partizione non crea, come A che crede di comunicare con B mentre B crede il contrario. Usare un timeout vuol dire supporre un limite ai ritardi, come nei sistemi sincroni della Sezione 1.2, ma quel limite può essere sbagliato: per ogni protocollo bisogna quindi dire se tollera solo i guasti dei siti o anche quelli di comunicazione.

6.3 Il problema del commit atomico

Prima di far eseguire il Commit, il TM deve sapere che ogni sito è pronto a eseguirlo. Un sito lo è se può garantire il Commit di T anche dopo un proprio guasto, cioè se:

- in quel sito T non ha letto valori scritti da transazioni non ancora confermate: se una di queste venisse annullata, andrebbe annullata anche T;
- tutti i valori che T ha scritto in quel sito sono già in memoria stabile. In un sito in cui T ha solo letto, la condizione è vera in automatico.

Dare il consenso è quindi una promessa. L'idea di soluzione è semplice: si fa eseguire il Commit solo dopo il consenso di tutti i siti, ed è il Two-Phase Commit del capitolo 7. La difficoltà sta nei guasti: siti che si fermano a metà, messaggi che si perdono.

Per concentrarsi sul commit, per ogni transazione T immaginiamo un processo in ogni sito in cui T ha lavorato. Quello dell'*home site* è il **coordinatore**, gli altri sono i **partecipanti**: il coordinatore conosce i partecipanti, mentre i partecipanti conoscono il coordinatore ma non necessariamente si conoscono tra loro. (In pratica gli stessi compiti li svolgono pochi processi per sito, condivisi da molte transazioni.) Ogni sito ha anche un **DT log** (*distributed transaction log*), un registro in memoria stabile in cui i processi annotano ciò che devono ricordare: dopo un guasto lo rileggono per sapere a che punto erano.

Ogni processo esprime un **voto**, Yes oppure No, e raggiunge una **decisione**, Commit oppure Abort. Vota Yes solo se il suo sito è pronto a confermare T; quando decide, il suo sito esegue il Commit o l'Abort di T per conto proprio.

Definizione (protocollo di commit atomico). Ogni processo può esprimere esattamente uno dei due voti, Yes o No, e può raggiungere esattamente una delle due decisioni, Commit o Abort. Un **protocollo di commit atomico** (ACP) è un algoritmo con cui i processi raggiungono le decisioni in modo che valgano cinque condizioni:

AC1 tutti i processi che raggiungono una decisione raggiungono la stessa;

AC2 un processo non può cambiare la propria decisione dopo averla raggiunta;

AC3 la decisione Commit si può raggiungere solo se tutti i processi hanno votato Yes;

AC4 se non ci sono guasti e tutti i processi hanno votato Yes, la decisione è Commit;

AC5 in ogni esecuzione in cui capitano solo guasti che l'algoritmo è progettato per tollerare, se a un certo punto tutti i guasti vengono riparati e per un tempo abbastanza lungo non se ne verificano di nuovi, prima o poi tutti i processi raggiungono una decisione.

La definizione merita tre commenti.

- AC1 non chiede che *tutti* decidano: un processo può guastarsi e non ripartire più, e come si vedrà nella Sezione 6.4 non si può pretendere nemmeno che decidano tutti i processi operativi. AC5 chiede invece che tutti decidano una volta riparati i guasti, ed esclude i protocolli che, dopo un guasto qualunque, lasciano i processi indecisi per sempre.
- AC4 esclude i protocolli banali che decidono sempre Abort, che soddisferebbero tutte le altre condizioni.
- AC4 è solo un inverso parziale di AC3: con dei guasti si può decidere Abort anche se tutti hanno votato Yes (Sezione 7.2).

6.4 Incertezza e blocco

Da AC3 segue che un processo che non ha ancora votato Yes può decidere Abort da solo, in qualunque momento: senza il suo Yes nessuno potrà decidere Commit. Dopo aver votato Yes, invece, non può più decidere da solo, perché gli altri potrebbero aver deciso Commit oppure Abort.

Definizione (periodo di incertezza). Il **periodo di incertezza** (*uncertainty period*) di un processo va dal momento in cui vota Yes al momento in cui riceve abbastanza informazioni per sapere quale sarà la decisione. Durante questo periodo il processo è **incerto**: non sa se la decisione sarà Commit o Abort, e

non può decidere Abort da solo.

Se un guasto colpisce un processo incerto, ci sono due scenari.

Scenario I: il processo resta isolato. Un guasto di comunicazione separa il processo incerto p da tutti gli altri, e p non può decidere finché il guasto non viene riparato. Un processo che deve aspettare la riparazione di guasti per andare avanti è **bloccato** (*blocked*). Incerto e bloccato non sono la stessa cosa: un processo incerto non conosce la decisione, ma può scoprirla chiedendo agli altri; uno bloccato è incerto e non può nemmeno scoprirla, finché un guasto non viene riparato. Il blocco può durare un tempo arbitrario, e intanto T resta aperta nel sito di p e tiene occupate risorse, come i *lock* con cui impedisce alle altre transazioni di toccare i suoi dati.

Scenario II: il processo si guasta. p si guasta mentre è incerto, e quando riparte deve chiedere agli altri che cosa è stato deciso. La capacità di decidere da soli quando si riparte dopo un guasto si chiama **recupero indipendente** (*independent recovery*). Rende il recupero più semplice ed economico, e senza di essa un guasto totale porta al blocco: il primo processo a ripartire, se è incerto, non trova nessuno a cui chiedere.

Il periodo di incertezza non si può eliminare: servirebbe che ogni processo votasse e conoscesse i voti di tutti gli altri nello stesso istante. Ne seguono due limiti, che valgono per qualunque protocollo.

Due limiti di ogni protocollo di commit atomico.

- Se sono possibili guasti di comunicazione o guasti totali, *ogni* ACP può far bloccare dei processi: è lo scenario I, o lo scenario II con un guasto totale.
- Nessun ACP può garantire il recupero indipendente ai processi che si guastano: è lo scenario II.

Il primo limite lascia aperta una possibilità: se si possono guastare solo i siti, e mai tutti insieme, esiste un ACP che non blocca mai, il *Three-Phase Commit*, che qui non si tratta.

Capitolo 7

Il protocollo Two-Phase Commit

Il **Two-Phase Commit** (2PC, commit in due fasi) è il protocollo di commit atomico più semplice e più usato. Lo si vede prima senza guasti, poi con i timeout, il recupero dopo un guasto e i costi. Indichiamo con c il coordinatore e con $p_1, \ldots, p_n$ i partecipanti, quindi i processi sono $n+1$; p e q sono partecipanti qualunque. Tre scritture distinguono tre cose con lo stesso nome: il voto Yes è la scelta di un processo, YES (in maiuscoletto) il messaggio che la comunica, `yes` (in carattere macchina) il record che il processo scrive nel DT log. Lo stesso vale per le decisioni Commit e Abort, i messaggi COMMIT e ABORT e i record `commit` e `abort`.

7.1 Il protocollo

Definizione (Two-Phase Commit, senza guasti).

1. Il coordinatore invia a tutti i partecipanti un messaggio VOTE-REQ (*vote request*, richiesta di voto).
2. Quando riceve VOTE-REQ, ogni partecipante risponde con il proprio voto, YES oppure NO. Se vota No, decide subito Abort e si ferma.
3. Il coordinatore raccoglie i voti. Se sono tutti YES e anche il suo voto è Yes (il coordinatore vota per il proprio sito), decide Commit e invia COMMIT a tutti i partecipanti; altrimenti decide Abort e invia ABORT a chi ha votato Yes. Poi si ferma.
4. Ogni partecipante che ha votato Yes aspetta COMMIT o ABORT, decide di conseguenza e si ferma.

I passi 1 e 2 formano la **fase di voto**, i passi 3 e 4 la **fase di decisione**.

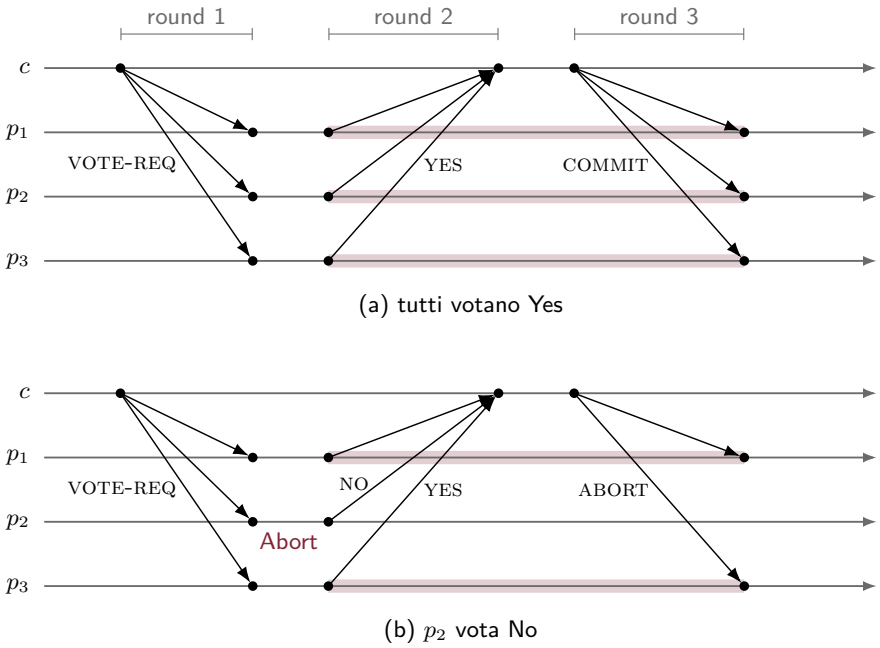


Figura 7.1. Due esecuzioni del 2PC senza guasti, con il coordinatore c e i partecipanti p_1, p_2, p_3. Le fasce rosse sono i periodi di incertezza dei partecipanti. In (a) sono segnati anche i tre round del protocollo (Sezione 7.4).

Esempio (due esecuzioni senza guasti). Nella Figura 7.1(a) tutti votano Yes: ogni partecipante è incerto dall'invio di YES all'arrivo di COMMIT, e c decide Commit appena ha tutti i voti. Nella Figura 7.1(b) p_2 vota No: decide subito Abort, senza mai essere incerto, e c invia ABORT solo a p_1 e p_3, gli unici che non hanno ancora deciso.

Il coordinatore, invece, non è mai incerto: decide nel momento in cui vota, quando conosce già tutti i voti. Senza guasti il 2PC soddisfa le condizioni da AC1 ad AC4: le decisioni vengono tutte dal coordinatore, che decide Commit solo se tutti i voti, compreso il suo, sono Yes, mentre chi vota No decide Abort come il coordinatore; ognuno decide una volta sola; e se non ci sono guasti e tutti votano Yes, la decisione è Commit. Non soddisfa invece AC5, per due motivi:

- un processo che aspetta un messaggio che non arriverà mai aspetterebbe per sempre: servono azioni da eseguire allo scadere di un timeout (Sezione 7.2);
- un processo che riparte dopo un guasto deve decidere in modo coerente con gli altri, quindi deve aver salvato qualcosa nel DT log (Sezione 7.3).

7.2 Timeout e protocollo di terminazione

Quando l'attesa di un messaggio supera il timeout, il processo esegue un'**azione di timeout** (*timeout action*). Nel 2PC si aspetta in tre punti, all'inizio dei passi 2, 3 e 4 (Tabella 7.1).

Tabella 7.1. Le azioni di timeout del 2PC.

Chi aspetta, e che cosa	Allo scadere del timeout	Perché è sicuro
un partecipante aspetta VOTE-REQ (passo 2)	decide Abort e si ferma	non ha ancora votato, quindi può decidere Abort da solo
il coordinatore aspetta i voti (passo 3)	decide Abort e invia ABORT ai partecipanti da cui ha ricevuto YES	non ha ancora deciso, e senza la sua decisione nessun partecipante può aver deciso Commit
un partecipante che ha votato Yes aspetta la decisione (passo 4)	avvia il protocollo di terminazione	è incerto: non può decidere da solo e deve chiedere agli altri

Nei primi due casi chi va in timeout decide Abort da solo. Il secondo mostra perché si può decidere Abort anche se tutti hanno votato Yes: basta che un YES arrivi al coordinatore in ritardo, o non arrivi per una partizione.

Nel terzo caso il partecipante p è incerto e deve consultare altri processi: è il **protocollo di terminazione** (*termination protocol*), cioè ciò che fa un processo incerto quando un messaggio atteso non arriva. Il più semplice è aspettare il coordinatore: prima o poi torna raggiungibile, e conosce sempre la decisione perché non è mai incerto. AC5 è rispettata, ma p può restare bloccato senza bisogno.

Esempio (un blocco inutile). Il coordinatore invia la decisione al partecipante q e si guasta prima di inviarla a p. p è incerto, ma q no: se p potesse chiedere a q, non dovrebbe aspettare che il coordinatore riparta.

Per chiedersi la decisione, i partecipanti devono conoscersi: basta che il coordinatore aggiunga a VOTE-REQ l'elenco dei partecipanti. Prima di riceverlo nessuno ne ha bisogno, perché chi va in timeout aspettando VOTE-REQ decide Abort da solo.

Definizione (protocollo di terminazione cooperativo). Un partecipante p che va in timeout mentre è incerto diventa l'**iniziatore** (*initiator*) del protocollo; i processi a cui si rivolge sono i **rispondenti** (*responder*).

- *Iniziatore.* p invia un messaggio DECISION-REQ (*decision request*, richiesta di decisione) a tutti gli altri processi, coordinatore compreso, e aspetta da uno qualunque di loro una risposta, COMMIT o ABORT. Quando la riceve, decide di conseguenza. Se scade il timeout senza risposte, ricomincia da capo: è bloccato.
- *Rispondente.* Un processo q che riceve DECISION-REQ da p si trova in uno di tre casi:

1. ha già deciso Commit o Abort: invia a p COMMIT o ABORT;
2. non ha ancora votato: decide Abort, come può fare da solo, e invia ABORT a p;
3. ha votato Yes ma non ha ancora deciso: è incerto anche lui, quindi non può aiutare p, e non risponde.

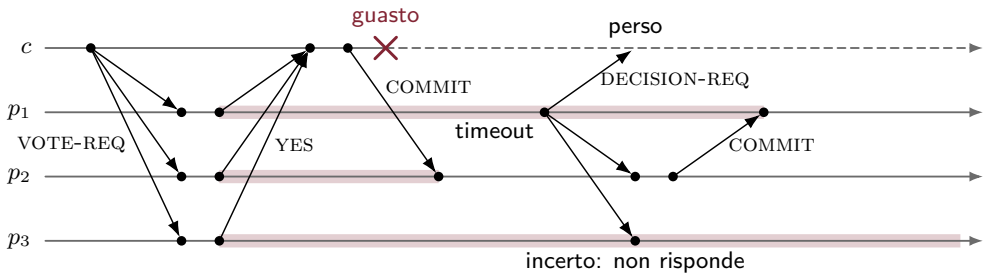


Figura 7.2. Il protocollo di terminazione cooperativo. Il coordinatore c invia COMMIT solo a p_2 e si guasta; p_1 va in timeout e chiede la decisione a tutti, e la riceve da p_2. Le fasce rosse sono i periodi di incertezza.

Esempio (terminazione cooperativa). Nella Figura 7.2 tutti votano Yes; c decide Commit, invia COMMIT solo a p_2 e si guasta. Quando il suo timeout scade, p_1 invia DECISION-REQ a tutti: la richiesta a c si perde, p_3 è incerto e non risponde (caso 3), p_2 risponde COMMIT (caso 1). Così p_1 decide Commit senza aspettare c, e lo stesso farà p_3 al suo timeout.

p resta bloccato solo se tutti i processi che riesce a raggiungere sono incerti, e lo resta finché la riparazione dei guasti non gli fa raggiungere un processo nel caso 1 o 2. Un processo così esiste sempre, il coordinatore, quindi AC5 è rispettata. I blocchi diventano meno probabili, ma non spariscono, come vuole il primo limite della Sezione 6.4: il 2PC si può bloccare anche quando si guastano soltanto i siti.

7.3 Recupero e DT log

Un processo p che riparte dopo un guasto deve arrivare a una decisione coerente con quella degli altri, subito o dopo la riparazione degli altri guasti (AC5). Se ricorda a che punto era:

- se si è guastato prima di inviare YES, decide Abort da solo;
- se aveva già deciso, non deve fare niente;
- se era incerto, non può decidere da solo (scenario II della Sezione 6.4): è come se fosse andato in timeout aspettando la decisione, e avvia il protocollo di terminazione, con il rischio di restare bloccato.

7.3.1 Che cosa scrivere nel DT log

Per ricordare a che punto era, ogni processo scrive dei record nel DT log del proprio sito.

1. Il coordinatore, quando invia i VOTE-REQ, scrive `start-2PC`, con i nomi dei partecipanti, prima o dopo l'invio.
2. Un partecipante che vota Yes scrive `yes`, con i nomi del coordinatore e degli altri partecipanti, *prima* di inviare YES. Se vota No, scrive `abort`, prima o dopo l'invio di NO.
3. Il coordinatore scrive `commit` *prima* di inviare COMMIT.
4. Il coordinatore scrive `abort` quando invia ABORT, prima o dopo l'invio.
5. Un partecipante, ricevuta la decisione, scrive `commit` o `abort`.

Scrivere `commit` o `abort` è l'atto con cui un processo decide; da quel momento il suo sito può eseguire il Commit o l'Abort di T (Figura 7.3).

Quando un sito riparte, dai record di ogni transazione ricava che cosa fare (Tabella 7.2).

Tabella 7.2. Che cosa fa un processo che riparte, in base ai record che trova nel DT log.

Record nel DT log	Situazione al momento del guasto	Che cosa fa
`start-2PC` e `commit` o `abort`	era il coordinatore, e aveva già deciso	niente: la decisione è quella del record
`start-2PC`, ma né `commit` né `abort`	era il coordinatore, e non aveva ancora deciso	decide Abort, scrivendo `abort`
niente `start-2PC`, ma `commit` o `abort`	era un partecipante, e aveva già deciso	niente
né `start-2PC` né `yes`	era un partecipante, e non aveva votato Yes: non aveva ancora votato, oppure aveva votato No senza aver ancora scritto `abort`	decide Abort, scrivendo `abort`
`yes`, ma né `commit` né `abort`	era un partecipante incerto	avvia il protocollo di terminazione, con i nomi dei processi presi dal record `yes`

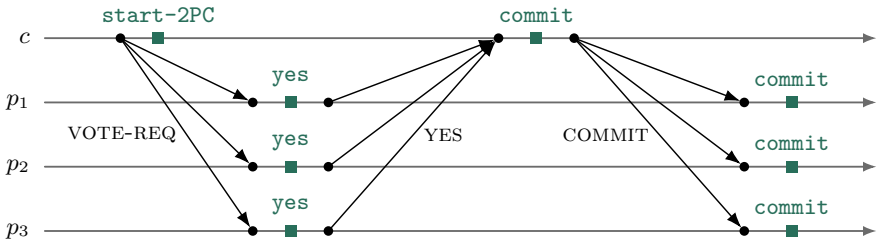


Figura 7.3. I record scritti nel DT log durante l'esecuzione della Figura 7.1(a): ogni quadratino verde è una scrittura. I partecipanti scrivono `yes` prima di inviare YES, il coordinatore scrive `commit` prima di inviare COMMIT.

Due record devono precedere il messaggio corrispondente, e la tabella mostra perché.

- *`commit` prima di* COMMIT. Altrimenti il coordinatore potrebbe inviare COMMIT e guastarsi prima di scrivere il record: ripartendo con il solo `start-2PC` deciderebbe Abort, mentre i partecipanti hanno deciso Commit, e AC1 sarebbe violata.
- *`yes` prima di* YES. Altrimenti un partecipante potrebbe inviare YES e guastarsi prima di scrivere il record: ripartendo senza `yes` deciderebbe Abort, mentre il coordinatore potrebbe aver deciso Commit.

Per gli altri record l'ordine è libero. Chi riparte senza aver deciso e non trova un record che lo impegni decide Abort, quindi un record che porta ad Abort può mancare senza danni; uno che impegna verso il Commit, invece, deve essere in memoria stabile prima che gli altri ci possano contare.

7.3.2 Il protocollo completo

Mettendo insieme i passi della Sezione 7.1, le azioni di timeout e le regole del DT log si ottiene il protocollo completo.

Definizione (Two-Phase Commit, protocollo completo). *Coordinatore.*

1. Invia VOTE-REQ a tutti i partecipanti, con l'elenco dei partecipanti, e scrive `start-2PC` nel DT log.
2. Aspetta i voti di tutti i partecipanti. Se scade il timeout, scrive `abort`, invia ABORT ai partecipanti da cui ha ricevuto YES e termina.
3. Se tutti i voti sono YES e anche il suo voto è Yes, scrive `commit` e *poi* invia COMMIT a tutti i partecipanti. Altrimenti scrive `abort` e invia ABORT ai partecipanti da cui ha ricevuto YES.

Partecipante.

1. Aspetta VOTE-REQ dal coordinatore. Se scade il timeout, scrive `abort` e termina.
2. Se vota Yes, scrive `yes`, *poi* invia YES al coordinatore e aspetta COMMIT o ABORT. Se scade il timeout, avvia il protocollo di terminazione cooperativo;

altrimenti scrive `commit` o `abort`, secondo il messaggio ricevuto.

3. Se vota No, scrive `abort` e invia NO al coordinatore.

Anche nel protocollo di terminazione decidere vuol dire scrivere `commit` o `abort`.

7.3.3 Più transazioni e ripresa di un sito

Il DT log di un sito contiene i record di molte transazioni, quindi ogni record porta il nome della sua. Per cancellare i record che non servono più (*garbage collection*) valgono due regole:

GC1 un sito non può cancellare i record di T dal proprio DT log prima di aver eseguito il Commit o l'Abort di T;

GC2 almeno un sito deve conservare i record di T finché non sa che il Commit o l'Abort di T è stato eseguito in tutti gli altri siti in cui T ha lavorato.

Senza GC2 un processo che riparte incerto potrebbe non trovare più nessuno che conosca la decisione, e AC5 sarebbe violata. GC1 si rispetta con le sole informazioni locali; GC2 richiede che ogni sito confermi di aver eseguito l'esito ai siti che conservano i record. Al limite li conserva solo il coordinatore, oppure tutti i siti.

Quando riparte, un sito deve completare il protocollo per le transazioni ancora indecise, ma non conviene che aspetti quelle bloccate prima di tornare a lavorare. Decide quelle che può, riprende a servire le altre transazioni e protegge i dati di quelle bloccate rimettendo i loro *lock* in scrittura. Quelli in lettura non servono più: una transazione bloccata ha finito di leggere, e conta solo che nessuno legga o sovrascriva i valori che ha scritto, che potrebbero ancora essere annullati. Poiché le tabelle dei *lock* stanno in memoria volatile, i *lock* in scrittura di T si annotano nel record `yes`, a meno che il sito non li possa ricavare dal proprio log.

7.4 Valutazione del 2PC

Un ACP si valuta secondo quattro criteri:

- **resilienza**: quali guasti tollera?
- **blocco**: i processi si possono bloccare, e in quali condizioni?
- **complessità in tempo**: quanto ci vuole per arrivare alla decisione?
- **complessità in messaggi**: quanti messaggi servono per arrivare alla decisione?

I primi due misurano l'affidabilità, gli altri due l'efficienza. Migliorare l'una costa nell'altra, e in ogni caso conviene ottimizzare il caso senza guasti, che si spera sia la normalità.

Round. La complessità in tempo si misura in **round**: un round è il tempo massimo che un messaggio impiega ad arrivare, lo stesso limite su cui si reggono i timeout. Due messaggi stanno in round diversi se e solo se uno non può partire prima che l'altro sia arrivato: nel 2PC un YES e un COMMIT stanno in round diversi, mentre tutti i VOTE-REQ stanno nello stesso round, e così tutti i COMMIT. Per contare i

round si immagina che i messaggi di un round partano insieme e arrivino insieme, come nella Figura 7.1(a). Si trascurano il tempo di elaborazione, le scritture in memoria stabile e il costo di preparare le copie di un *broadcast* (l'invio dello stesso messaggio a molti destinatari), che a volte contano. La complessità in messaggi si misura contando i messaggi, che qui sono tutti brevi.

Resilienza. Il 2PC tollera sia i guasti dei siti sia quelli di comunicazione, partizioni o *timeout failure* che siano, perché le azioni di timeout della Tabella 7.1 sono sicure qualunque sia la causa del timeout.

> **Timeout sbagliati.** Le condizioni da AC1 ad AC4 non dipendono mai dalla durata del timeout: un timeout troppo corto può al più far decidere Abort quando si poteva confermare, o avviare il protocollo di terminazione quando non serviva. I timeout servono solo a non aspettare per sempre (AC5), e per questo le decisioni del 2PC restano coerenti anche in un sistema asincrono, dove nessun periodo di timeout è sicuro (Sezione 1.2).

Blocco. Un processo si blocca se va in timeout mentre è incerto e riesce a raggiungere solo processi incerti; succede anche quando si guastano soltanto i siti.

Complessità in tempo. Senza guasti servono tre round: i VOTE-REQ, i voti, la decisione. Il protocollo di terminazione ne aggiunge al più due, la richiesta e la risposta, anche se lo avviano più partecipanti, perché i loro round si sovrappongono. Quindi tutti i processi né guasti né bloccati decidono entro cinque round, *qualunque sia il numero dei guasti*; i processi bloccati, che possono aspettare senza limite, si escludono dal conto.

Complessità in messaggi. Senza guasti il 2PC usa al più $3n$ messaggi, dove n è il numero dei partecipanti: in ognuno dei tre round ne partono al più n. Il protocollo di terminazione costa molto di più. Nel caso peggiore tutti i partecipanti vanno in timeout mentre sono incerti, e ognuno invia DECISION-REQ agli altri n processi: le sole richieste sono n^2, e con le risposte di chi ha già deciso si arriva al più a $n(3n+1)/2$ messaggi. Il costo cresce quindi con il quadrato del numero dei partecipanti.

Capitolo 8

Consenso e Paxos

Nel commit atomico i processi dovevano prendere la stessa decisione, Commit o Abort. Il **consenso** è un problema di accordo più generale: un gruppo di processi deve accordarsi su un valore, anche quando alcuni si guastano. Questo capitolo presenta **Paxos**, il protocollo di consenso più noto: le sue regole, perché non sceglie mai due valori diversi e quando riesce davvero a sceglierne uno.

8.1 Il problema del consenso

Ogni processo può proporre un valore; tra i valori proposti se ne deve scegliere uno solo, che poi tutti devono poter conoscere.

> **Esempio (una cartella condivisa).** Google usa Paxos nei sistemi che conservano i dati dei suoi servizi. Prendiamo, molto semplificata, una cartella condivisa di Google Drive. I suoi dati, come i nomi dei file, stanno su più server in data center diversi, così la cartella resta disponibile anche se un server si guasta. Ogni modifica è un'operazione che tutte le copie devono applicare, e nello stesso ordine. Anna e Bruno rinominano nello stesso momento il file `Relazione.pdf`: Anna in `Tesi.pdf`, Bruno in `Bozza.pdf`. Se un server applicasse prima la rinomina di Anna e un altro prima quella di Bruno, alla fine il primo mostrerebbe `Bozza.pdf` e il secondo `Tesi.pdf`: Anna e Bruno vedrebbero nomi diversi a seconda del server che risponde loro. I server devono quindi accordarsi su quale rinomina applicare per prima, cioè sceglierne una tra le due proposte.

Il consenso non cerca di stabilire chi ha cliccato prima. In un sistema asincrono, senza un orologio comune, non lo si può sapere con certezza (capitolo 3), e del resto ad Anna e a Bruno basta vedere lo stesso nome. Si chiede solo di scegliere una delle due rinomine, una sola e la stessa per tutti. Quale vinca non è deciso in anticipo: dipende da come vanno i messaggi, cioè da quale proposta raccoglie per prima abbastanza voti. In pratica un solo server raccoglie le richieste e le propone una alla volta, quindi vince di solito quella che gli arriva prima, e l'altra si applica subito dopo, come operazione successiva. Se l'ordine reale contasse davvero, come per l'ultimo posto su un volo, il consenso non lo garantirebbe: garantisce un ordine su cui tutti sono d'accordo, e il posto va a chi viene primo in quell'ordine.

I compiti sono divisi fra tre ruoli:

- un ***proposer*** propone un valore;
- un ***acceptor*** vota i valori proposti;
- un ***learner*** viene a sapere quale valore è stato scelto.

Un processo può avere più ruoli, e di solito li ha tutti e tre: nella cartella condivisa ogni server è proposer quando riceve la richiesta di un utente, acceptor quando vota e learner quando deve sapere quale rinomina applicare. Gli acceptor sono in numero fisso, n; proposer e learner possono essere quanti si vuole.

Il sistema è asincrono (Sezione 1.2): un messaggio può impiegare un tempo qualunque ad arrivare. I processi si guastano fermandosi (*crash*): un processo guasto smette di funzionare, ma non esegue mai azioni sbagliate.

Definizione (consenso). Un protocollo risolve il consenso se valgono tre proprietà di ***safety*** (sicurezza), chiamate CS1–CS3 da *consensus safety*:

CS1 si può scegliere solo un valore che è stato proposto;

CS2 si sceglie un solo valore;

CS3 un learner **corretto**, cioè non guasto, può apprendere solo un valore scelto;

e una proprietà di ***liveness***: prima o poi un valore viene scelto, e i learner corretti lo apprendono.

Una proprietà di safety dice che non succede mai niente di male; una di liveness, che prima o poi succede qualcosa di buono. Servono tutte e due: un semaforo che non dà mai il verde insieme a due strade che si incrociano rispetta la safety, ma la rispetta anche un semaforo sempre rosso, che però non ha liveness. Nel commit atomico AC1–AC4 erano proprietà di safety, e AC5 di liveness.

Nella cartella condivisa la safety vuol dire che si sceglie una delle due rinomine chieste (CS1), una sola (CS2), e che nessun server mostra un nome che non sia stato scelto (CS3); la liveness, che prima o poi una delle due ha effetto. Paxos garantisce la safety qualunque sia il numero dei guasti; la liveness, solo se funziona una maggioranza degli acceptor e un solo proposer alla volta prova a far scegliere un valore.

Consenso e commit atomico. Nel bonifico tra due banche (Sezione 6.1) i due siti hanno dati diversi, i due conti, e devono prendere la stessa decisione, Commit o Abort. Ognuno può rifiutare, perché il Commit richiede il Yes di tutti, e un sito incerto può restare bloccato ad aspettare chi si è guastato. Nella cartella condivisa, invece, i server hanno copie degli stessi dati e devono scegliere una delle operazioni proposte: nessuno ha diritto di veto e basta una maggioranza, quindi il guasto di una minoranza di server non impedisce la scelta.

8.2 Decidere a maggioranza

Paxos decide a maggioranza. Un **quorum** è un insieme di $n - f$ acceptor, dove

$$f = \left\lfloor \frac{n-1}{2} \right\rfloor$$

è il numero di guasti (*failures*) che il protocollo tollera: il più grande numero di acceptor che si possono togliere lasciandone una maggioranza ($\lfloor \cdot \rfloor$ arrotonda per difetto). Con $n = 3$ si ha $f = 1$, e un quorum ha 2 acceptor; con $n = 7$ si ha $f = 3$, e un quorum ne ha 4. Per tollerare f guasti servono quindi $2f + 1$ acceptor: perché la cartella condivisa resti modificabile anche con due data center fuori uso, servono cinque copie.

Perché proprio una maggioranza? Pensa al numero legale di un'assemblea di cinque persone, in cui una votazione vale solo se partecipano almeno in tre. Due votazioni valide non possono essere fatte da persone tutte diverse: se alla prima partecipano 1, 2 e 3, fuori restano solo 4 e 5, che da sole non bastano, quindi alla seconda partecipa per forza almeno uno fra 1, 2 e 3. Con i numeri: servirebbero $3 + 3 = 6$ posti, ma le persone sono 5. Chi partecipa a entrambe le votazioni fa da testimone: ricorda com'è andata la prima, e può riferirlo durante la seconda.

In Paxos le due «votazioni» sono il voto con cui una maggioranza sceglie un valore e, più tardi, la domanda «che cosa hai votato?» fatta a una maggioranza; il testimone è l'acceptor che sta in entrambi i gruppi. Prendiamo la cartella su cinque server, $S_1, \ldots, S_5$, con gruppi di tre.

- Se S_1, S_2 e S_3 salvano `Bozza.pdf`, e più tardi alla domanda del server di Anna rispondono S_3, S_4 e S_5, S_3 dice «`Bozza.pdf`»: chiunque raccolga tre risposte trova almeno un voto per la rinomina scelta.
- Se bastasse una minoranza, per esempio due server, due gruppi potrebbero non avere nessuno in comune, come $\{S_1, S_2\}$ e $\{S_3, S_4\}$. Il primo potrebbe scegliere `Tesi.pdf` e il secondo `Bozza.pdf`, e chi ricevesse risposta solo da S_4 e S_5 non saprebbe niente della scelta del primo.
- Se invece servissero tutti e cinque, basterebbe un server guasto per bloccare tutto, come nel 2PC. Con tre, la cartella resta modificabile anche con due server guasti.

In generale un quorum contiene più della metà degli acceptor, quindi due quorum insieme ne contano più di n: $(n - f) + (n - f) > n$. Poiché gli acceptor sono solo n, almeno uno sta in entrambi. Tutto il protocollo si regge su due fatti:

- due quorum hanno sempre almeno un acceptor in comune, il testimone;
- gli f acceptor fuori da un quorum non bastano a formarne un altro, perché $f < n - f$.

8.3 Un round, passo per passo

Paxos procede per **round**: un round è un tentativo, identificato da un numero, di far scegliere un valore, e di solito ne basta uno. Questi round non c'entrano con quelli della Sezione 7.4, che misuravano il tempo.

Che cosa ricorda ogni processo. Un proposer ricorda il round che sta eseguendo, *crnd* (*current round*), e il valore che vi propone, *cval* (*current value*). Ogni acceptor ricorda tre cose:

- *rnd* (*round*): il round più alto a cui ha promesso di partecipare;
- *vrnd* (*voted round*): il round del suo ultimo voto;
- *vval* (*voted value*): il valore del suo ultimo voto.

All'inizio queste variabili valgono -1, che vuol dire «nessuno»: nessun round, nessun voto.

Definizione (Paxos). Un round i si svolge in cinque passi.

1. *Prepare.* Un proposer sceglie un suo round i, più alto di quelli che ha già usato, pone $crnd = i$ e invia $\langle \textsc{prepare}, i \rangle$ a tutti gli acceptor.
2. *Promise.* Un acceptor che riceve $\langle \textsc{prepare}, i \rangle$ con $i > rnd$ pone $rnd = i$ e risponde $\langle \textsc{promise}, i, vrnd, vval \rangle$: promette di non votare più in nessun round minore di i, e comunica il suo ultimo voto. Se $i \leq rnd$, ignora il messaggio.
3. *Accept.* Quando ha le promesse di $n - f$ acceptor, il proposer sceglie il valore *cval* da proporre:
   - se nessuno di loro ha mai votato, il valore che voleva proporre;
   - altrimenti, il valore votato nel round più alto, cioè il *vval* della promessa con il *vrnd* più alto. È uno solo, perché in quel round si è proposto un solo valore.

   Poi invia $\langle \textsc{accept}, i, cval \rangle$ a tutti gli acceptor.
4. *Learn.* Un acceptor che riceve $\langle \textsc{accept}, i, v \rangle$, dove v è il valore proposto, con $i \geq rnd$ vota v nel round i: pone $rnd = vrnd = i$ e $vval = v$, e invia $\langle \textsc{learn}, i, v \rangle$ a tutti i learner. Se $i < rnd$, ignora il messaggio.
5. *Scelta.* Un learner che riceve $\langle \textsc{learn}, i, v \rangle$ da $n - f$ acceptor diversi, con lo stesso round i e lo stesso valore v, apprende che v è stato scelto.

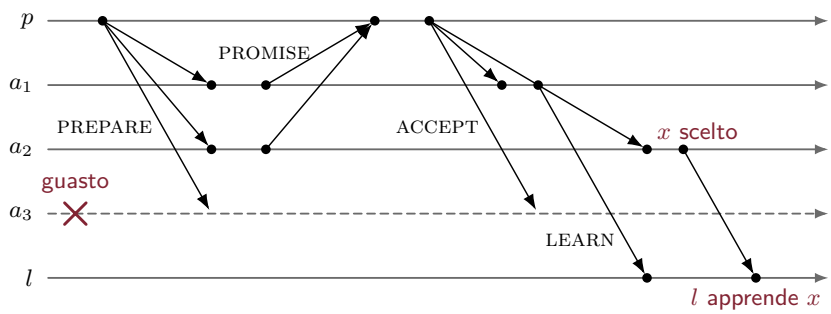


Figura 8.1. Un round di Paxos con il proposer p, tre acceptor e il learner l. a_3 è guasto, e i messaggi che gli arrivano si perdono.

Seguiamo un round nella cartella condivisa, con la Figura 8.1. Gli acceptor sono $n = 3$, quindi un quorum ne ha 2, e a_3 è guasto. Il proposer p è il server che ha ricevuto la richiesta di Anna e vuole far scegliere x, la rinomina in `Tesi.pdf`; il learner l è il server da cui legge Bruno.

1. *p invia $\langle \text{PREPARE}, 1\rangle$ a tutti, ma aspetta solo una maggioranza.* Non può aspettare tutte e tre le risposte: a_3 è guasto, e aspettarlo vorrebbe dire aspettare per sempre. Né può accorgersi che a_3 è guasto, perché in un sistema asincrono un processo guasto non si distingue da uno lento. Per questo aspetta solo le prime $n - f$ risposte, come un'assemblea che comincia appena c'è il numero legale, senza aspettare gli assenti. Gli bastano, qualunque siano: fra loro c'è sempre il testimone della Sezione 8.2, che conta quando qualcuno ha già votato. Se arrivano altre risposte non fanno danni, ma p non le aspetta.

2. *a_1 e a_2 promettono.* Entrambi rispondono $\langle \text{PROMISE}, 1, -1, -1\rangle$: -1 vuol dire che non hanno mai votato. In questo round la promessa e il voto riportato non cambiano niente, perché nessuno ha mai votato; diventano decisivi quando un round va ripetuto, come nella prossima sezione.

3. *p propone x.* Nessuna promessa riporta un voto, quindi p è libero di proporre il proprio valore, e invia $\langle \text{ACCEPT}, 1, x\rangle$ a tutti.

4. *a_1 e a_2 votano x.* Qui vanno tenute distinte tre parole. Un acceptor **vota** un valore in un round. Un valore è **scelto** quando lo ha votato un quorum di acceptor nello stesso round: x è scelto nel momento in cui vota a_2, il secondo. Un learner lo **apprende** quando sa che è stato scelto. In quel momento, però, non lo sa nessuno: a_2 sa solo come ha votato lui. Scelto, quindi, non vuol dire saputo. Nella cartella un voto è la rinomina salvata da un server: se l'avesse salvata solo a_1, e a_1 si guastasse, la rinomina si potrebbe perdere.

5. *l conta i* LEARN. Ogni acceptor che vota manda $\langle \text{LEARN}, 1, x\rangle$ a tutti i learner. Il primo LEARN, quello di a_1, non basta: è un voto, non una decisione. Quando arriva il secondo, l ha i voti di una maggioranza nello stesso round, e apprende che x è scelto. Solo da quel momento Bruno vede `Tesi.pdf`, perché un server mostra agli utenti solo ciò che ha appreso.

6. *a_3 non vota mai, e non importa.* La decisione è presa nel momento in cui vota la maggioranza, ed è definitiva: non serve l'unanimità. Quando a_3 riparte non vota per adeguarsi, ma viene a sapere la decisione dagli altri, come gli assenti di un'assemblea leggono il verbale.

8.4 Quando bisogna riprovare

Nella Sezione 8.3 tutto è andato bene, e un round è bastato. Un round, però, può non concludersi, o concludersi senza che qualcuno lo sappia. Succede, per esempio, se:

- il proposer si guasta a metà del round;
- i messaggi sono in ritardo o si sono persi;
- il round viene interrotto da un round più alto di un altro proposer;

- un acceptor si guasta prima di votare, e il valore resta senza maggioranza: il round non sceglie niente, come una votazione senza numero legale;
- cambia il server che coordina, e quello nuovo non sa che cosa ha fatto il precedente;
- un learner non riceve abbastanza LEARN.

In tutti questi casi un proposer, scaduto un timeout, riprova con un round nuovo. Il pericolo è che chi riprova non sa se la decisione c'è già, perché scelto non vuol dire saputo. Il round nuovo deve quindi ritrovare un'eventuale decisione e confermarla, mai crearne una seconda: è come premere di nuovo «Paga» quando la pagina si blocca, che non deve far pagare due volte.

Round di proposer diversi. Ora i round possono essere avviati da proposer diversi, e ogni numero di round appartiene a uno solo di loro: così in ogni round si propone un solo valore, quello del suo proposer. I numeri si alternano: con due proposer p_1 usa i round dispari e p_2 quelli pari; con tre, p_1 usa $1, 4, 7, \ldots$, p_2 usa $2, 5, 8, \ldots$ e p_3 usa $3, 6, 9, \ldots$ Non si dividono a blocchi, come $1, 2, 3$ a p_1 e $4, 5, 6$ a p_2, perché i round non hanno un limite: ognuno deve poter trovare sempre un numero più alto di quelli degli altri. In alternativa si usano coppie formate da un contatore e dal nome del processo, confrontate prima per contatore. Ogni proposer usa i suoi round in ordine crescente.

Un altro proposer subentra. Supponiamo che il proposer p_1, il server di Anna, esegua il round 1 con il valore x, la rinomina in `Tesi.pdf`, e si guasti a metà, dopo aver inviato $\langle \text{ACCEPT}, 1, x \rangle$ solo ad alcuni acceptor. Intanto il server di Bruno, p_2, avvia il round 2 per proporre il suo valore y, la rinomina in `Bozza.pdf`. Forse x è già stato scelto, e qualche server mostra già `Tesi.pdf`, o lo sarà quando arriveranno i messaggi ancora in viaggio: se il round 2 scegliesse y, utenti diversi vedrebbero nomi diversi per lo stesso file, e CS2 sarebbe violata. Ma p_2 non sa che cosa è successo nel round 1, né può prevedere quali voti arriveranno ancora.

I passi Prepare e Promise del round 2 risolvono il problema con un **inventario**: prima di proporre, p_2 chiede a tutti che cosa hanno votato, aspetta le risposte di una maggioranza e, se qualcuno ha già votato, propone il valore votato nel round più alto invece del proprio. Ma l'inventario serve solo se, dopo le risposte, il passato non cambia più. È come lo spoglio di un'elezione, che si fa dopo aver chiuso le urne: altrimenti una scheda arrivata più tardi potrebbe cambiare un risultato già annunciato. La promessa chiude le urne. Rispondendo a $\langle \text{PREPARE}, 2 \rangle$, un acceptor dice «questo è il mio ultimo voto, e da adesso non voto più nei round minori di 2», così la risposta che ha dato resta vera mentre p_2 ci conta. Con le promesse, p_2 non deve prevedere i voti che potrebbero ancora arrivare nei round vecchi: li impedisce.

Si chiudono solo i round più bassi perché ogni round guarda indietro. Il proposer del round 2 si occupa del round 1: lo ispeziona e lo chiude. Dei round successivi non deve preoccuparsi: chi avvierà il round 3 farà a sua volta l'inventario, e ispezionerà e chiuderà anche il round 2. Ogni round si costruisce sopra quelli precedenti, e l'unico pericolo è un round vecchio che si «risveglia» dopo che uno nuovo ha deciso guardando un inventario in cui quel voto non c'era.

Ora si capisce anche che cosa vuol dire che un round è *più nuovo* di un altro: ha il numero più alto. È una convenzione, non un orario. Tra proposer diversi non c'è nessun ordine: p_2 può avviare il round 2 alle 10:00 e p_1 il round 1 alle 10:01. Il round 2 è comunque più nuovo, e gli acceptor che gli hanno già promesso non votano più nel round 1, quindi p_1 riprova con il suo round successivo, il 3. Non serve un orologio, che in un sistema asincrono non sarebbe affidabile: tutti confrontano gli stessi numeri e sono sempre d'accordo sull'ordine, come con i clock logici del capitolo 3. È come in un'asta, dove conta l'importo dell'offerta e non l'ora in cui arriva. Diversamente da un'asta, però, il round più alto vince solo il diritto di proporre: con l'inventario deve riproporre un valore che può essere già stato scelto.

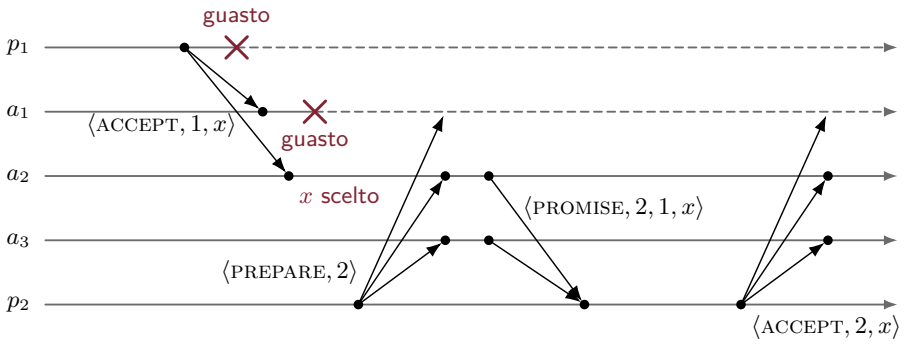


Figura 8.2. Il proposer p_1 si guasta dopo aver inviato $\langle$ACCEPT, $1, x\rangle$ ad a_1 e ad a_2, e a_1 si guasta dopo aver votato x; p_2 avvia il round 2. Qui a_2 riceve $\langle$ACCEPT, $1, x\rangle$ prima di $\langle$PREPARE, $2\rangle$. I learner non sono disegnati.

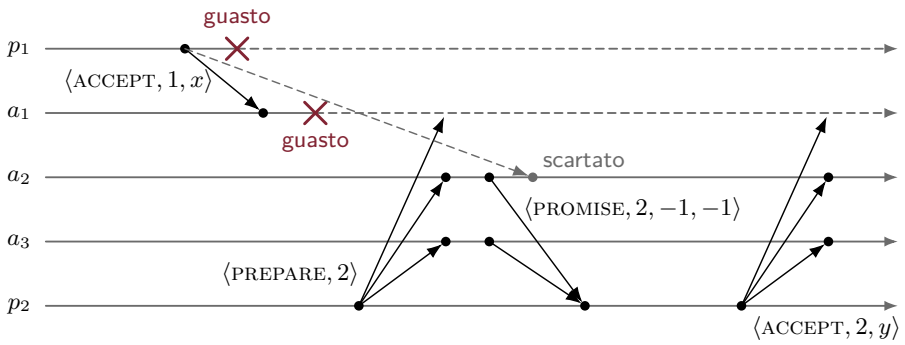


Figura 8.3. La stessa situazione della Figura 8.2, ma a_2 riceve $\langle$PREPARE, $2\rangle$ prima di $\langle$ACCEPT, $1, x\rangle$, che arriva in ritardo e viene scartato.

Esempio (due esecuzioni a confronto). Gli acceptor sono tre, quindi un quorum ne ha due. In entrambe le esecuzioni p_2 riceve le promesse di a_2 e di a_3, che non ha mai votato e risponde $\langle$PROMISE, $2, -1, -1\rangle$.

- Nella Figura 8.2 a_2 riceve $\langle$ACCEPT, $1, x\rangle$ prima di $\langle$PREPARE, $2\rangle$, e vota x: con i voti di a_1 e a_2, x è scelto nel round 1. Nella promessa per il round 2, a_2 riporta il suo voto, $\langle$PROMISE, $2, 1, x\rangle$, quindi p_2 propone x invece di y: la cartella mostrerà `Tesi.pdf`.

- Nella Figura 8.3 a_2 riceve prima $\langle \text{PREPARE}, 2\rangle$: promette, e quando arriva $\langle \text{ACCEPT}, 1, x\rangle$ lo scarta. x resta votato dal solo a_1, e non sarà mai scelto, perché a_2 e a_3 hanno promesso di non votare più nel round 1. Nessuna promessa riporta un voto, quindi p_2 propone il suo y, che viene scelto: la cartella mostrerà `Bozza.pdf`, e la rinomina di Anna potrà essere applicata dopo, come operazione successiva.

In entrambe le esecuzioni si sceglie un solo valore, e per lo stesso motivo: a_2 sta sia nel quorum che potrebbe scegliere x nel round 1, $\{a_1, a_2\}$, sia in quello che promette a p_2, $\{a_2, a_3\}$. Se ha votato x prima di promettere, lo riferisce a p_2; se ha promesso prima, non vota più x, e il round 1 non può più sceglierlo. Senza la promessa, nella Figura 8.3 a_2 voterebbe x all'arrivo del messaggio in ritardo, e verrebbero scelte sia `Tesi.pdf`, nel round 1, sia `Bozza.pdf`, nel round 2.

Nel dubbio, la mossa sicura. Un proposer non sa mai se il voto che vede ha portato a una scelta. Nella Figura 8.2, dalla promessa di a_2 il proposer p_2 sa solo che a_2 ha votato x nel round 1. Non sa se anche a_1 l'ha votato, e quindi x ha la maggioranza ed è scelto, o se l'ha votato il solo a_2, e x non è scelto. Per lui le due situazioni sono identiche: riceve la stessa promessa, e a_1 è guasto, quindi non può chiedergli niente. Non gli serve saperlo, come mostra la Tabella 8.1.

Tabella 8.1. Le due mosse possibili di p_2 nelle due situazioni che non sa distinguere.

	x era già scelto	x non era scelto
p_2 propone x	va bene: conferma la decisione	va bene: x viene scelto adesso, ed è un valore proposto
p_2 propone y	disastro: sono scelti sia x sia y	va bene

Proporre x va bene in entrambe le situazioni; proporre y va bene solo se x non era scelto, e altrimenti viola CS2. Siccome p_2 non sa in quale situazione si trova, sceglie la mossa sicura in tutte e due: adotta x. Se x non era scelto, la rinomina di Bruno perde un'occasione, ma non è un problema: non è stabilito in anticipo chi deve vincere, e la rinomina di Bruno si applicherà dopo. Conta solo che la decisione sia una e la stessa per tutti. Lo stesso ragionamento vale per un round rimasto senza maggioranza: con tre acceptor, se a_2 si guasta prima di votare e il valore z ha il solo voto di a_1, l'unica maggioranza rimasta è $\{a_1, a_3\}$, e l'inventario del round nuovo trova il voto di a_1 e ripropone z.

Esempio (un round dopo la scelta). La cartella ha cinque copie, sui server $S_1, \ldots, S_5$, e un quorum ne ha tre.

1. Nel round 2, S_1, S_2 e S_3 votano `Bozza.pdf`, che quindi è scelta. S_4 aveva votato `Tesi.pdf` nel round 1, e S_5 non ha votato niente.
2. I messaggi LEARN non sono ancora arrivati al server di Anna, che, scaduto il timeout, riprova con il round 3: chiede a tutti che cosa hanno votato e

procede con le prime tre risposte.

3. Fra quelle tre risposte c'è per forza uno di S_1, S_2, S_3, perché $3 + 3 > 5$, e riporta il voto per `Bozza.pdf` nel round 2. Il voto di S_4 per `Tesi.pdf` è del round 1, più vecchio, e non conta: il server di Anna propone `Bozza.pdf`. Se anche S_4 e S_5 votano nel round 3, votano `Bozza.pdf`.

4. Se S_1 e S_2 fossero guasti, risponderebbero S_3 (`Bozza.pdf`, round 2), S_4 (`Tesi.pdf`, round 1) e S_5 (nessun voto), e dalle risposte non si capirebbe se `Bozza.pdf` è stata scelta. Non serve: proporla va bene in entrambi i casi, come nella Tabella 8.1. L'inventario non dice qual è la decisione, ma che cosa si può proporre senza rischi, e aspettare anche S_1 e S_2 non aiuterebbe, perché sono guasti e il dubbio resterebbe.

5. Intanto arrivano i LEARN del round 2, in ritardo. Il server di Anna, come learner, non li scarta perché sono vecchi: li conta, sono tre e dello stesso round, quindi apprende che ha vinto `Bozza.pdf`. Che come proposer abbia già avviato un round più alto non conta, perché i due ruoli sono indipendenti. Abbandona il round 3, ormai inutile, senza danni: chi vi ha già votato ha votato `Bozza.pdf`.

6. Non riprova `Tesi.pdf` in questa decisione, che è chiusa per sempre: qualunque round nuovo dovrebbe riproporre `Bozza.pdf`. La propone per la posizione successiva nella sequenza delle operazioni: Paxos decide un valore alla volta, e ogni posizione è un consenso separato, con i suoi round. Se la seconda decisione sceglie `Tesi.pdf`, tutti i server applicano prima `Bozza.pdf` e poi `Tesi.pdf`, e alla fine il file si chiama `Tesi.pdf` ovunque.

Un learner in ritardo. Resta S_4, a cui dei LEARN del round 2 è arrivato, per esempio, solo quello di S_1. S_4 è in ritardo, non in disaccordo: la decisione c'è, e quando la apprenderà sarà `Bozza.pdf`, perché si apprende solo un valore scelto (CS3) e il valore scelto è uno solo (CS2). Intanto non applica niente e continua a mostrare `Relazione.pdf`, non il `Tesi.pdf` che pure aveva votato: un server mostra solo ciò che ha appreso, non ciò che ha votato. È la differenza della Sezione 1.1 tra uno stato obsoleto e uno inconsistente. Per recuperare ha tre strade:

- aspettare i LEARN in ritardo;
- chiedere a un learner che sa già, di cui può fidarsi, perché i processi si guastano ma non mentono;
- avviare un round nuovo, come il server di Anna: l'inventario ritrova `Bozza.pdf`, gli acceptor la rivotano, e i LEARN del nuovo round gliela fanno apprendere.

Chiedere agli acceptor che cosa hanno votato, invece, non sempre basta: se qualcuno è guasto il dubbio resta, come al passo 4 dell'esempio. Il round nuovo funziona sempre, se si guastano al più f acceptor, perché non ha bisogno di trovare qualcuno che sappia. Nel 2PC, invece, un partecipante che non riceve la decisione può solo chiederla agli altri, e resta bloccato se tutti quelli che raggiunge sono incerti (Sezione 7.2).

Per una stessa decisione, quindi, i round sono di solito uno, qualcuno in più dopo un guasto o un ritardo, e potenzialmente infiniti solo quando due proposer continuano a interrompersi a vicenda: un problema che si risolve facendo avviare i round a un solo proposer.

8.5 Le regole sui round, caso per caso

Le regole dei passi Promise e Learn confrontano il round del messaggio con *rnd*. Si può pensare a *rnd* come a una sbarra: l'acceptor rifiuta tutto ciò che riguarda round più bassi, e la sbarra si sposta solo in avanti. La Tabella 8.2 segue un acceptor che riceve sette messaggi, in quest'ordine.

Tabella 8.2. Un acceptor che riceve sette messaggi, in quest'ordine. All'inizio $rnd = -1$.

	Messaggio ricevuto	Confronto	Che cosa fa	*rnd* dopo
1	$\langle \textsc{prepare}, 2 \rangle$	$2 > -1$	promette: $\langle \textsc{promise}, 2, -1, -1 \rangle$	2
2	$\langle \textsc{accept}, 2, x \rangle$	$2 \geq 2$	vota x nel round 2	2
3	$\langle \textsc{prepare}, 5 \rangle$	$5 > 2$	promette: $\langle \textsc{promise}, 5, 2, x \rangle$	5
4	$\langle \textsc{accept}, 3, y \rangle$ in ritardo	$3 < 5$	lo scarta	5
5	$\langle \textsc{prepare}, 4 \rangle$	$4 < 5$	lo ignora	5
6	$\langle \textsc{prepare}, 5 \rangle$ di nuovo	$5 = 5$	lo ignora	5
7	$\langle \textsc{accept}, 7, z \rangle$ senza $\langle \textsc{prepare}, 7 \rangle$	$7 \geq 5$	vota z nel round 7	7

- *Righe 1–3.* È il funzionamento normale. Per votare nel round a cui ha appena promesso basta $i = rnd$, per questo l'ACCEPT chiede $i \geq rnd$. Per promettere serve invece un round strettamente più alto, $i > rnd$, per i motivi delle righe 5 e 6.
- *Riga 4.* L'acceptor ha promesso di non votare più nei round minori di 5: il round 3, per lui, è chiuso. Sono le urne chiuse della Sezione 8.4: se votasse, la risposta data al proposer del round 5 diventerebbe falsa.
- *Riga 5.* Nel round 4 non potrebbe più votare, quindi una promessa per quel round non servirebbe, e farebbe solo credere al suo proposer di poter contare su di lui.
- *Riga 6.* Lo stesso messaggio può arrivare due volte, per esempio perché il proposer non ha visto risposte entro il timeout e lo ha rimandato, come un'email rimandata quando non arriva risposta. Al round 5 l'acceptor ha già promesso, e non c'è niente di nuovo da promettere; se a perdersi era stata la sua risposta, il proposer riproverà con un round più alto.

- *Riga 7.* Il proposer invia l'ACCEPT a tutti gli acceptor, non solo al quorum che gli ha promesso, perché non sa chi ha ignorato il suo PREPARE: per lui un silenzio sembra uguale a un messaggio perso o lento, o a un acceptor guasto. Così l'ACCEPT arriva anche a chi non sta in quel quorum: a chi ha promesso troppo tardi, quando il proposer aveva già le sue $n - f$ promesse, e a chi non ha mai visto $\langle \text{PREPARE}, 7 \rangle$, perché si è perso o è stato superato dall'ACCEPT, come all'acceptor della tabella. Questo vota, perché non rompe nessuna delle sue promesse, che riguardano solo i round sotto la sbarra, e perché il valore è già garantito dall'inventario che il proposer ha fatto su una maggioranza. Inviare a tutti dà più possibilità di arrivare a una maggioranza di voti: se uno degli acceptor che hanno promesso si guasta prima di votare, un altro ne prende il posto. Chi invece aveva ignorato $\langle \text{PREPARE}, 7 \rangle$ perché aveva già promesso a un round più alto scarta anche $\langle \text{ACCEPT}, 7, z \rangle$, come nella riga 4; chi aveva ignorato soltanto un doppione, come nella riga 6, vota.

Le regole confrontano il numero del round scritto nel messaggio, e non l'ordine di arrivo, perché i canali non sono FIFO (Sezione 1.2): i messaggi possono arrivare in qualunque ordine, anche quelli dello stesso mittente, come una lettera spedita lunedì può arrivare dopo una spedita martedì. Per esempio un proposer invia $\langle \text{PREPARE}, 3 \rangle$, non riceve risposte in tempo e invia $\langle \text{PREPARE}, 5 \rangle$. Se a un acceptor arriva prima il secondo, promette al round 5, e quando arriva il primo lo ignora, perché $3 < 5$.

I learner contano round per round. La sbarra vale solo per l'acceptor, quando vota, perché serve a tenere valido l'inventario. Il learner non vota, conta: non scarta mai un LEARN perché è vecchio, e appena ha i LEARN di $n - f$ acceptor diversi con lo stesso round e lo stesso valore sa che quel valore è scelto, qualunque sia il round. I LEARN di round diversi non si contraddicono mai, perché una scelta è definitiva e ogni round successivo può solo riproporre lo stesso valore. La maggioranza, però, non si accumula tra round: un valore è scelto solo quando lo vota una maggioranza *nello stesso round*, e sommando voti di round diversi si potrebbe credere scelto un valore che non lo è.

Esempio (perché non si sommano i round). Gli acceptor sono tre.

1. Nel round 1 vota `Tesi.pdf` il solo a_1.
2. Nel round 2 il proposer riceve le promesse di a_2 e a_3, non vede il voto di a_1, propone `Bozza.pdf`, e a_2 lo vota.
3. Nel round 3 il proposer riceve le promesse di a_1 e a_3: il voto più recente che vede è quello di a_1 per `Tesi.pdf`, quindi la ripropone, e a_3 la vota.

Un learner che sommasse il voto di a_1 nel round 1 e quello di a_3 nel round 3 vedrebbe due voti su tre per `Tesi.pdf` e la crederebbe scelta. Non lo è: un round 4 che riceve le promesse di a_1 e a_2 trova come voto più recente quello del round 2, per `Bozza.pdf`, la ripropone e, con i voti di a_1 e a_2, la fa scegliere. Il fatto che due maggioranze si intersecano, e che l'inventario ritrova il voto giusto, vale per voti dello stesso round: per questo il learner aspetta $n - f$ LEARN dello stesso round.

8.6 Perché Paxos è sicuro

CS1 e CS3 sono facili da verificare. Un proposer propone il proprio valore oppure uno già votato, quindi proposto in precedenza, e gli acceptor votano solo i valori che ricevono dai proposer: vale CS1. Un learner apprende un valore solo dopo $n - f$ LEARN uguali, cioè solo se lo ha votato un quorum nello stesso round, che è proprio la condizione per essere scelto: vale CS3.

Per CS2 conviene verificare una proprietà che la implica e che guarda all'indietro, da un voto ai round precedenti:

CS se un acceptor ha votato v nel round i, nessun valore diverso da v è stato scelto, né lo sarà mai, in un round precedente.

Da CS segue CS2. Due valori diversi non possono essere scelti nello stesso round, perché in un round si propone un solo valore. E se v fosse scelto in un round e v' in un round successivo, qualche acceptor avrebbe votato v' nel round successivo, e per CS nessun valore diverso da v' potrebbe essere scelto prima: quindi $v = v'$.

Per verificare CS si guarda il proposer che ha proposto v nel round i. Prima di proporre ha raccolto le promesse di un quorum, che chiamiamo Q; sia j il round più alto tra gli ultimi voti riportati, cioè il *vrnd* più alto nelle promesse. Ogni promessa dice due cose dell'acceptor che la invia: ha votato l'ultima volta nel round *vrnd*, e non voterà mai nei round compresi tra *vrnd* e i, estremi esclusi. La Figura 8.4 mostra un caso con $n = 7$ acceptor.

		j					i
		4	5	6	7	8	9
Q	a_1	x	–	–	–	–	
	a_2	x	–	–	–	–	
	a_3	–	–	–	–	–	
	a_4	–	–	–	–	–	
fuori da Q	a_5	?	?	?	?	?	
	a_6	?	?	?	?	?	
	a_7	?	?	?	?	?	

Figura 8.4. Le promesse raccolte dal proposer del round $i = 9$, con $n = 7$ acceptor (quindi un quorum ne ha 4). Le righe sono gli acceptor, le colonne i round. a_1 e a_2 hanno votato x nel round 4, a_3 e a_4 l'ultima volta prima del round 4, e nessuno dei quattro voterà più nei round fino all'8 (i trattini). Degli acceptor fuori da Q non si sa niente (i punti interrogativi). Il round più alto con un voto è $j = 4$.

Nella figura il proposer del round 9 propone x, il valore votato nel round $j = 4$. Che nessun valore diverso da x possa essere scelto prima del round 9 si vede in tre passi.

1. *Nei round da* 5 *a* 8 *non si sceglie niente.* Gli acceptor di Q non hanno votato in quei round e hanno promesso di non votarci, e i 3 rimasti, a_5, a_6 e a_7, non bastano a formare un quorum di 4.

2. *Nel round* 4 *si può scegliere solo* x. In quel round si è proposto un solo valore, e a_1 ha votato x, quindi quel valore è x.

3. *Prima del round* 4 *non si può scegliere un valore diverso da* x. a_1 ha votato x nel round 4: è la situazione di partenza, con il round 4 al posto del 9. Lo stesso ragionamento, rifatto per il proposer del round 4 e le promesse che ha raccolto, mostra che prima del round 4 non si può scegliere un valore diverso da x. Quel ragionamento si appoggia a sua volta a un round ancora precedente, e così via all'indietro fino al round 0, prima del quale non c'è niente da controllare.

In generale, nei round tra j e i non si sceglie niente, nel round j solo il valore votato, e prima di j vale lo stesso ragionamento fatto per il round j. Se nessuno in Q ha votato ($j = -1$), il primo passo copre da solo tutti i round prima di i, e il proposer può proporre il valore che vuole. Si vede anche perché va adottato proprio il voto del round più alto: se il proposer adottasse un voto più vecchio, per esempio un y votato da a_3 nel round 2, potrebbe contraddire x, che nel round 4 può essere stato scelto con i voti di a_1, a_2 e di due acceptor fuori da Q.

Il ragionamento non usa mai il numero dei guasti: Paxos è sicuro anche se si guastano più di f acceptor. In quel caso non si forma nessun quorum e non si sceglie niente, ma non si sceglie mai un valore sbagliato.

8.7 Liveness

Anche senza guasti, Paxos può non scegliere mai niente, se più proposer avviano round nello stesso periodo (Figura 8.5).

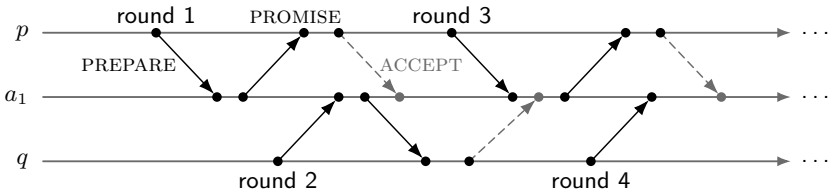


Figura 8.5. Due proposer in competizione: p usa i round dispari, q i pari. Gli acceptor sono tre, ma a_3 è guasto e a_2 riceve gli stessi messaggi di a_1, quindi è disegnato solo a_1. Gli ACCEPT, grigi e tratteggiati, arrivano sempre dopo il PREPARE di un round più alto, e vengono scartati.

Esempio (due proposer in competizione). Nella cartella condivisa, p e q sono i server di Anna e di Bruno. p avvia il round 1 e ottiene le promesse. Prima che il suo $\langle \text{ACCEPT}, 1, x \rangle$ arrivi, gli acceptor ricevono $\langle \text{PREPARE}, 2 \rangle$ da q e promettono di non votare più in round minori di 2, quindi scartano l'ACCEPT di p. p avvia allora il round 3, le cui promesse fanno scartare $\langle \text{ACCEPT}, 2, y \rangle$ di q; q avvia il round 4, e così via, senza fine: nessuna delle due rinomine ha mai effetto.

Succede come a due persone che, in un corridoio, si spostano entrambe dalla stessa parte per lasciarsi passare, poi entrambe dall'altra, e così via: nessuna sbaglia, ma nessuna passa.

Il problema non si può eliminare del tutto. Fischer, Lynch e Paterson hanno dimostrato (il **risultato FLP**) che in un sistema asincrono nessun algoritmo deterministico può garantire sia la safety sia la liveness del consenso, se anche un solo

processo può guastarsi; intuitivamente, perché un processo lento non si distingue da uno guasto (Sezione 1.2). Paxos garantisce sempre la safety, e la liveness solo quando le cose vanno abbastanza bene.

Coordinatore. Per andare avanti si elegge uno dei proposer, il **coordinatore**, e solo lui avvia nuovi round: come nel corridoio, basta che uno si fermi e lasci passare l'altro. Nella cartella condivisa il coordinatore è il server che fa da leader per la cartella: le richieste di Anna e di Bruno passano entrambe da lui, che le propone una alla volta. Senza concorrenza, e se si guastano al più f acceptor, il suo round si conclude con un valore scelto; se il numero del suo round è troppo basso, perché gli acceptor hanno già promesso a round più alti, ne prova uno più alto.

Il risultato FLP, però, vale anche per l'elezione: in un sistema asincrono con guasti nemmeno l'elezione di un leader si può garantire. Paxos accetta che l'elezione sbagli e che per un po' ci siano due o più coordinatori: si perde solo la liveness, mai la safety, come con i timeout sbagliati del 2PC (Sezione 7.4). Se il leader della cartella è solo lento, e gli altri server ne eleggono un altro allo scadere di un timeout, per un po' i leader sono due: le modifiche rallentano, ma nessun utente vede due nomi diversi.

Il failure detector Ω. L'elezione si descrive con un ***failure detector*** (rilevatore di guasti) chiamato Ω. In ogni processo, un modulo Ω indica il processo che in quel momento è considerato il leader. Ω può sbagliare, e processi diversi possono indicare leader diversi, ma ha una garanzia: *da un certo istante in poi, tutti i processi corretti indicano sempre lo stesso processo corretto.* Un proposer si considera coordinatore quando il suo Ω indica sé stesso. Prima o poi, quindi, i proposer concordano su un solo coordinatore e, se si guastano al più f acceptor, Paxos sceglie un valore.

Capitolo 9

Multi-Paxos ed elezione del leader

Paxos sceglie un valore. Alla cartella condivisa del capitolo 8, però, una decisione non basta: gliene serve una per ogni operazione, per tutto il tempo in cui la cartella esiste. Questo capitolo mostra come eseguire Paxos molte volte di seguito senza pagarne ogni volta il costo intero (**Multi-Paxos**), come si elegge il leader su cui Multi-Paxos si appoggia, e come un leader nuovo riprende il lavoro lasciato a metà da quello guasto.

9.1 Una sequenza di decisioni

Nella cartella condivisa ogni modifica, come creare, rinominare o cancellare un file, è un'operazione che tutte le copie devono applicare, nello stesso ordine. Ogni server applica le operazioni alla sua copia, una dopo l'altra, e le operazioni sono deterministiche: la stessa operazione, applicata alla stessa cartella, dà sempre lo stesso risultato. Due server che partono dalla stessa cartella e applicano le stesse operazioni nello stesso ordine hanno quindi sempre la stessa cartella. È una **macchina a stati replicata** (*replicated state machine*): ogni server è una copia della stessa macchina, e basta che i server si accordino sulla sequenza delle operazioni.

La sequenza si decide una posizione alla volta. Ogni posizione è decisa da un'esecuzione separata di Paxos, un'**istanza** (*instance*): l'istanza 1 sceglie la prima operazione, l'istanza 2 la seconda, e così via. Istanza e round non vanno confusi: l'istanza dice *quale posizione* si decide, il round *quale tentativo*, dentro un'istanza, di deciderla. Ogni istanza ha i suoi round e, in ogni acceptor, le sue variabili: il round più alto a cui l'acceptor ha promesso, *rnd*, e il round e il valore del suo ultimo voto, *vrnd* e *vval* (Sezione 8.3). Un server applica un'operazione solo dopo aver appreso tutte quelle che la precedono: se conosce la 7 e la 9 ma non la 8, applica la 7 e aspetta, come chi segue una ricetta e non può fare il passo 9 senza sapere il passo 8.

Chi propone. Supponiamo che tre richieste, a, b e c, arrivino nello stesso momento a tre server diversi. Come finiscono nella sequenza dipende da chi propone.

- *Senza coordinatore c'è una gara.* Ognuno dei tre server avvia i suoi round nell'istanza 8, e i round si interrompono a vicenda, come nel duello della

Sezione 8.7. Quando un round arriva in fondo si sceglie un valore solo, per esempio b, e quale vinca dipende da come vanno i messaggi. a e c hanno perso, e i loro server le ripropongono nell'istanza 9, dove la gara ricomincia, anche con le richieste arrivate nel frattempo.

- *Con il coordinatore della Sezione 8.7 la gara non c'è.* I tre server gli inoltrano le richieste, e lui le mette nelle istanze nell'ordine in cui gli arrivano, una per istanza: per esempio b nella 8, a nella 9 e c nella 10. In ogni istanza c'è una sola proposta, e i voti non scelgono tra a, b e c: rendono definitiva la scelta del coordinatore.

Multi-Paxos usa il secondo modo, con lo stesso coordinatore, il **leader**, per tutte le istanze. Il leader quindi non nasce con Multi-Paxos: c'era già in Paxos, per la liveness. Multi-Paxos sfrutta il fatto che resti sempre lo stesso per risparmiare messaggi.

Prepare e Promise una volta sola. Con un round completo per ogni operazione, ogni modifica richiederebbe quattro messaggi, uno dopo l'altro: PREPARE, PROMISE, ACCEPT e LEARN. Le istanze restano una per operazione, ma i primi due passi sarebbero uguali in tutte. Prepare e Promise servono a fare l'inventario (Sezione 8.4), e nelle istanze future:

- nessuno ha ancora votato, quindi l'inventario dà ovunque la stessa risposta, «nessun voto», e il leader è libero di proporre quello che vuole;
- la promessa, «non voto più nei round minori di i», dove i è il numero del round, non dipende dall'istanza;
- il valore da proporre non serve: entra in gioco solo al passo Accept, quindi l'inventario si può fare prima di sapere quali richieste arriveranno.

Mille inventari identici si fanno con un messaggio solo. Il leader invia un solo $\langle \text{PREPARE}, i \rangle$, che vale per tutte le istanze di cui non conosce ancora la decisione. Ogni acceptor risponde con una sola PROMISE, che riporta il suo ultimo voto solo per le poche istanze ancora in corso. Poi il leader usa lo stesso round i in tutte le istanze, e la promessa resta valida finché non cambia il leader. È come l'appello fatto una volta sola all'apertura di una seduta: vale per tutti i punti dell'ordine del giorno, ma ogni punto ha la sua votazione.

Da quel momento ogni operazione richiede solo i passi Accept e Learn. Prepare e Promise si rifanno solo quando cambia il leader, cosa rara, quindi il loro costo si divide su moltissime istanze.

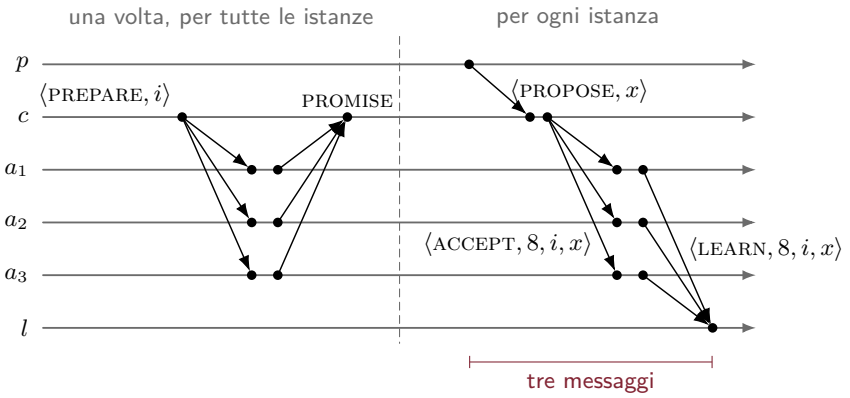


Figura 9.1. Multi-Paxos a regime. A sinistra della linea tratteggiata il leader c raccoglie le promesse, una volta per tutte le istanze; a destra c'è un'istanza, la 8: il proposer p inoltra il valore x al leader, che sceglie la posizione e lo propone agli acceptor, e gli acceptor informano i learner. Dalla proposta all'apprendimento passano tre messaggi. Dei learner è disegnato solo l.

Esempio (un'operazione a regime). La cartella ha cinque copie, sui server $S_1, \ldots, S_5$: gli acceptor sono $n = 5$, e un quorum, cioè una maggioranza, ne ha 3. Il leader è S_1, che ha già le promesse per il round i di tutte le istanze dalla 8 in poi. Anna chiede di rinominare `Relazione.pdf` in `Tesi.pdf` (il valore x), e la sua richiesta arriva a S_3, il server più vicino a lei: nella Figura 9.1, S_3 è il proposer p e S_1 il leader c.

1. *S_3 inoltra la richiesta al leader*, con $\langle \text{PROPOSE}, x \rangle$. Non avvia un round suo, per non riaprire la gara tra proposer. Nel messaggio non c'è un numero di istanza: la posizione la sceglie il leader.

2. *Il leader assegna a x la prima posizione libera*, la 8, e invia $\langle \text{ACCEPT}, 8, i, x \rangle$ a tutti gli acceptor. Il primo campo è il numero dell'istanza: ogni messaggio che riguarda una sola istanza lo porta, nell'ordine istanza, round, valore, così ogni server sa a quale posizione si riferisce. Non serve un PREPARE: le promesse per il round i coprono già l'istanza 8, e non vi riportano voti, quindi il leader è libero di proporre x.

3. *Gli acceptor votano x nell'istanza* 8 e inviano $\langle \text{LEARN}, 8, i, x \rangle$ a tutti i learner. Un learner che riceve tre LEARN dell'istanza 8 apprende che l'operazione 8 è x, e la applica dopo la 7. L'ordine, quindi, non resta nella testa del leader: sta nei voti degli acceptor, istanza per istanza, e i LEARN lo fanno conoscere a tutti. Dalla proposta di S_3 all'apprendimento passano tre messaggi: con un round completo sarebbero almeno quattro.

4. *Intanto il leader va avanti.* Arriva la richiesta di Bruno, la rinomina in `Bozza.pdf` (y), e il leader invia subito $\langle \text{ACCEPT}, 9, i, y \rangle$, senza aspettare che la 8 sia decisa. Le istanze si decidono in parallelo: ognuna è un consenso separato, con i suoi voti, e aspettare renderebbe solo tutto più lento.

5. *Le operazioni si applicano in fila.* Se i LEARN della 8 tardano, un server può apprendere prima che la 9 è y. La tiene da parte, e la applica subito dopo la 8, appena la apprende.

Le istanze restano quindi separate, e in comune hanno solo il round e la promessa. Lo si vede dalle variabili di un acceptor alla fine dell'esempio (Tabella 9.1): il PREPARE unico ha riempito la colonna *rnd* di tutte le righe insieme, mentre ogni ACCEPT scrive solo nella riga dell'istanza indicata nel suo primo campo. Le decisioni sono due, la 8 sceglie x e la 9 sceglie y, ognuna con la sua maggioranza di voti.

Tabella 9.1. Le variabili di un acceptor alla fine dell'esempio, istanza per istanza. Il valore -1 vuol dire «nessuno».

Istanza	*rnd*	*vrnd*	*vval*
8	i	i	x
9	i	i	y
$10, 11, \ldots$	i	-1	-1

Tutto questo regge finché il leader funziona. Se S_1 si guasta, gli altri server devono accorgersene, accordarsi su un leader nuovo, e il leader nuovo deve riprendere le istanze rimaste a metà.

9.2 Eleggere il leader in un sistema asincrono

Multi-Paxos si regge su un leader, e anche la liveness di Paxos ne ha bisogno (Sezione 8.7). L'idea per eleggerlo è semplice: ogni processo tiene d'occhio quali processi sono vivi, e tutti indicano come leader lo stesso, quello con l'identificatore più piccolo.

Definizione (elezione del leader con gli heartbeat).

1. Ogni processo ha un identificatore diverso, e gli identificatori sono ordinati: quello di S_1 è 1, quello di S_2 è 2, e così via.
2. A intervalli regolari ogni processo invia a tutti un HEARTBEAT (battito cardiaco), che vuol dire «sono vivo».
3. Se da un processo non arriva nessun HEARTBEAT per più di un certo tempo, il timeout, quel processo è considerato guasto.
4. Il leader è il processo con l'identificatore più piccolo tra quelli che non risultano guasti, compreso sé stesso.

Se il leader S_1 si guasta, smette di inviare HEARTBEAT. Allo scadere del timeout gli altri lo considerano guasto, e il nuovo leader è S_2, il più piccolo tra quelli rimasti. Non tutti se ne accorgono nello stesso momento, perché l'ultimo HEARTBEAT di S_1 può essere arrivato ad alcuni e non ad altri: per un po' qualcuno indica ancora S_1, ma non fa danni, perché S_1 è guasto e l'unico leader attivo è S_2.

Il punto debole è il timeout. In un sistema asincrono non c'è un limite noto ai ritardi (Sezione 1.2), quindi il timeout è solo una stima, e un HEARTBEAT può arrivare dopo la sua scadenza. Un processo soltanto lento sembra allora guasto: è il motivo per cui, per il risultato FLP, l'elezione non si può garantire (Sezione 8.7). In un sistema sincrono, con un ritardo massimo noto, basterebbe un timeout calcolato su quel ritardo per non sbagliare mai.

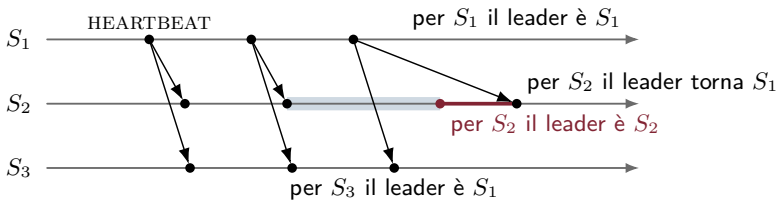


Figura 9.2. Due leader. S_1 è vivo, ma un suo HEARTBEAT arriva a S_2 in ritardo, dopo la scadenza del timeout di S_2 (la fascia azzurra, che parte dall'ultimo HEARTBEAT ricevuto). Nel tratto rosso S_2 si considera leader, mentre per S_1 e S_3 il leader è S_1. Sono disegnati solo alcuni heartbeat di S_1.

Esempio (due leader). I server sono tre, e il leader è S_1. Nella Figura 9.2 S_1 invia un HEARTBEAT a S_2 e a S_3: a S_3 arriva in tempo, a S_2 in ritardo, per un rallentamento della rete.

1. *Il timeout di S_2 scade prima che l'*HEARTBEAT *arrivi.* S_2 crede S_1 guasto e si considera leader. Per S_1 e per S_3, invece, il leader è ancora S_1: i leader sono due.

2. *Prevale il round più alto.* S_2 avvia un round suo, più alto, e con le promesse di una maggioranza chiude quello di S_1, i cui ACCEPT vengono scartati. Dove propongono entrambi torna la gara della Sezione 9.1: le operazioni rallentano, ma Paxos resta sicuro.

3. *L'*HEARTBEAT *arriva.* S_2 vede che S_1 è vivo e torna a indicarlo come leader. Smette di proporre e allunga il suo timeout, così la prossima volta sbaglia meno. S_1 trova le promesse fatte al round di S_2, e riparte con un round più alto, come nella Sezione 8.7.

Il secondo leader nasce perché a sbagliare è proprio S_2, il primo dopo S_1. Se fosse S_3 a credere guasto S_1, indicherebbe come leader S_2, che però non si considera leader: S_3 perderebbe solo un po' di tempo.

Se i ritardi a un certo punto si stabilizzano, allungando il timeout a ogni errore prima o poi non si sbaglia più, e da lì in poi tutti i processi corretti indicano lo stesso processo corretto: è proprio la garanzia che si chiedeva al failure detector Ω.

9.3 Il nuovo leader

Quando S_2 diventa leader, alcune istanze possono essere rimaste a metà, perché S_1 ne aveva avviate diverse senza aspettarne l'esito: S_2 le riprende tutte con un solo inventario.

Esempio (le istanze lasciate a metà). S_2 ha appreso le operazioni delle istanze da 1 a 7. Prima di guastarsi, S_1 aveva inviato gli ACCEPT delle istanze 8 (x, la rinomina di Anna), 9 (y, quella di Bruno) e 10 (una terza operazione, z), ma quelli dell'istanza 9 si sono persi tutti.

1. *Un solo* PREPARE *per tutte le istanze aperte.* S_2 sceglie un suo round i', più alto di i, e invia $\langle \text{PREPARE}, i' \rangle$ per tutte le istanze dalla 8 in poi.
2. *Le promesse riportano i voti istanza per istanza.* Fra le prime tre promesse, per esempio, una riporta un voto per x nell'istanza 8 e una un voto per z nella 10; nessuna riporta voti nella 9.
3. *In ogni istanza vale la regola di Paxos.* Nelle istanze 8 e 10 S_2 ripropone il valore votato, con $\langle \text{ACCEPT}, 8, i', x \rangle$ e $\langle \text{ACCEPT}, 10, i', z \rangle$, perché può essere già stato scelto. Nell'istanza 9 nessuna promessa riporta un voto, quindi lì non è stato scelto niente: una scelta lascerebbe almeno un voto in ogni maggioranza (Sezione 8.2).
4. *I buchi si riempiono con un no-op.* Nell'istanza 9 S_2 propone un ***no-op***, un'operazione che non cambia niente, con $\langle \text{ACCEPT}, 9, i', \text{no-op} \rangle$. Potrebbe aspettare una richiesta nuova, ma intanto nessuno potrebbe applicare la 10, che viene dopo la 9; con il no-op, invece, si applicano subito la 8, la 9 e la 10.
5. *Dalla* 11 *in poi si va a regime.* Le promesse per il round i' coprono già le istanze successive, quindi S_2 propone le richieste nuove con il solo ACCEPT.

La rinomina di Bruno non va persa: il server che l'aveva ricevuta non la vede applicata, allo scadere di un timeout la inoltra di nuovo, questa volta a S_2, che le assegna una posizione libera, per esempio la 11. Qui è giusto, perché nella 9 è finito un no-op, ma il server farebbe lo stesso se la rinomina fosse già stata scelta e i suoi LEARN fossero solo in ritardo: allora finirebbe in due posizioni. Paxos impedisce due decisioni nella stessa istanza, non la stessa operazione in due istanze. Per questo di solito ogni richiesta ha un identificatore, e i doppioni si saltano.

Capitolo 10

Fast Paxos

In Multi-Paxos un'operazione fa tre salti prima di essere appresa: dal proposer al leader, dal leader agli acceptor, dagli acceptor ai learner. **Fast Paxos** toglie il primo: il proposer invia il suo valore direttamente agli acceptor, e i messaggi diventano due. Il prezzo è che nello stesso round gli acceptor possono votare valori diversi, cosa che in Paxos non succedeva mai.

10.1 Saltare il coordinatore

In Multi-Paxos ogni valore passa dal leader, che fa da coordinatore di tutte le istanze, perché è lui a scegliere che cosa proporre. Quando però le promesse non riportano voti, il coordinatore non ha niente da ritrovare e può proporre qualunque valore (Sezione 8.3): in quel caso può lasciare la scelta agli acceptor. Al posto di un valore invia $\langle \text{ACCEPT}, i, \bot \rangle$, dove i è il numero del round e $\bot$ vuol dire «nessun valore». È un *accept any*, «accettate qualunque cosa»: nel round i gli acceptor possono votare qualunque valore arrivi loro da un proposer. Un round così è un **round veloce** (*fast round*); quelli in cui il coordinatore propone un valore, come in Paxos, sono **round classici** (*classic round*).

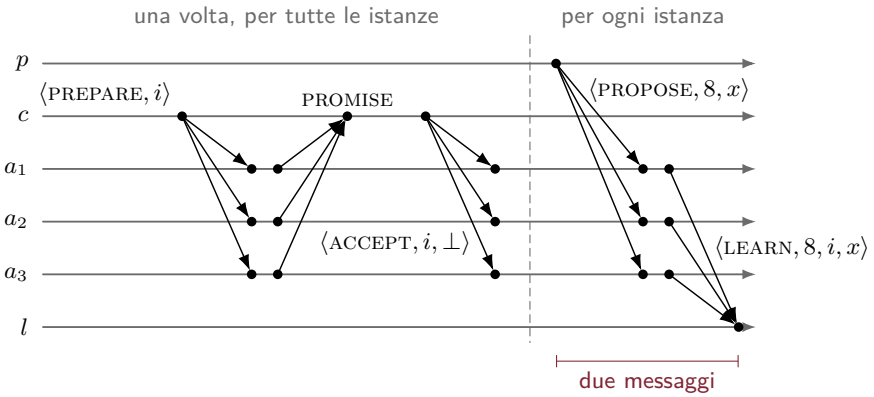


Figura 10.1. Fast Paxos a regime, da confrontare con la Figura 9.1. Dalla proposta all'apprendimento passano due messaggi.

Come PREPARE e PROMISE, anche $\langle \text{ACCEPT}, i, \bot \rangle$ non contiene un valore, quindi il coordinatore lo invia una volta sola per tutte le istanze (Sezione 9.1). Da quel momento ogni operazione richiede due messaggi soli (Figura 10.1): il proposer invia $\langle \text{PROPOSE}, 8, x \rangle$ direttamente a tutti gli acceptor, che votano x nell'istanza 8 e inviano $\langle \text{LEARN}, 8, i, x \rangle$ ai learner. Ora il numero dell'istanza lo sceglie il proposer, e lo scrive nel PROPOSE: in Multi-Paxos lo aggiungeva il leader. Il coordinatore quindi non sparisce: apre i round veloci con l'*accept any*, e resta lui ad avviare un round nuovo quando qualcosa va storto. Esce solo dal percorso dei valori, come un arbitro che dà il via e poi interviene solo se c'è una contestazione.

Il risparmio ha un costo, che si vede quando due proposer inviano un valore nello stesso momento.

Esempio (due rinomine nello stesso istante). La cartella ha ora sette copie, sui server $S_1, \ldots, S_7$, e il coordinatore ha già inviato $\langle \text{ACCEPT}, 1, \bot \rangle$ per le istanze ancora libere. Anna e Bruno rinominano `Relazione.pdf` nello stesso momento.

1. *I due proposer inviano direttamente agli acceptor.* Il server di Anna invia $\langle \text{PROPOSE}, 8, x \rangle$ a tutti, quello di Bruno $\langle \text{PROPOSE}, 8, y \rangle$: entrambi scelgono la prima istanza libera, la 8. Nessuno dei due passa dal coordinatore, quindi nessuno mette in fila le due richieste.
2. *Ogni acceptor vota il primo valore che riceve.* I messaggi arrivano a server diversi in ordini diversi: S_1, S_2, S_3 e S_4 ricevono prima x e votano x; S_5, S_6 e S_7 ricevono prima y e votano y.
3. *Il secondo valore viene ignorato.* Quando a S_1 arriva y, S_1 ha già votato nell'istanza 8, nel round 1, e in ogni istanza un acceptor vota una volta sola per round. Se votasse anche y, potrebbe contribuire sia a una maggioranza per x sia a una per y, e nella stessa istanza si sceglierebbero due valori. Un valore per l'istanza 9, invece, lo voterebbe normalmente: la regola vale dentro ogni istanza. In Paxos non serviva dirlo, perché in un round arrivava un solo valore, quello del suo proposer.

La gara che in Multi-Paxos il leader aveva tolto (Sezione 9.1) qui torna, di proposito, per risparmiare un messaggio: nessuno mette più in fila le richieste, e due proposte arrivate insieme possono finire nella stessa istanza. Se Anna avesse usato l'istanza 8 e Bruno la 9, non ci sarebbe stato nessun conflitto. La Tabella 10.1 mette a confronto i modi di proporre visti finora.

Tabella 10.1. I modi di proporre visti finora.

	Chi propone	C'è competizione?	Prepare e Promise
Paxos senza coordinatore	chiunque	sì, con il rischio del duello	in ogni round
Paxos con coordinatore	solo il coordinatore	no, finché è uno solo	in ogni istanza
Multi-Paxos	solo il leader	no, finché è uno solo	una volta per tutte le istanze
Fast Paxos	ogni proposer, direttamente	sì: le collisioni	una volta, più l'*accept any*

In un round veloce, quindi, i voti si possono dividere tra valori diversi. È stato scelto qualcosa? Dipende da quanti voti servono, e qui la maggioranza della Sezione 8.2 non basta più.

10.2 Quorum più grandi

Proviamo con i quorum di Paxos. Gli acceptor sono $n = 7$: Paxos tollera $f = \lfloor (n-1)/2 \rfloor = 3$ guasti ($\lfloor \cdot \rfloor$ arrotonda per difetto), e un quorum, una maggioranza, ha $n - f = 4$ acceptor.

1. *x è scelto.* S_1, S_2, S_3 e S_4 l'hanno votato nel round 1, e sono un quorum. Un learner che riceve i loro quattro LEARN apprende x, e mostra `Tesi.pdf`.
2. *Il coordinatore non lo sa.* I LEARN gli arrivano in ritardo, e allo scadere di un timeout avvia il round classico 2: chiede a tutti che cosa hanno votato, e aspetta le prime quattro promesse. Arrivano quelle di S_4, S_5, S_6 e S_7: un voto per x e tre per y, tutti nel round 1.
3. *La regola di Paxos non decide.* In Paxos si proponeva il valore votato nel round più alto, e in quel round il valore era uno solo, quello del coordinatore. Qui il round più alto è il 1, e i valori sono due.

Il coordinatore deve capire quale valore può essere stato scelto, e ha solo le quattro promesse che ha aspettato. Quelle degli altri acceptor non sono arrivate: questi possono essere guasti, lenti o avere i messaggi in ritardo, e il coordinatore non li aspetta, perché non sa distinguere questi casi (Sezione 8.3). Di loro non sa come hanno votato, quindi deve considerare che possano aver votato qualunque valore. Un valore può essere stato scelto se i voti che vede, più quelli degli acceptor di cui mancano le promesse, possono arrivare a un quorum. Qui mancano le promesse di tre acceptor, S_1, S_2 e S_3, e i conti sono quelli della Tabella 10.2.

Tabella 10.2. I valori che possono essere stati scelti, con quorum di 4 acceptor su 7: arrivano le promesse di S_4, S_5, S_6 e S_7, non quelle di S_1, S_2 e S_3.

Valore	Voti visti	Ne servono	Mancano	Possono venire da S_1, S_2, S_3?
x	1	4	3	sì, se hanno votato tutti e tre x
y	3	4	1	sì, se almeno uno ha votato y

Tutti e due possono essere stati scelti, e il coordinatore non sa quale, perché non vede S_1, S_2 e S_3. Qualunque valore proponga, rischia di contraddire una scelta già fatta: in questa esecuzione è stato scelto x, ma con le stesse promesse, se S_1, S_2 e S_3 avessero votato y, sarebbe stato scelto y. Nella Tabella 8.1 c'era sempre una mossa sicura; qui no. E non basta proporre il valore più votato tra le promesse: sarebbe y, con tre voti contro uno, mentre qui è stato scelto proprio x. Il conteggio delle promesse non dice chi ha vinto.

Il motivo sta nei numeri. Gli acceptor di cui mancano le promesse sono 3, cioè un quorum meno uno: basta un solo voto visibile per completare un quorum, quindi ogni valore che compare tra le promesse è un candidato. In Paxos non era un problema, perché nel round più alto il candidato era sempre uno solo.

Più voti per scegliere. La soluzione è chiedere più voti, così che fuori da un quorum restino meno acceptor. Fast Paxos tollera

$$f' = \left\lfloor \frac{n-1}{3} \right\rfloor$$

guasti, cioè il più grande numero per cui gli acceptor sono più del triplo dei guasti, $n > 3f'$, e un quorum ha $n - f'$ acceptor. Con $n = 7$ si ha $f' = 2$, e un quorum ne ha 5. Vale per tutti i round, veloci e classici, sia per i voti sia per le promesse. Fuori da un quorum restano ora solo 2 acceptor, e valgono due fatti.

- *Un valore scelto lascia almeno* 3 *voti fra le promesse.* È stato votato da 5 acceptor, e al più 2 stanno fuori dal quorum di cui il coordinatore ha le promesse. In generale, dei suoi $n - f'$ voti al più f' stanno fuori, quindi almeno $n - 2f'$ stanno fra le promesse. Al contrario, un valore con meno di 3 voti fra le promesse non è stato scelto.
- *Due valori non possono avere* 3 *voti ciascuno* fra 5 promesse, perché $3 + 3 = 6 > 5$. In generale servirebbero $2(n - 2f')$ promesse, che sono più di $n - f'$ proprio perché $n > 3f'$.

Le promesse mostrano quindi al più un candidato, ed è quello che il coordinatore deve proporre. È come lo spoglio parziale di un'elezione con 7 schede. Se per vincere ne servono 4 e ne hai contate 4, le 3 che mancano possono ancora far vincere chiunque abbia almeno un voto. Se ne servono 5 e ne hai contate 5, le 2 che mancano bastano solo a chi ha già almeno 3 voti, e un candidato così ce n'è al più uno.

Definizione (il valore da proporre in Fast Paxos). Quando ha le promesse di $n - f'$ acceptor per il round i, il coordinatore guarda j, il round più alto tra gli ultimi voti riportati, cioè il *vrnd* (*voted round*) più alto nelle promesse.

1. Se nessuno ha mai votato ($j = -1$), avvia un round veloce: invia $\langle \text{ACCEPT}, i, \perp \rangle$.
2. Se un valore ha almeno $n - 2f'$ voti nel round j, fra le promesse, propone quel valore: è l'unico che può essere stato scelto.
3. Altrimenti nel round j non è stato scelto niente, e propone uno qualunque dei valori votati nel round j.

Torniamo alla collisione, ora con quorum di 5: è qui che il coordinatore torna in gioco.

1. *Il round veloce non sceglie niente.* x ha 4 voti e y ne ha 3: nessuno dei due arriva a 5. È una **collisione**: in un round veloce i voti si sono divisi, e nessun valore ha un quorum.
2. *Il coordinatore avvia un round classico.* Se ne accorge allo scadere di un timeout, oppure dai LEARN, appena nessun valore può più arrivare a 5 voti. Avvia il round 2 solo per l'istanza 8: le altre istanze restano nel round veloce 1, perché ogni istanza ha i suoi round.
3. *Aspetta le prime cinque promesse.* Invia $\langle \text{PREPARE}, 8, 2 \rangle$ a tutti, e comincia appena ha cinque promesse, per esempio quelle di $S_3, \ldots, S_7$. Quelle di S_1 e S_2 non sono arrivate: il coordinatore non sa che S_1 e S_2 hanno votato x, e deve considerare che possano aver votato qualunque valore.
4. *Fa i conti dei candidati*, gli stessi della Tabella 10.2, ma ora mancano solo due promesse (Tabella 10.3). x ha 2 voti visti, e anche con quelli di S_1 e S_2 non arriverebbe a 5. y ne ha 3, e ci arriverebbe se S_1 e S_2 avessero votato y. Il candidato è uno solo, y, e il coordinatore lo propone con $\langle \text{ACCEPT}, 8, 2, y \rangle$ (caso 2).

Tabella 10.3. I valori che possono essere stati scelti, con quorum di 5 acceptor su 7: arrivano le promesse di $S_3, \ldots, S_7$, non quelle di S_1 e S_2.

Valore	Voti visti	Ne servono	Mancano	Possono venire da S_1 e S_2?
x	2	5	3	no: fuori ce ne sono solo 2
y	3	5	2	sì, se hanno votato tutti e due y

Il coordinatore non sa se y è stato scelto, e non gli serve: x di sicuro non lo è, e proporre y va bene in entrambi i casi, come nella Tabella 8.1. Se invece arrivano le promesse di $S_1, \ldots, S_5$, contano 4 voti per x e uno per y, e il candidato è x. In entrambi i casi il round 2 sceglie un solo valore, e la rinomina che perde passa a un'istanza successiva.

Il caso 3 si presenta quando nessun valore ha abbastanza voti tra le promesse. Supponiamo che nel round veloce S_1 e S_2 abbiano votato x, S_3 e S_4 abbiano votato y, e che a S_5, S_6 e S_7 i valori non siano ancora arrivati quando il coordinatore avvia il round classico 2.

1. *Le prime cinque promesse sono di* $S_1, \ldots, S_5$. Quattro riportano un voto, due per x e due per y. Quella di S_5 non ne riporta, perché promettere e votare sono cose separate: S_5 promette al round 2 e riporta il suo ultimo voto, che non c'è, come le promesse con -1 della Sezione 8.3. Da quel momento non vota più nel round 1, anche se x o y gli arrivano dopo.
2. *Le promesse di* S_6 *e* S_7 *non sono arrivate.* In realtà S_6 e S_7 non hanno votato, ma il coordinatore non lo sa, e deve considerare che possano aver votato qualunque valore.
3. *Nessun valore è un candidato* (Tabella 10.4). x e y hanno 2 voti visti ciascuno, e anche con quelli di S_6 e S_7 arriverebbero al più a 4, non a 5. Nel round 1 non è stato scelto niente, e non lo sarà più.
4. *Il coordinatore propone* x *oppure* y (caso 3). Non c'è nessuna scelta da proteggere, quindi vanno bene tutti e due.

Tabella 10.4. I valori che possono essere stati scelti nel caso 3, con quorum di 5 acceptor su 7: arrivano le promesse di $S_1, \ldots, S_5$ (quella di S_5 senza voti), non quelle di S_6 e S_7.

Valore	Voti visti	Ne servono	Mancano	Possono venire da S_6 e S_7?
x	2	5	3	no: fuori ce ne sono solo 2
y	2	5	3	no: fuori ce ne sono solo 2

Nel caso 2 un valore aveva almeno 3 voti visti, ed era l'unico candidato; qui nessuno ci arriva, e il coordinatore sceglie tra i valori votati. Non può invece proporre un valore che nel round j nessuno ha votato. In questo esempio non farebbe danni, perché prima del round 1 non c'è niente, ma in generale il round j può essere un round classico che ha riproposto un valore forse scelto in un round ancora precedente, come nella Sezione 8.4. Un valore votato nel round j ha già superato l'inventario del round j, che ha controllato i round prima; uno nuovo no. È ancora il fatto che ogni round guarda indietro.

Il prezzo. Fast Paxos paga la velocità in due modi.

- *Tollera meno guasti.* Con 7 acceptor Paxos ne tollera 3 e Fast Paxos 2. Per tollerare 2 guasti, la cartella ha bisogno di 5 copie con Paxos e di 7 con Fast Paxos, perché gli acceptor devono essere più del triplo dei guasti.
- *Una collisione costa cara.* Il round veloce va sprecato, e il coordinatore deve aggiungere un round classico intero, con PREPARE, PROMISE, ACCEPT e LEARN: molti più dei tre messaggi di Multi-Paxos.

Fast Paxos conviene quindi quando due proposte per la stessa istanza arrivano di rado nello stesso momento; con molte richieste simultanee conviene far passare tutto dal leader. Per la liveness valgono le condizioni di Paxos: un solo coordinatore, che avvia i round e risolve le collisioni, e al più f' acceptor guasti.

10.3 Perché Fast Paxos è sicuro

Le proprietà da verificare sono quelle di Paxos (Sezione 8.1): si sceglie solo un valore proposto (CS1), se ne sceglie uno solo (CS2), e un learner corretto apprende solo un valore scelto (CS3). CS1 e CS3 si verificano come in Paxos: gli acceptor votano solo valori arrivati da un proposer o dal coordinatore, e un learner apprende un valore solo dopo $n - f'$ LEARN uguali dello stesso round, cioè solo se è stato scelto.

Per CS2 si verifica di nuovo la proprietà CS: se un acceptor ha votato v nel round i, nessun valore diverso da v è stato scelto, né lo sarà mai, in un round precedente. Che da CS segua CS2 si vede come nella Sezione 8.6, con una differenza: in un round veloce si votano valori diversi, quindi bisogna escludere che due di essi siano scelti nello stesso round. Non succede: due quorum di 5 acceptor su 7 ne hanno almeno $5 + 5 - 7 = 3$ in comune, e ognuno vota una volta sola per round.

Per verificare CS si guarda il coordinatore che ha proposto v nel round i: ha raccolto le promesse di un quorum, che chiamiamo Q, e j è il round più alto tra gli ultimi voti riportati. La Figura 10.2 mostra un caso con $n = 7$, come la Figura 8.4, ma con un quorum di 5 e un round j veloce, in cui sono stati votati due valori.

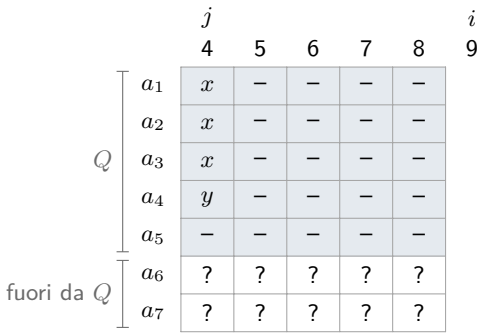


Figura 10.2. Le promesse raccolte dal coordinatore del round $i = 9$, con $n = 7$ acceptor (quindi un quorum ne ha 5). Nel round veloce $j = 4$, a_1, a_2 e a_3 hanno votato x, e a_4 ha votato y; a_5 ha votato l'ultima volta prima del round 4. Nessuno dei cinque voterà più nei round fino all'8 (i trattini). Di a_6 e a_7, fuori da Q, non si sa niente (i punti interrogativi).

Il coordinatore del round 9 trova 3 voti per x nel round $j = 4$, quanti ne chiede il caso 2 ($n - 2f' = 3$), e propone x. Che nessun valore diverso da x possa essere scelto prima del round 9 si vede negli stessi tre passi di Paxos; cambia solo il secondo.

1. *Nei round da* 5 *a* 8 *non si sceglie niente.* Gli acceptor di Q non hanno votato in quei round e hanno promesso di non votarci, e i 2 rimasti, a_6 e a_7, non bastano a formare un quorum di 5.

2. *Nel round* 4 *si può scegliere solo* x*.* Perché y fosse scelto servirebbero 5 voti per y nel round 4. Ma a_1, a_2 e a_3 nel round 4 hanno già votato x, e non votano due volte; a_5 nel round 4 non ha votato, e ha promesso di non farlo più. Restano a_4, a_6 e a_7, cioè al più 3 voti. Lo stesso vale per ogni altro valore diverso da x: è il primo dei due fatti della Sezione 10.2, con in più le promesse, che impediscono i voti futuri.

3. *Prima del round* 4 *non si può scegliere un valore diverso da* x. a_1 ha votato x nel round 4, e lo stesso ragionamento, rifatto per il round 4, va all'indietro come in Paxos.

Negli altri casi della regola il ragionamento si adatta. Nel caso 3 il secondo passo mostra che nel round j non è stato scelto niente, e il terzo vale per il valore votato nel round j che il coordinatore propone. Nel caso 1 il primo passo copre da solo tutti i round prima di i, e per questo nel round veloce va bene qualunque valore. Come in Paxos, il ragionamento non usa mai il numero dei guasti: con più di f' guasti Fast Paxos non sceglie niente, ma non sceglie mai due valori.

Capitolo 11

Raft

Multi-Paxos tiene d'accordo le copie di una macchina a stati replicata (Sezione 9.1), ma è difficile da capire: è costruito sopra Paxos, che decide un valore alla volta, e molti suoi dettagli pratici non sono mai stati fissati del tutto, tanto che i sistemi reali finiscono per allontanarsene. **Raft** risolve lo stesso problema, con lo stesso risultato e la stessa efficienza, ma è progettato prima di tutto per essere facile da capire. Per questo fa due scelte:

- *un leader forte*: tutto passa dal leader, le operazioni vanno solo dal leader agli altri server, e la loro sequenza cresce in ordine, senza buchi;
- *tre problemi separati*, che si capiscono uno alla volta: eleggere il leader, copiare le operazioni sugli altri server, e garantire che un'operazione decisa non vada mai persa.

11.1 Un leader forte

La cartella condivisa ha di nuovo cinque copie, sui server $S_1, \ldots, S_5$. Raft decide a maggioranza, quindi con cinque server tollera due guasti, come Paxos (Sezione 8.2). Il sistema è asincrono, e i messaggi possono perdersi, arrivare in ritardo, due volte o fuori ordine. I server si guastano fermandosi, e possono ripartire con ciò che hanno salvato in memoria stabile.

In ogni momento un server è in uno di tre stati:

- **leader**: riceve le richieste dei client e le copia sugli altri server; se un client si rivolge a un altro server, questo lo rimanda al leader;
- ***follower*** (seguace): è passivo, non invia richieste di sua iniziativa e risponde soltanto al leader e ai candidati;
- **candidato** (*candidate*): vuole diventare leader, e chiede i voti agli altri.

Nel funzionamento normale c'è un leader, e tutti gli altri server sono follower.

Il tempo è diviso in ***term*** (mandati), numerati con interi consecutivi: 1, 2, 3, e così via. Ogni term comincia con un'elezione, e chi la vince è il leader per il resto del term. Se l'elezione non ha un vincitore, il term finisce senza leader, e ne comincia subito un altro, con un'elezione nuova (Figura 11.1). In un term c'è al più un leader.

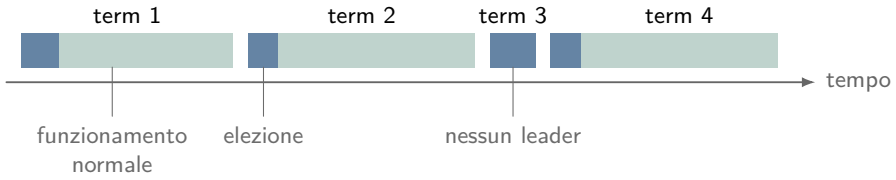


Figura 11.1. Il tempo diviso in term. Ogni term comincia con un'elezione; se l'elezione ha un vincitore, il resto del term è funzionamento normale con quel leader. Il term 3 finisce senza leader.

Come i round di Paxos (Sezione 8.4), i term sono numeri e non orari: un term più alto è più nuovo, e i server confrontano i numeri, non gli orologi, come con i clock logici del capitolo 3. Server diversi si accorgono del cambio di term in momenti diversi, e un server rimasto isolato può saltare term interi.

I server comunicano con due sole RPC (*remote procedure call*, chiamata di procedura remota), cioè richieste a cui il destinatario risponde:

- RequestVote, con cui un candidato chiede il voto;
- AppendEntries, con cui il leader copia le operazioni sugli altri server; senza operazioni, lo stesso messaggio fa da heartbeat (Sezione 9.2).

Un server invia le sue RPC a tutti in parallelo, e le rimanda se la risposta non arriva. L'obiettivo è quello della Sezione 9.1: tutte le copie devono applicare le stesse operazioni nello stesso ordine.

11.2 Eleggere il leader

Il leader può guastarsi, o restare isolato, e gli altri devono accorgersene e sceglierne uno nuovo. Come nella Sezione 9.2, il leader invia a intervalli regolari un heartbeat, e chi non lo riceve per più di un timeout lo considera guasto. Qui però non vince il server con l'identificatore più piccolo: chi si accorge per primo del guasto si candida, e diventa leader se lo vota una maggioranza.

Ogni server ricorda due cose:

- *currentTerm* (*current term*, term corrente): il term più alto che ha visto, all'inizio 0;
- *votedFor* (*voted for*, votato per): il candidato che ha votato nel term corrente, se ne ha votato uno.

Le salva in memoria stabile prima di rispondere a qualunque messaggio: un server che si guasta e riparte non deve dimenticare per chi ha votato, altrimenti potrebbe votare due volte nello stesso term.

Definizione (elezione in Raft).

1. *Il term nei messaggi.* Ogni messaggio porta il term di chi lo invia. Chi riceve un term più alto del suo *currentTerm* lo adotta e, se era candidato o leader, torna follower. Chi riceve una richiesta con un term più basso la rifiuta, e nella risposta indica il proprio term.

2. *Heartbeat.* Il leader invia a tutti, a intervalli regolari, un AppendEntries senza operazioni.
3. *Candidatura.* Un follower che per un **timeout di elezione** (*election timeout*) non riceve niente dal leader, né dà il suo voto, diventa candidato: incrementa *currentTerm*, vota per sé e invia a tutti un RequestVote con il nuovo term.
4. *Voto.* Un server vota al più un candidato per term: il primo che glielo chiede.
5. *Esito.* Il candidato resta tale finché succede una di tre cose:
   - riceve i voti di una maggioranza, compreso il suo: diventa leader, e invia subito un heartbeat a tutti;
   - riceve un AppendEntries con un term uguale o più alto del suo: c'è già un leader, e torna follower;
   - gli scade di nuovo il timeout: comincia un'altra elezione, con il term successivo.
6. *Timeout casuali.* Ogni server sceglie il suo timeout a caso in un intervallo fisso, per esempio tra 150 e 300 millisecondi, e lo estrae di nuovo a ogni elezione.

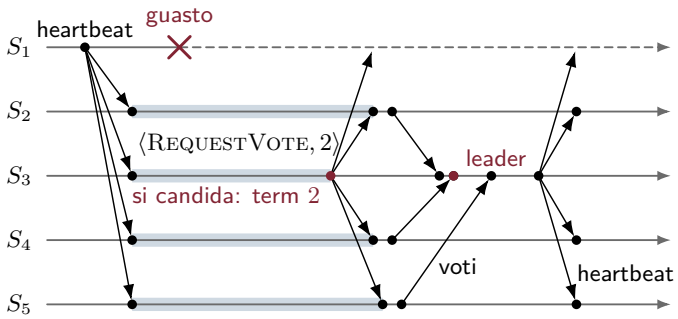


Figura 11.2. Un'elezione dopo il guasto del leader S_1. Le fasce azzurre sono i timeout di elezione, che partono dall'ultimo heartbeat ricevuto: quello di S_3 scade per primo, quelli degli altri sono ancora in corso quando arriva la richiesta di voto. S_3 diventa leader al secondo voto (i punti rossi).

Esempio (il leader si guasta). Il leader è S_1, nel term 1, e si guasta subito dopo un heartbeat (Figura 11.2).

1. *I timeout scadono in momenti diversi.* L'ultimo heartbeat fa ripartire il timeout di tutti i follower, ma ognuno l'ha estratto a caso, quindi non scadono insieme.
2. *Il timeout di S_3 scade per primo.* S_3 passa al term 2, vota per sé e invia a tutti ⟨RequestVote, 2⟩, cioè una richiesta di voto per il term 2.

3. *S_2, S_4 e S_5 votano per S_3.* Sono ancora follower, perché i loro timeout non sono scaduti. Il term 2 è più alto del loro, quindi lo adottano e, poiché nel term 2 non hanno ancora votato, votano S_3. Dare il voto fa ripartire il loro timeout.

4. *S_3 diventa leader al secondo voto.* Con il suo sono tre su cinque, una maggioranza, e non aspetta il terzo, come un proposer di Paxos non aspetta tutte le promesse (Sezione 8.3). Invia subito un heartbeat, che arriva agli altri prima che scada un altro timeout: restano follower, e S_3 è il leader del term 2.

Nel term 2 non può esserci un secondo leader. Per vincere servono i voti di una maggioranza, e due maggioranze hanno sempre un server in comune, il testimone della Sezione 8.2: se due candidati vincessero lo stesso term, il testimone avrebbe votato due volte in quel term. È la proprietà ***Election Safety*** (sicurezza dell'elezione): in ogni term c'è al più un leader. Paxos otteneva un risultato simile in un altro modo: i numeri dei round erano divisi tra i proposer, e in ogni round proponeva uno solo (Sezione 8.4). In Raft i term sono consecutivi, e chi si candida usa semplicemente il successivo del suo, quindi due candidati possono avere lo stesso term: a decidere chi vince è il voto.

Esempio (un voto diviso). Questa volta, dopo il guasto di S_1, i timeout di S_2 e di S_4 scadono quasi insieme.

1. *Due candidati nello stesso term.* S_2 e S_4 passano entrambi al term 2, votano per sé e chiedono i voti.

2. *I voti si dividono.* A S_3 arriva prima la richiesta di S_2, a S_5 quella di S_4, e ognuno vota il primo che glielo chiede. S_2 e S_4 hanno due voti ciascuno, e nessuno arriva a tre: il term 2 finisce senza leader, come il term 3 della Figura 11.1.

3. *Si riprova con timeout nuovi.* I due candidati estraggono un timeout nuovo, e quello di S_2 scade per primo. S_2 passa al term 3 e chiede di nuovo i voti. Nel term 3 nessuno ha ancora votato, quindi lo votano tutti, anche S_4, che vede un term più alto del suo e torna follower. S_2 è il leader del term 3.

Con timeout uguali i voti divisi potrebbero ripetersi all'infinito, come le due persone nel corridoio della Sezione 8.7: basta che una aspetti un po' più dell'altra, e i timeout casuali lo ottengono quasi sempre. In pratica bastano pochi millisecondi di differenza tra i timeout per rendere i voti divisi rari, e risolverli in fretta.

Esempio (un leader soltanto lento). S_1 è il leader del term 1, ma per un rallentamento della rete i suoi heartbeat smettono di arrivare: in un sistema asincrono un leader lento sembra guasto (Sezione 9.2).

1. *Un'elezione inutile.* Il timeout di S_2 scade, e S_2 vince il term 2 con i voti di S_3 e S_4. S_1 non ne sa niente, e si crede ancora leader.

2. *Il vecchio leader non ha più una maggioranza.* I suoi AppendEntries portano il term 1, e S_2, S_3 e S_4, che sono al term 2, li rifiutano. Gli resta al più S_5: con lui sono due server su cinque, troppo pochi per decidere qualcosa.
3. *Il vecchio leader si fa da parte.* Nelle risposte legge il term 2, più alto del suo: lo adotta e torna follower.

Anche in Raft, quindi, due server possono credersi leader nello stesso momento, come nella Sezione 9.2, ma in term diversi, e quello del term vecchio non trova più una maggioranza disposta ad ascoltarlo. Il term fa la parte della sbarra della Sezione 8.5: si sposta solo in avanti, e chi ha visto il term 2 rifiuta tutto ciò che porta un term più basso. La Figura 11.3 mostra tutti i passaggi tra i tre stati.

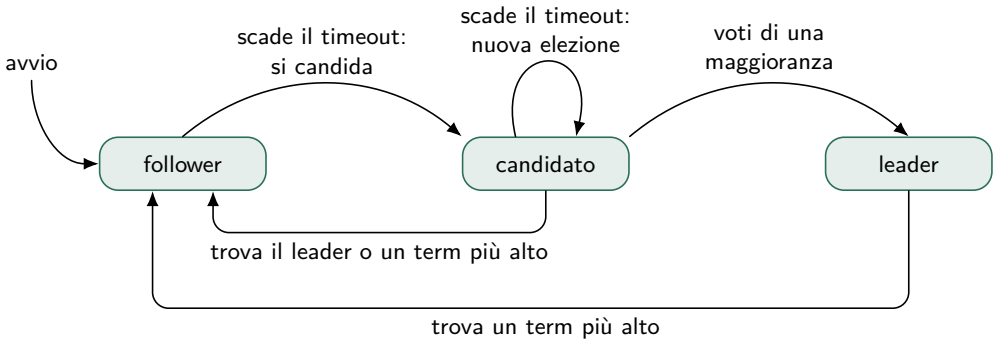


Figura 11.3. Gli stati di un server e i passaggi tra loro. Ogni server parte come follower.

Quanto deve durare il timeout. Il timeout di elezione va scelto tra due tempi:

$$broadcastTime \ll electionTimeout \ll MTBF,$$

dove $\ll$ vuol dire «molto più piccolo di». *broadcastTime* (tempo di trasmissione) è il tempo medio che serve a un server per inviare una RPC a tutti e ricevere le risposte, *electionTimeout* è il timeout di elezione, e *MTBF* (*mean time between failures*) è il tempo medio tra due guasti di un server. Il timeout deve essere molto più lungo di un giro di messaggi, altrimenti scade mentre gli heartbeat sono ancora in viaggio, e i follower avviano elezioni inutili anche con un leader che funziona; in più, un candidato non farebbe in tempo a raccogliere i voti prima che scadano i timeout degli altri, e i voti divisi sarebbero frequenti. E deve essere molto più corto del tempo tra due guasti, perché a ogni guasto del leader il sistema resta fermo per circa un timeout. Un giro di messaggi dura da meno di un millisecondo a qualche decina, il timeout si sceglie tra le decine e le centinaia di millisecondi, e un server si guasta in media dopo mesi.

I tempi contano solo per la liveness. Con timeout sbagliati Raft può fare elezioni inutili, o restare senza leader per un po', ma non elegge mai due leader nello stesso term. Il risultato FLP (Sezione 8.7) vale anche qui: in un sistema asincrono nessuno garantisce che prima o poi si elegga un leader, ma con timeout casuali e ben scelti succede quasi sempre, e in fretta.

11.3 Replicare il log

Una volta eletto, il leader riceve le richieste dei client: ogni richiesta è un'operazione da applicare a tutte le copie, come una rinomina nella cartella. Il leader la aggiunge in fondo al suo **log** (registro), la sequenza delle operazioni, e la copia sugli altri server. Ogni elemento del log è un'***entry*** (voce) e contiene due cose: l'operazione e il term in cui il leader l'ha ricevuta. La posizione di un'entry nel log è il suo **indice**, che fa la parte dell'istanza di Multi-Paxos (Sezione 9.1). Le posizioni però si riempiono una dopo l'altra, senza buchi, e le assegna solo il leader: se le rinomine di Anna e di Bruno arrivano insieme, il leader le mette in coda nell'ordine in cui le riceve, e non c'è nessuna gara.

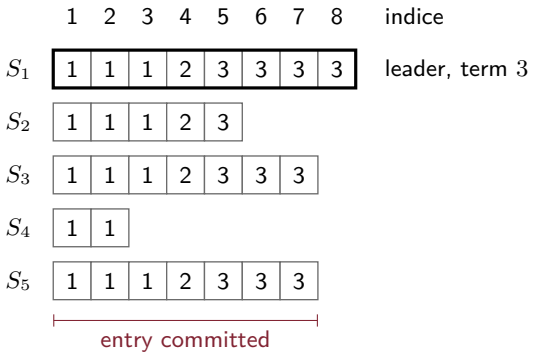


Figura 11.4. I log dei cinque server quando il leader S_1, nel term 3, ha appena aggiunto l'entry 8. Ogni casella è un'entry, con il term in cui è stata creata; in alto gli indici. Le entry fino alla 7 stanno su almeno tre server, e sono committed.

Un'entry è ***committed*** (definitiva) quando è sicuro applicarla alle copie: da quel momento non verrà più cancellata, e prima o poi tutti i server la applicheranno. Lo decide il leader, con le regole seguenti.

Definizione (replicazione del log).

1. *Richiesta.* Il client invia l'operazione al leader, che aggiunge in fondo al suo log un'entry con l'operazione e il suo *currentTerm*, e invia a tutti un AppendEntries con l'entry nuova.
2. *Copia.* Un follower aggiunge l'entry al suo log, la salva in memoria stabile e solo allora risponde sì al leader: un sì vuol dire «ce l'ho, e non la perdo nemmeno se mi guasto».
3. *Commit.* Il leader conta le copie solo delle entry che ha creato nel suo term. Ogni sì gli dice fin dove il log di quel follower ha le sue stesse entry, e lui lo annota. Quando un'entry del suo term sta su una maggioranza, contando anche il suo log, è committed, e con lei tutte le entry precedenti. Il leader tiene in *commitIndex* (*commit index*, indice di commit) l'indice dell'ultima entry che sa committed, applica alla sua copia le entry fino a quella e risponde al client.

4. *Notizia ai follower.* Il leader mette il suo *commitIndex* in ogni AppendEntries successivo, heartbeat compresi, e ogni follower applica, in ordine, le entry che ha fino a quell'indice.
5. *Insistenza.* Il leader rimanda gli AppendEntries a chi non risponde, anche dopo aver risposto al client, finché tutti i follower hanno tutte le entry.

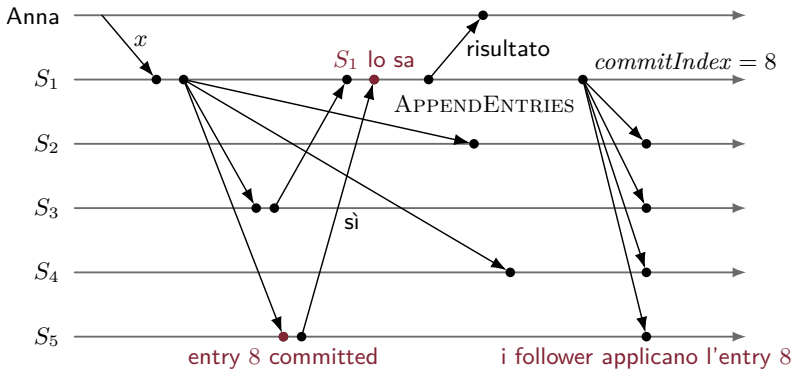


Figura 11.5. Un'operazione a regime, con i log della Figura 11.4. S_2 e S_4, rimasti indietro, ricevono l'AppendEntries più tardi, insieme alle entry che mancano loro.

Esempio (un'operazione a regime). Il leader è S_1, nel term 3, e i log sono quelli della Figura 11.4. Anna chiede di rinominare `Relazione.pdf` in `Tesi.pdf` (il valore x), e la Figura 11.5 segue la sua richiesta.

1. *Il leader aggiunge l'entry.* x diventa l'entry 8 del log di S_1, con il term 3, e S_1 invia AppendEntries a tutti.
2. *I follower la copiano.* S_3 e S_5 hanno già le entry fino alla 7: aggiungono la 8 e rispondono sì. S_2 e S_4 sono rimasti indietro, e il leader invia loro anche le entry che mancano, nello stesso AppendEntries: restano entry distinte, ognuna con il suo indice. Intanto non li aspetta.
3. *L'entry è committed, ma nessuno lo sa.* Quando S_5 la salva, l'entry 8 sta su tre server, S_1, S_3 e S_5: è committed. S_5, però, sa solo di averla salvata lui. Committed non vuol dire saputo, come scelto nella Sezione 8.3.
4. *Il leader lo sa dalle risposte.* S_1 sapeva già che i log di S_3 e di S_5 avevano le sue entry fino alla 7, e i loro sì gli dicono che ora arrivano all'8. Con il suo log fanno tre su cinque: S_1 pone $commitIndex = 8$, applica x alla sua copia e risponde ad Anna. Applica prima di rispondere perché la risposta contiene il risultato dell'operazione, per esempio «rinomina riuscita», che la macchina a stati calcola applicandola: quando Anna lo riceve, la rinomina è già committed e applicata.
5. *I follower lo sanno dal messaggio successivo.* Il prossimo AppendEntries, anche un semplice heartbeat, porta $commitIndex = 8$: i follower applicano

> l'entry 8, e anche le loro copie mostrano `Tesi.pdf`. Fino a quel momento la copia del leader è avanti di un'entry, ma è un ritardo, non un disaccordo: tutti applicano le stesse entry nello stesso ordine, e lo stato dei follower è obsoleto, non inconsistente (Sezione 1.1), come quello del learner in ritardo della Sezione 8.4. Nessun client se ne accorge, perché i client parlano solo con il leader: se Bruno chiede il nome del file, gli risponde S_1, che mostra già `Tesi.pdf`.

Nel caso normale, quindi, a un'operazione basta un giro di messaggi tra il leader e una maggioranza, e un follower lento, come S_2 o S_4, non rallenta nessuno. Le risposte vanno solo al leader, che fa sapere il commit agli altri con il messaggio successivo; in Multi-Paxos, invece, ogni acceptor mandava un LEARN a tutti i learner (Figura 9.1).

Log uguali fino in fondo. Il leader invia a un follower solo le entry nuove: come fa a sapere che il resto del log del follower è uguale al suo? Raft garantisce la proprietà ***Log Matching*** (log che combaciano): se due log hanno un'entry con lo stesso indice e lo stesso term,

- le due entry contengono la stessa operazione;
- i due log sono uguali in tutte le entry precedenti.

La prima parte vale perché in un term c'è un solo leader, che crea al più un'entry per ogni indice, e un'entry non cambia mai posizione. La seconda la garantisce un controllo. Ogni AppendEntries porta l'indice e il term dell'entry che precede quelle nuove (*prevLogIndex* e *prevLogTerm*, da *previous log index* e *previous log term*), e il follower accetta le entry nuove solo se nel suo log, a quell'indice, c'è un'entry di quel term; altrimenti risponde no. È il **controllo di coerenza** (*consistency check*), e funziona come una catena, in cui ogni anello si aggancia solo a quello giusto. All'inizio i log sono vuoti, quindi uguali; ogni AppendEntries accettato allunga un log uguale a quello del leader fino all'entry precedente, e lo lascia uguale fino all'ultima entry nuova. Nell'esempio, il messaggio con l'entry 8 porta l'indice 7 e il term 3: S_3 e S_5 hanno l'entry 7 del term 3, e accettano. Quando un follower risponde sì, quindi, il leader sa che il suo log è uguale al proprio fino all'ultima entry inviata: per questo un solo sì vale anche per tutte le entry precedenti, come quello di S_2, a cui il leader ha mandato insieme la 6, la 7 e l'8.

Log che divergono. Finché il leader funziona, i log dei follower restano uguali al suo, e il controllo non fallisce mai. Un leader, però, può guastarsi dopo aver copiato una sua entry solo su alcuni server, e a ogni guasto le differenze si accumulano. La Figura 11.6 mostra che cosa può trovare il leader del term 8 quando viene eletto: a un follower possono mancare entry, come in (a) e (b), può avere entry in più che non sono committed, come in (c) e (d), o tutte e due le cose, come in (e) e (f). Il caso (f), per esempio, è un server che è stato il leader del term 2, ha aggiunto alcune entry al suo log e si è guastato prima di renderle committed; è ripartito subito, ha vinto il term 3, ha aggiunto altre entry e si è guastato di nuovo, restando fermo per diversi term.

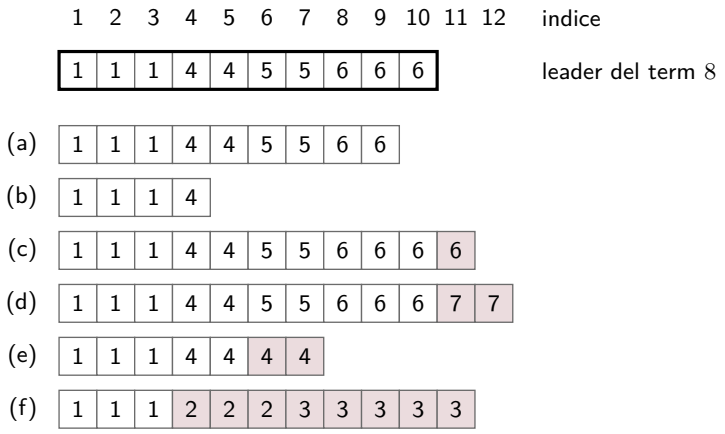


Figura 11.6. I log che il leader del term 8 può trovare nei follower quando viene eletto: ognuno dei casi (a)–(f) è possibile. In rosso chiaro le entry diverse da quelle del leader nella stessa posizione, o in più.

Il leader impone il suo log. Le differenze si risolvono sempre allo stesso modo: il follower copia il log del leader. Il leader cerca l'ultima entry su cui i due log concordano, il follower cancella tutto quello che viene dopo, e il leader gli invia le sue entry da lì in poi. Per farlo il leader tiene, per ogni follower, *nextIndex* (*next index*, indice successivo): l'indice della prossima entry da inviargli. Appena eletto lo pone, per tutti, subito dopo la sua ultima entry, quindi nella Figura 11.6 vale 11. A ogni no del controllo di coerenza il leader diminuisce di uno il *nextIndex* di quel follower e riprova; al primo sì il follower cancella le sue entry in conflitto e prende quelle del leader. La Tabella 11.1 segue il follower (e).

Tabella 11.1. Il leader del term 8 ripara il log del follower (e) della Figura 11.6. Ogni riga è un AppendEntries, con le entry dalla *nextIndex* in poi.

nextIndex	Entry precedente	Nel log di (e)	Risposta
11	indice 10, term 6	manca	no
10	indice 9, term 6	manca	no
9	indice 8, term 6	manca	no
8	indice 7, term 5	term 4	no
7	indice 6, term 5	term 4	no
6	indice 5, term 4	term 4	sì: cancella la sua 6 e la sua 7, e copia dalla 6 alla 10

I primi tre no vengono da entry che a (e) mancano, i due successivi da entry che ha, ma con un term diverso: le sue entry 6 e 7 vengono da un leader del term 4 che non le ha mai rese committed. Si può risparmiare qualche tentativo: con il no il follower indica il term dell'entry in conflitto e la prima entry che ha di quel term, e il leader salta tutte quelle entry in una volta. Il leader, invece, non cancella e non sovrascrive mai niente nel proprio log, aggiunge soltanto: è la proprietà ***Leader Append-Only*** (il leader aggiunge soltanto). Cancellare le entry dei follower è sicuro solo se non sono committed, e questo dipende da chi viene eletto leader.

Guasti dei follower. Se un follower o un candidato si guasta, le RPC che gli arrivano restano senza risposta, e chi le ha inviate le rimanda finché il server non riparte. Se un server si guasta dopo aver eseguito una RPC, ma prima di rispondere, alla ripartenza la riceve di nuovo. Non fa danni, perché le RPC sono **idempotenti**: eseguirle due volte equivale a eseguirle una. Per esempio, un follower che riceve in un AppendEntries entry che ha già le ignora, come l'acceptor della riga 6 della Tabella 8.2 ignora un doppione.

11.4 Chi può diventare leader

Il leader impone il suo log a tutti, quindi conta molto chi viene eletto. Con le regole viste finora può vincere anche un server rimasto indietro.

> **Esempio (un server rimasto indietro).** Il leader è S_1, nel term 2, e tutti i server hanno le entry fino alla 7.
>
> 1. S_5 *si ferma per un guasto.* Intanto S_1 aggiunge le entry 8 e 9, la rinomina di Anna e un'altra operazione, e le copia su S_2 e S_3: con tre server su cinque sono committed, e la cartella mostra `Tesi.pdf`. S_4, lento, ha ancora solo le entry fino alla 7.
> 2. S_5 *riparte, e* S_1 *si guasta.* Il timeout di S_5 scade per primo: S_5 passa al term 3 e chiede i voti.
> 3. *Se vincesse,* imporrebbe il suo log, che si ferma alla 7: gli altri cancellerebbero le entry 8 e 9, già committed e già applicate, e in quelle posizioni finirebbero operazioni diverse. Copie diverse applicherebbero operazioni diverse allo stesso indice: la cartella mostrerebbe `Tesi.pdf` su alcuni server e un altro nome su altri.

Bisogna quindi impedire che vinca un candidato a cui manca un'entry committed. Paxos risolveva lo stesso problema nel verso opposto: il nuovo coordinatore faceva l'inventario (Sezione 8.4), e dalle promesse ritrovava i voti che gli mancavano; in Multi-Paxos lo faceva per tutte le istanze aperte con un solo prepare (Sezione 9.3). I dati andavano quindi anche verso il leader. In Raft no: le entry vanno solo dal leader ai follower, e il compito di fermare chi è indietro passa al voto. Così vale la proprietà ***Leader Completeness*** (completezza del leader): se un'entry è committed in un term, si trova nel log dei leader di tutti i term successivi.

Votare solo chi è aggiornato. Ogni RequestVote porta anche l'indice e il term dell'ultima entry del candidato (*lastLogIndex* e *lastLogTerm*, da *last log index* e *last log term*). Alla regola del voto si aggiunge una condizione: un server vota il candidato solo se il log del candidato è aggiornato almeno quanto il suo. Per dire quale di due log è **più aggiornato** si confrontano le loro ultime entry:

- se hanno term diversi, è più aggiornato il log con il term più alto;
- se hanno lo stesso term, è più aggiornato il log più lungo.

La Tabella 11.2 mostra tre casi.

Tabella 11.2. Tre richieste di voto. Di ogni log si indica l'ultima entry, con il suo term e il suo indice.

Candidato	Votante	Più aggiornato	Voto
term 3, indice 7	term 2, indice 9	il candidato: term più alto	sì
term 2, indice 9	term 3, indice 7	il votante: term più alto	no
term 3, indice 7	term 3, indice 8	il votante: stesso term, più lungo	no

Il term conta prima della lunghezza perché un'entry di un term più alto l'ha creata un leader più recente, che, per questa stessa regola, aveva già tutte le entry committed prima di lui. La lunghezza, da sola, non dice niente: il follower (f) della Figura 11.6 ha undici entry, una più del leader del term 8, ma la sua ultima è del term 3, contro il 6 del leader. Il suo log è meno aggiornato, e infatti le sue entry dalla 4 in poi non sono committed.

Nell'esempio, l'ultima entry di S_5 è la 7, del term 2, mentre S_2 e S_3 hanno la 9, dello stesso term: i loro log sono più aggiornati, e gli negano il voto. S_5 può avere al più il voto di S_4, e con il suo arriva a due su cinque: non vince. Il prossimo leader sarà S_2 o S_3, che hanno le entry 8 e 9. Che basti lo dice ancora il testimone della Sezione 8.2: un'entry committed sta su una maggioranza, un candidato ha bisogno dei voti di una maggioranza, quindi almeno un votante ha l'entry, e non vota un candidato meno aggiornato di lui.

L'elezione fa quindi il lavoro che in Multi-Paxos facevano PREPARE e PROMISE, una volta per leader (Sezione 9.1): chiude i term vecchi, come la promessa chiudeva i round minori, ma senza l'inventario, perché il nuovo leader ha già tutto quello che gli serve.

11.5 Le entry dei term precedenti

Un leader appena eletto può avere nel log entry dei term precedenti senza sapere se sono committed: il leader che le ha create può essersi guastato prima di copiarle su una maggioranza, o dopo, senza averlo fatto sapere a nessuno. Il nuovo leader le copia sugli altri insieme alle sue. Verrebbe da considerarle committed appena stanno su una maggioranza, ma è sbagliato, come mostra la Figura 11.7.

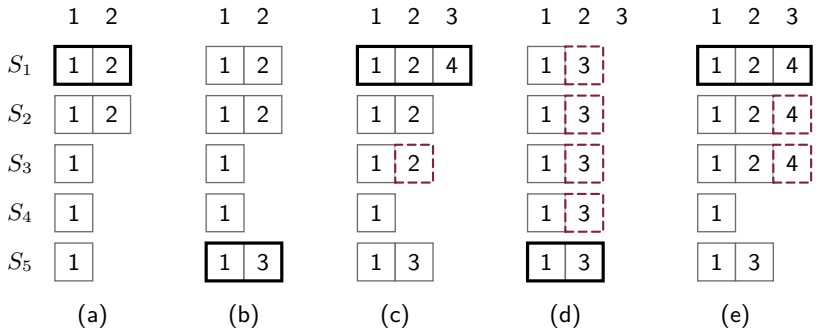


Figura 11.7. I log dei cinque server in cinque momenti. Il log del leader ha il contorno spesso, le entry appena copiate il bordo rosso tratteggiato.

Esempio (un'entry su una maggioranza che va persa). Nella figura l'entry 2 del term 2 è la rinomina di Anna, quella del term 3 la rinomina di Bruno, e l'entry 3 del term 4 una terza operazione.

(a) S_1 *è il leader del term* 2 e copia la sua entry 2 solo su S_2: due server su cinque, non è committed.

(b) S_1 *si guasta.* S_5 vince il term 3 con i voti di S_3, S_4 e il suo: i loro log finiscono tutti con l'entry 1, quindi il suo è aggiornato quanto i loro. S_5 mette nella posizione 2 la rinomina di Bruno, e si guasta prima di copiarla.

(c) S_1 *riparte e vince il term* 4. Copia l'entry 2 su S_3, e aggiunge al suo log una nuova entry, la 3. Ora l'entry 2 del term 2 sta su tre server su cinque. S_1 è lo stesso server che l'ha creata, ma qui è il leader del term 4: un leader è un server in un term, e l'entry 2 non è del suo term.

(d) *Eppure non è committed.* Se ora S_1 si guasta, S_5 può vincere il term 5 con i voti di S_2, S_3 e S_4: la sua ultima entry è del term 3, più alto del term 2 con cui finiscono i log di S_2 e S_3, quindi il suo log è più aggiornato. S_5 impone il suo log, e la rinomina di Bruno prende il posto di quella di Anna ovunque, anche su S_1 quando riparte. Se S_1 avesse considerato committed l'entry di Anna e l'avesse applicata, la sua copia mostrerebbe `Tesi.pdf` e le altre `Bozza.pdf`.

(e) *Con un'entry del term corrente le cose cambiano.* Se invece, prima di guastarsi, S_1 copia su una maggioranza la sua entry 3, del term 4, S_5 non può più vincere: i log di S_1, S_2 e S_3 finiscono con un'entry del term 4, più aggiornata della sua, e gli negano il voto. L'entry 3 è committed, e con lei l'entry 2 che la precede.

Per questo la regola del commit della Sezione 11.3 conta le copie solo delle entry del term corrente. Un'entry di un term precedente non diventa committed perché sta su una maggioranza, ma solo di riflesso, quando lo diventa un'entry successiva del term corrente, come al passo (e): per il Log Matching chi ha quell'entry ha anche tutte le precedenti uguali alle sue.

Il motivo sta nel numero scritto nell'entry. In Paxos un voto porta il round in cui è stato dato: quando il coordinatore del round 4 ripropone un valore votato nel round 2, gli acceptor lo votano di nuovo, e il loro voto porta il round 4 (Sezione 8.3); un inventario successivo lo vede come un voto recente, più di un voto del round 3. In Raft, invece, un'entry tiene per sempre il term in cui il leader l'ha creata, così è la stessa in ogni log e in ogni momento, ed è più facile ragionarci. La copia che S_1 fa su S_3 nel term 4, però, porta ancora scritto term 2, e nel passo (d) il confronto tra i log vede solo quel 2, che perde contro il 3 di S_5. Le tre copie del passo (c), del resto, sono state fatte in term diversi, due nel term 2 e una nel term 4, e voti di tentativi diversi non si sommano, come i LEARN di round diversi nella Sezione 8.5. Le copie di un'entry del term corrente, invece, sono fatte tutte nel term corrente, dal suo leader: contarle è come contare i voti di un solo round.

Finché un leader nuovo non rende committed un'entry del suo term, quindi, le entry dei term precedenti restano in sospeso. Per questo un leader appena eletto

aggiunge di solito al log un'entry *no-op*, che non cambia niente, come nella Sezione 9.3: appena è committed, lo sono anche tutte le precedenti.

Raft arriva così allo stesso risultato di Multi-Paxos per un'altra strada, e la Tabella 11.3 mette a confronto le due.

Tabella 11.3. Multi-Paxos e Raft a confronto.

	Multi-Paxos	Raft
Chi sceglie il leader	un algoritmo a parte (Sezione 9.2)	una votazione, dentro il protocollo
Numeri dei tentativi	round divisi tra i proposer	term consecutivi; decide il voto
Il leader nuovo	fa l'inventario delle istanze aperte	ha già tutte le entry committed
Dove vanno i dati	anche verso il leader, nelle promesse	solo dal leader ai follower
Le posizioni	decise ognuna per conto suo; i buchi diventano no-op	committed in ordine, senza buchi
Il numero scritto	nel voto: il round in cui è dato	nell'entry: il term in cui è nata

11.6 Perché Raft è sicuro

Resta da verificare che tutte le copie applichino le stesse operazioni nello stesso ordine. La proprietà da dimostrare è la ***State Machine Safety*** (sicurezza della macchina a stati): se un server ha applicato alla sua copia l'entry di un certo indice, nessun server applicherà mai un'entry diversa allo stesso indice. Ci si arriva dalla Leader Completeness della Sezione 11.4: un'entry committed si trova nel log di tutti i leader successivi.

La Leader Completeness si verifica per assurdo, su un caso con i numeri. S_1, leader del term 2, rende committed un'entry del suo term, l'entry 4, copiandola su S_2 e S_3. Supponiamo che S_5, leader del term 5, sia il primo leader dopo il term 2 a non avere l'entry 4: i leader dei term 3 e 4 ce l'hanno (Figura 11.8).

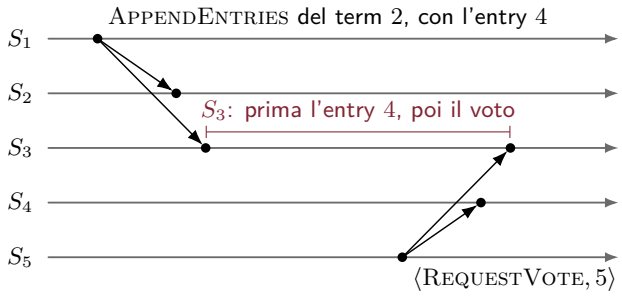


Figura 11.8. Il caso supposto per assurdo: S_1, leader del term 2, copia l'entry 4 su S_2 e su S_3; più tardi S_5 vince il term 5 con i voti di S_3, S_4 e il suo. S_3 sta in entrambe le maggioranze.

1. *S_5 non aveva l'entry 4 quando è stato eletto.* Un leader non cancella mai entry dal suo log (Leader Append-Only): se non ce l'ha adesso, non ce l'aveva nemmeno allora.
2. *C'è un testimone.* L'entry 4 sta su S_1, S_2 e S_3, e S_5 ha avuto i voti di S_3, S_4 e il suo. Le due maggioranze hanno in comune S_3, come sempre due maggioranze (Sezione 8.2).
3. *S_3 aveva l'entry 4 quando ha votato.* L'ha ricevuta prima di votare: dopo aver votato nel term 5, avrebbe rifiutato l'AppendEntries del term 2. E non l'ha persa nel frattempo, perché un follower cancella solo le entry in conflitto con quelle del leader, e i leader dei term 3 e 4 hanno l'entry 4.
4. *S_3 ha votato S_5, quindi il log di S_5 era aggiornato almeno quanto il suo.* I casi sono due, e in entrambi S_5 ha l'entry 4.
   - *Le due ultime entry hanno lo stesso term.* Allora il log di S_5 è lungo almeno quanto quello di S_3. Le due ultime entry, dello stesso term, le ha create lo stesso leader, e per il Log Matching entrambi i log coincidono con il suo fin dove arrivano: quello di S_5, che arriva almeno fin dove arriva quello di S_3, contiene tutte le sue entry, compresa la 4.
   - *L'ultima entry di S_5 ha un term più alto.* L'ultima entry di S_3 è di un term almeno 2, perché S_3 ha l'entry 4 del term 2; quella di S_5, quando è stato eletto, era di un term minore di 5. È quindi del term 3 o 4, e l'ha creata il leader di quel term, in fondo a un log che conteneva già l'entry 4: per il Log Matching il log di S_5 coincide con il suo fino a lì, e contiene anche lui l'entry 4.
5. *Contraddizione.* In entrambi i casi S_5 ha l'entry 4, contro l'ipotesi: un leader a cui manca un'entry committed non può esistere. Lo stesso vale per le entry committed di riflesso, come l'entry 2 del passo (e) della Figura 11.7: chi ha l'entry committed che le segue ha anche loro, per il Log Matching.

Da qui segue la State Machine Safety. Un server applica un'entry solo quando è committed, e il suo log, fino a quell'entry, è uguale a quello del leader, che lo ha verificato con il controllo di coerenza. Tutti i leader successivi hanno la stessa entry allo stesso indice, e i follower prendono le entry solo dal leader: chiunque applichi

quell'indice, in qualunque term, applica la stessa entry. E poiché ogni server applica le entry in ordine di indice, tutte le copie applicano le stesse operazioni nello stesso ordine: la cartella mostra a tutti gli stessi nomi.

Come per Paxos (Sezione 8.6), il ragionamento non usa mai i tempi né il numero dei guasti: con timeout sbagliati o troppi server guasti Raft può fermarsi, ma non applica mai due operazioni diverse allo stesso indice.